

CONFIDENTIAL · INTERNAL

---

## Product Design Specification

---

Draft v1.0 · May 2026

# 01 — Overview

---

## Vision

### Conversational AI for any application

The **AI Chat Platform** enables any application to give its users a persistent, context-aware conversational interface — without building AI infrastructure. The experience is modelled on the best native AI desktop applications: rich rendered output, transparent tool usage, and a conversation that remembers where you left off. It is not a general-purpose assistant layer — each deployment is a **specialist** tuned to its host application's domain.

**Governing intent:** Give any application team the ability to drop a production-grade AI assistant into their product within days — fully branded, scoped to their domain, connected to their data, and backed by a complete audit trail.

### What the platform is and is not

| It is                                                                                         | It is not                                                                                 |
|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| A white-label assistant layer any application can embed as a web component                    | A standalone AI product with its own brand or identity                                    |
| A multi-tenant platform where each host application brings its own scope, tools, and branding | A shared assistant all host applications configure from a single pool                     |
| A complete audit trail for every conversation turn and artefact                               | A transient chat tool with no persistent record                                           |
| A controlled, host-configured MCP integration surface                                         | An open API that allows arbitrary tool connections without host approval                  |
| A read-and-write assistant — queries, reasoning, and actions via host-registered MCP tools    | A system that bypasses host application security models or acts without user confirmation |

### The assistant has no platform name

The platform has no end-user-facing name. Every tenant names their own assistant in their application config ( `identity.assistantName` ). End users see only that name — they are not exposed to the underlying platform. This document uses **[AssistantName]** as a placeholder wherever the assistant name would appear.

---

## Scope

### In scope

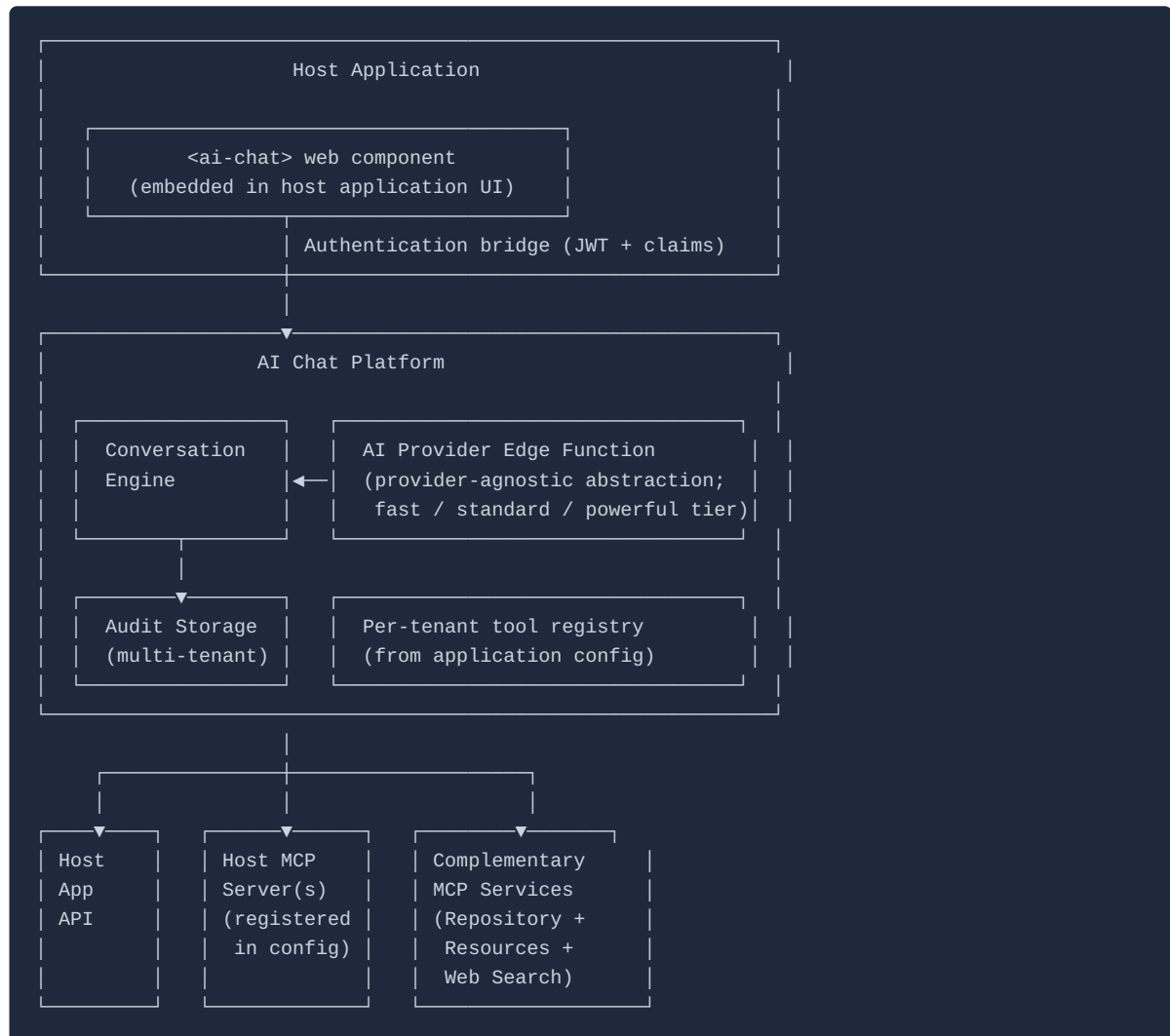
- Embeddable `<ai-chat>` web component with full branding token support
- Multi-tenant architecture with complete per-tenant data isolation
- Persistent, named conversation threads scoped to the authenticated user within their tenant
- Conversation branching via message edit or regeneration — preserving the complete audit trail
- Full-text search across all user conversations within the tenant
- Host-configured `@`-binding for application-defined object types
- Host-configured Display ID pattern detection and auto-resolution
- Document attachments (PDF, Excel, Word, images) stored as platform artefacts
- Model switching within the host-configured allowed model set; provider-agnostic architecture
- Communication style and verbosity driven by host-provided user profile claims
- Host-registered MCP servers with always-on and opt-in access tiers
- Host-defined guided workflow prompts accessible from the Workflow Library panel
- Tool call transparency — every MCP invocation rendered as a collapsible disclosure card
- Write operations — actions via host MCP tools with explicit user confirmation before execution
- Shared conversations — up to ten participants within the same tenant, equal-participant model
- Personal memory (user-managed) and application context (Application Admin-managed)
- Session artefact tray accumulating all input and output artefacts
- Document canvas — iterable working-document surface in the conversation panel; versioned, editable, model-revisable
- Three-zone responsive layout (history panel, conversation area, conversation panel) embedded within host app UI
- Rich content rendering: Mermaid, Vega-Lite, JSON inspector, data tables, syntax-highlighted code, prose markdown, math (KaTeX)
- Continuous improvement signal capture and per-tenant improvement issue pipeline
- Full audit trail per turn: raw prompt, resolved prompt, tool call log, output artefacts, token counts
- Complementary MCP ecosystem services: MCP Repository (tool discovery), MCP Resources (shared skills and artefacts), and Web Search (real-time web search)

### Out of scope

- Semantic search over application data (pgvector RAG) — structured MCP tool calls are the primary data access pattern in v1
- Conversation export (PDF or markdown) — planned; data classification complexity must be resolved first
- Voice or multimodal input
- Context globbing (pulling context from multiple past conversations into one session)
- Incognito or temporary chat (no-history mode) — conflicts with audit completeness
- Platform-owned web search — available via the Web Search complementary MCP service registered by the host (see [17-complementary-mcp-services.md](#))

- Image or content generation
- Code execution sandbox
- Public shareable links — all sharing is participant-controlled and tenant-scoped
- Read receipts in shared conversations
- Dark mode — not in v1

## Platform architecture



## Components

**AI Chat Platform** owns the conversational surface, content rendering, audit trail, shared conversation model, memory management, and improvement signal pipeline. It depends on, but does not duplicate, the capabilities of the host application or its MCP servers.

**Host Application** owns the user identity model, application business logic, and MCP server endpoints. It embeds the platform via the web component and passes user context via the authentication bridge.

**Host MCP Servers** are the host application's data and action providers. The platform routes tool calls to these servers and surfaces the results transparently in the conversation thread.

**Complementary MCP Services** — the MCP Repository, MCP Resources Service, and Web Search Service — are ecosystem-level services that operate alongside both the platform and host MCP servers. They are not owned by the platform or by individual host applications. See [17-complementary-mcp-services.md](#).

## Dependencies

| Dependency             | Role                                                                                                                                                                                                                                                                               |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AI provider            | Provider-agnostic abstraction exposing three model tiers: <code>fast</code> , <code>standard</code> (default), <code>powerful</code> . The platform maps tiers to the tenant's configured provider's current models. Multiple providers planned — see <a href="#">ROADMAP.md</a> . |
| Platform storage       | Relational database with row-level security for conversation records; object storage for binary artefacts.                                                                                                                                                                         |
| Platform edge function | JWT handling, AI provider API request construction, SSE stream passthrough, MCP call routing. Provider-agnostic interface.                                                                                                                                                         |
| Host authentication    | The host application issues JWTs for its users. The platform validates these tokens and trusts the embedded claims. No re-authentication is performed by the platform.                                                                                                             |
| Host MCP server(s)     | The host's registered MCP endpoints providing data access and action capabilities.                                                                                                                                                                                                 |
| MCP Repository         | Complementary ecosystem service — discoverable registry of available MCP tools.                                                                                                                                                                                                    |
| MCP Resources Service  | Complementary ecosystem service — centralised skills, static resources, and reusable prompt artefacts.                                                                                                                                                                             |
| Web Search Service     | Complementary ecosystem service — real-time web search and page retrieval; registered by hosts as an opt-in or always-on MCP server.                                                                                                                                               |

## 00 — Host Application Configuration

---

Every tenant on the AI Chat Platform is defined by a **JSON application config** provided by the host application at registration time. The config is the single source of truth for how the assistant behaves within that application — its identity, scope, tools, bindings, workflows, branding, and feature flags.

---

### Config delivery

Configs are registered and updated via the **Platform Admin API** (authenticated by a platform API key scoped to the tenant). A web-based **Config Editor UI** provides a visual wrapper over the same API for non-technical Application Admins. Both target the same config document — there is no divergence between API and UI state.

Config takes effect on the **next new conversation session** after the update is applied. Active sessions are not interrupted by config changes.

---

### Config schema

```
{
  "$schema": "https://chat-platform.io/config/v1/schema.json",
  "version": "1.0.0",

  "identity":    { ... },
  "branding":    { ... },
  "scope":       { ... },
  "bindableTypes": [ ... ],
  "mcpServers":  [ ... ],
  "workflows":   [ ... ],
  "renderers":   [ ... ],
  "models":      { ... },
  "userProfile": { ... },
  "memory":      { ... },
  "conversations": { ... },
  "features":    { ... },
  "widget":      { ... }
}
```

## identity

```
{
  "identity": {
    "tenantId": "acme-corp",
    "applicationName": "Acme Data Hub",
    "assistantName": "Atlas",
    "assistantDescription": "Your data governance assistant for Acme Corp.",
    "supportUrl": "https://help.acme.com/atlas",
    "privacyPolicyUrl": "https://acme.com/privacy"
  }
}
```

| Field                             | Required | Description                                                             |
|-----------------------------------|----------|-------------------------------------------------------------------------|
| <code>tenantId</code>             | Yes      | Unique slug identifier for this tenant on the platform                  |
| <code>applicationName</code>      | Yes      | Name of the host application — shown in platform admin UI               |
| <code>assistantName</code>        | Yes      | The name end users see for the assistant (e.g. "Atlas", "Nova", "Sage") |
| <code>assistantDescription</code> | Yes      | One-sentence description shown in the onboarding welcome state          |
| <code>supportUrl</code>           | No       | URL linked from error states for end-user support                       |
| <code>privacyPolicyUrl</code>     | No       | URL displayed in the memory consent UI                                  |

## branding

Design tokens applied to the `<ai-chat>` web component. The platform's default design system is used for any token not provided.

```
{
  "branding": {
    "colorPrimary": "#0057B8",
    "colorSurface": "#FFFFFF",
    "colorSurfaceVariant": "#F4F6F9",
    "colorOnSurface": "#111827",
    "colorAccent": "#00A86B",
    "fontFamily": "Inter, system-ui, sans-serif",
    "borderRadius": "8px",
    "logoUrl": "https://cdn.acme.com/logo.svg",
    "faviconUrl": "https://cdn.acme.com/favicon.ico"
  }
}
```

All colour values accept hex, RGB, or HSL. Logo assets must be SVG or PNG and served from a CSP-compatible origin declared in the host's Content Security Policy. See [16-embedding-and-web-component.md](#) for CSP requirements.

## scope

Defines the assistant's domain, what it should decline, and how it handles out-of-scope queries.

```
{
  "scope": {
    "systemPrompt": "You are Atlas, the data governance assistant for Acme Corp. You help users find, understand, and act on data assets governed by the Acme Data Hub platform...",
    "inScopeDescription": "Data assets, governance policies, quality metrics, and data ownership within Acme Corp",
    "outOfScopeRedirect": "I'm here to help with Acme Corp's data governance. For general questions, please contact your team's relevant resource.",
    "language": "en"
  }
}
```

| Field                           | Required | Description                                                                                                                            |
|---------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------|
| <code>systemPrompt</code>       | Yes      | Base system prompt for the assistant. The platform appends tool descriptions, memory blocks, and platform-managed safety instructions. |
| <code>inScopeDescription</code> | Yes      | Plain-language description of scope; used in the onboarding state and out-of-scope redirect messages                                   |
| <code>outOfScopeRedirect</code> | No       | Custom message for out-of-scope queries. The platform uses a generic fallback if not set.                                              |
| <code>language</code>           | No       | BCP 47 language tag (default: <code>en</code> ). Applies to UI strings and the assistant's response language hint.                     |

## System prompt authoring guidelines

- Establish the assistant's persona and domain clearly in the first 200 tokens
- Do not include tool descriptions — they are injected automatically at session start
- Do not include memory blocks — injected automatically
- Do not attempt to override platform safety, audit, or tool-transparency instructions
- Keep under 3,000 tokens to leave headroom for injected context, memory blocks, and tool descriptions
- Avoid referencing specific MCP tool names — describe capabilities in natural language so the model chooses the right tool

## bindableTypes

Defines the object types that end users can reference via `@`-binding in the input field. The platform provides the `@`-binding mechanism; the host defines what can be bound.



```
{
  "bindableTypes": [
    {
      "id": "data_domain",
      "label": "Data Domain",
      "pluralLabel": "Data Domains",
      "icon": "database",
      "searchEndpoint": "https://api.acme.com/chat/search/domains",
      "contextTemplate": "Data Domain: {{name}} (ID: {{displayId}})\nOwner: {{owner}}\nDescription: {{description}}",
      "displayIdPattern": "DOM-[0-9]+",
      "rank": 1
    },
    {
      "id": "policy",
      "label": "Policy",
      "pluralLabel": "Policies",
      "icon": "shield",
      "searchEndpoint": "https://api.acme.com/chat/search/policies",
      "contextTemplate": "Policy: {{name}}\nStatus: {{status}}\nEffective: {{effectiveDate}}\nSummary: {{summary}}",
      "rank": 2
    }
  ]
}
```

| Field                         | Required | Description                                                                                                                                                                                                                                                                                                       |
|-------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>id</code>               | Yes      | Unique identifier for this type within the tenant                                                                                                                                                                                                                                                                 |
| <code>label</code>            | Yes      | Singular display label shown in the typeahead panel                                                                                                                                                                                                                                                               |
| <code>pluralLabel</code>      | Yes      | Plural label used in empty states and section headers                                                                                                                                                                                                                                                             |
| <code>icon</code>             | No       | Platform icon name or absolute URL for the chip icon                                                                                                                                                                                                                                                              |
| <code>searchEndpoint</code>   | Yes      | Host-provided HTTPS endpoint the platform calls to resolve @-binding searches. Must accept a <code>?q=</code> query parameter and return <code>{ items: [{ id, name, displayId?, inactive? }] }</code> . Called with the user's bearer token forwarded — the host endpoint enforces its own permission filtering. |
| <code>contextTemplate</code>  | Yes      | Mustache template for the context block injected into the model prompt when this binding is resolved. Use <code>{{fieldName}}</code> for any field returned by the search endpoint's item objects.                                                                                                                |
| <code>displayIdPattern</code> | No       | Regex; pasted text matching this pattern triggers an auto-lookup confirmation prompt                                                                                                                                                                                                                              |
| <code>rank</code>             | No       | Sort order in typeahead results (ascending). Defaults to alphabetical by label if omitted.                                                                                                                                                                                                                        |

## Permission model for bindable types

The `searchEndpoint` is called with the authenticated user's bearer token forwarded in the `Authorization` header. The host endpoint is responsible for filtering results to objects the user is

permitted to access. The platform does not enforce bindable-type permissions independently — it trusts the host's endpoint response.

In shared conversations, if a binding chip resolves to an object another participant cannot access (because that participant's search endpoint would not have returned it), the restricted participant sees the chip labelled **[Restricted object]**. See [08-shared-conversations.md](#).

mcpServers

Registers the MCP servers available within this tenant. The platform assumes no MCP servers by default — hosts must register at least one for the assistant to be useful beyond its system prompt knowledge.

Host teams can discover and browse available MCP servers in the **MCP Repository** (see [17-complementary-mcp-services.md](#)) before registering them here.

```
{
  "mcpServers": [
    {
      "id": "governance-mcp",
      "name": "Governance Platform",
      "description": "Provides access to data governance entities, quality metrics, ownership records, and policy objects. Use for any question about data assets, governance status, or compliance.",
      "endpoint": "https://api.acme.com/mcp/governance",
      "authType": "bearer",
      "accessTier": "always-on",
      "roles": []
    },
    {
      "id": "warehouse-mcp",
      "name": "Data Warehouse",
      "description": "Read-only access to the Acme data warehouse. Use to answer questions about actual data values, row counts, and sample records.",
      "endpoint": "https://api.acme.com/mcp/warehouse",
      "authType": "bearer",
      "accessTier": "opt-in",
      "roles": ["data_practitioner", "admin"]
    }
  ]
}
```

| Field       | Required | Description                                                                                                                                                |
|-------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| id          | Yes      | Unique identifier within the tenant                                                                                                                        |
| name        | Yes      | Display name shown in the tool selection panel                                                                                                             |
| description | Yes      | Plain-language description injected into the system prompt when the tool is active. Write it for the model — describe what it provides and when to use it. |
| endpoint    | Yes      | MCP-compliant HTTPS endpoint URL                                                                                                                           |

| Field                   | Required | Description                                                                                                                                                                                                      |
|-------------------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>authType</code>   | Yes      | <code>bearer</code> — the user's host JWT is forwarded; <code>api-key</code> — the platform holds a key per tenant (configured separately in the Admin API); <code>none</code> — no authentication               |
| <code>accessTier</code> | Yes      | <code>always-on</code> — active in every session, cannot be disabled by the end user; <code>opt-in</code> — off by default, user enables per session via the tool selection panel                                |
| <code>roles</code>      | No       | Array of host-defined role identifiers (from user profile claims). If populated, only users whose role claim includes one of these values can see or activate this server. Empty array = available to all users. |

A tenant may have multiple `always-on` servers. If no `always-on` servers are configured, the platform operates in **prompt-only mode** — the assistant answers from system prompt knowledge only, with no live tool access.

---

### workflows

Guided workflows are host-defined conversation starters that launch a structured multi-turn interaction. They appear in the **Workflow Library** panel and as optional starters on the onboarding screen.

```

{
  "workflows": [
    {
      "id": "governance-health-check",
      "name": "Governance Health Check",
      "description": "Summarise governance coverage, quality gaps, and policy compliance across a selected domain.",
      "icon": "activity",
      "prompt": "Run a governance health check for {{domain}}. Cover: (1) entity coverage, (2) data quality completeness, (3) ownership gaps, (4) classification compliance. Present as an executive summary with a traffic-light status for each area.",
      "parameters": [
        {
          "id": "domain",
          "label": "Data Domain",
          "type": "binding",
          "bindableTypeId": "data_domain",
          "required": true
        }
      ],
      "roles": []
    },
    {
      "id": "weekly-status",
      "name": "Weekly Status Report",
      "description": "Generate a plain-language weekly status summary for a selected team.",
      "icon": "calendar",
      "prompt": "Generate a weekly status report for {{team}}. Summarise open actions, recent completions, and any blockers.",
      "parameters": [
        {
          "id": "team",
          "label": "Team",
          "type": "text",
          "required": true
        }
      ],
      "roles": []
    }
  ]
}

```

| Field                    | Required | Description                                                                                                                                                                                                                                                                                                                             |
|--------------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>id</code>          | Yes      | Unique identifier within the tenant                                                                                                                                                                                                                                                                                                     |
| <code>name</code>        | Yes      | Display name in the Workflow Library                                                                                                                                                                                                                                                                                                    |
| <code>description</code> | Yes      | One-sentence description shown in the library and onboarding                                                                                                                                                                                                                                                                            |
| <code>icon</code>        | No       | Platform icon name or URL                                                                                                                                                                                                                                                                                                               |
| <code>prompt</code>      | Yes      | Mustache template for the prompt submitted on launch. References <code>parameters</code> by their <code>id</code> .                                                                                                                                                                                                                     |
| <code>parameters</code>  | No       | Parameters collected from the user before the workflow launches. Types: <code>binding</code> (opens an <code>@</code> -binding typeahead scoped to a <code>bindableTypeId</code> ), <code>text</code> (free text field), <code>select</code> (dropdown; include an <code>options</code> array of <code>{ value, label }</code> objects) |

| Field              | Required | Description                                                           |
|--------------------|----------|-----------------------------------------------------------------------|
| <code>roles</code> | No       | Role restriction. Empty array = available to all authenticated users. |

Workflow prompts are host-managed — users cannot create or modify them.

## renderers

Host applications may register **custom content renderers** that the platform loads at runtime. When the model produces a fenced code block tagged with a registered renderer's trigger, the platform invokes the host's renderer instead of its built-in pipeline. This enables domain-specific visualisations — risk gauges, org charts, Gantt views, financial waterfall charts, compliance scorecards — that the platform's built-in renderers do not cover.

See [10-content-rendering.md](#) for the full runtime rendering contract and system prompt guidance injection.

```
{
  "renderers": [
    {
      "id": "risk-gauge",
      "trigger": "risk-gauge",
      "name": "Risk Gauge",
      "description": "Interactive risk score gauge with traffic-light colouring",
      "moduleUrl": "https://cdn.acme.com/ai-renderers/risk-gauge.js",
      "exportName": "RiskGaugeRenderer",
      "systemPromptGuidance": "Use a ```risk-gauge block when asked for a risk score or risk assessment. The block must contain a JSON object: { \"score\": <0-100>, \"label\": \"<risk level label>\", \"breakdown\": [ { \"factor\": \"<name>\", \"score\": <0-100> } ] }. Only use this block when a numeric risk score is directly requested."
    }
  ]
}
```

| Field                    | Required | Description                                                                                                                                                                                                                                                                                                                                                   |
|--------------------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>id</code>          | Yes      | Unique identifier within the tenant                                                                                                                                                                                                                                                                                                                           |
| <code>trigger</code>     | Yes      | Fenced code block tag that activates this renderer (e.g. <code>risk-gauge</code> → <code>```risk-gauge</code> in the model output). Must be lowercase alphanumeric with hyphens; must not conflict with a built-in trigger ( <code>mermaid</code> , <code>vega-lite</code> , <code>math</code> , <code>json</code> , <code>csv</code> , <code>table</code> ). |
| <code>name</code>        | Yes      | Display name shown in the platform admin UI and in the content block header within the conversation thread                                                                                                                                                                                                                                                    |
| <code>description</code> | Yes      | One-sentence description shown in the admin UI. Not injected into the system prompt.                                                                                                                                                                                                                                                                          |
| <code>moduleUrl</code>   | Yes      | HTTPS URL to an ES module exporting the renderer. Must be served from an origin declared in the tenant's registered origins. Loaded once per session; cached for the session lifetime.                                                                                                                                                                        |

| Field                             | Required | Description                                                                                                                                                                                                                                                                                                                                                        |
|-----------------------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>exportName</code>           | No       | Named export from the module. Default: <code>"default"</code> .                                                                                                                                                                                                                                                                                                    |
| <code>systemPromptGuidance</code> | No       | Text injected verbatim into the system prompt to instruct the model when and how to produce this content type. Include the expected fenced block format and any content schema the renderer requires. If omitted, the platform injects a minimal generic instruction: <i>"You may produce <code>{trigger}</code> blocks using <code>{trigger}</code> fencing."</i> |

## Origin registration

The `moduleUrl` origin must be declared during tenant registration (or updated via the Platform Admin API). At config submission, the platform validates that the declared origin is reachable and that the module can be loaded. Modules served from unregistered origins are rejected. The host's CSP `script-src` must include the renderer origin — see [16-embedding-and-web-component.md](#).

### models

```
{
  "models": {
    "defaultModel": "standard",
    "allowedModels": ["fast", "standard", "powerful"],
    "userCanSwitch": true,
    "provider": "anthropic"
  }
}
```

| Field                      | Required | Description                                                                                                                                                                                                                 |
|----------------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>defaultModel</code>  | Yes      | Model tier used for new conversations unless the user switches. Accepted values: <code>"fast"</code> , <code>"standard"</code> , <code>"powerful"</code> . See <a href="#">06-model-configuration.md</a> for tier profiles. |
| <code>allowedModels</code> | Yes      | Array of model tiers users may switch to. Must include <code>defaultModel</code> .                                                                                                                                          |
| <code>userCanSwitch</code> | No       | Whether users can switch model tier mid-conversation (default: <code>true</code> ). Set to <code>false</code> to lock all sessions to <code>defaultModel</code> .                                                           |
| <code>provider</code>      | Yes      | AI provider identifier. The platform maps tier names to the provider's current models. Multiple providers planned — see <a href="#">ROADMAP.md</a> .                                                                        |

### userProfile

Describes how the platform should interpret user-profile claims for communication style personalisation. These claims are passed via the authentication bridge (see [16-embedding-and-web-component.md](#)).

```
{
  "userProfile": {
    "styleField": "communication_style",
    "styleValues": {
      "technical": "Respond with technical precision. Include field names, IDs, and system details.",
      "business": "Respond in plain business language. Avoid jargon. Use analogies where helpful.",
      "executive": "Respond concisely in three to five sentences. Lead with the conclusion. Omit process detail."
    },
    "verbosityField": "response_verbosity",
    "verbosityValues": {
      "concise": "Keep responses to the essential point. Tables preferred over prose lists.",
      "standard": "Provide sufficient context for the response to stand alone.",
      "detailed": "Include full explanation, examples, and supporting detail."
    }
  }
}
```

The `styleField` and `verbosityField` values must match claim keys passed in the user's JWT payload via the authentication bridge. If a user's JWT contains no matching claim, the platform uses its default tone (no style injection). If `userProfile` is omitted from the config entirely, style personalisation is disabled for the tenant.

---

## memory

Configures the memory system for this tenant. See [15-memory-and-recall.md](#) for the full behaviour specification.

```
{
  "memory": {
    "personalMemory": {
      "enabled": true,
      "defaultOn": false,
      "categories": ["Role", "Preference", "Correction", "Context"],
      "tokenBudget": 2000
    },
    "applicationContext": {
      "enabled": true,
      "categories": ["Terminology", "Policy", "Structure", "Domain context"],
      "tokenBudget": 4000,
      "approvalRequired": true
    }
  }
}
```

| Field                                       | Default                                                               | Description                                                                 |
|---------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>personalMemory.enabled</code>         | <code>true</code>                                                     | Enable the personal memory feature for this tenant                          |
| <code>personalMemory.defaultOn</code>       | <code>false</code>                                                    | Whether personal memory injection is on or off when a user first enables it |
| <code>personalMemory.categories</code>      | <code>["Role", "Preference", "Correction", "Context"]</code>          | Category labels for personal memory items (displayed in the management UI)  |
| <code>personalMemory.tokenBudget</code>     | <code>2000</code>                                                     | Maximum tokens for personal memory in the system prompt                     |
| <code>applicationContext.enabled</code>     | <code>true</code>                                                     | Enable Application Context (tenant-wide org memory)                         |
| <code>applicationContext.categories</code>  | <code>["Terminology", "Policy", "Structure", "Domain context"]</code> | Category labels for application context items                               |
| <code>applicationContext.tokenBudget</code> | <code>4000</code>                                                     | Maximum tokens for application context in the system prompt                 |



| Field                                            | Default           | Description                                                            |
|--------------------------------------------------|-------------------|------------------------------------------------------------------------|
| <code>applicationContext.approvalRequired</code> | <code>true</code> | Require a two-user approval workflow for new application context items |

#### conversations

```
{
  "conversations": {
    "maxTurnLimit":      100,
    "maxParticipants":   10,
    "retentionDays":     1095,
    "allowSharing":      true,
    "allowAttachments":  true,
    "maxAttachmentMbPerFile": 10,
    "maxAttachmentMbPerConversation": 100
  }
}
```

All fields have platform defaults (shown above). Hosts may lower limits but not exceed platform-level maximums.

#### features

Feature flags that enable or disable platform capabilities for this tenant.

```
{
  "features": {
    "atBinding": true,
    "displayIdDetection": true,
    "guidedWorkflows": true,
    "sharedConversations": true,
    "personalMemory": true,
    "applicationContext": true,
    "csatSurvey": true,
    "csatSampleRate": 0.20,
    "artefactTray": true,
    "starterWorkflows": ["governance-health-check"],
    "starterQuestions": [
      "What are the highest-priority issues right now?",
      "Show me anything that needs my attention.",
      "What does my team own and are there any gaps?"
    ]
  }
}
```

| Flag                             | Default           | Description                                                                                                                        |
|----------------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <code>atBinding</code>           | <code>true</code> | Enable <code>@</code> -binding typeahead. Requires at least one <code>bindableType</code> to be configured.                        |
| <code>displayIdDetection</code>  | <code>true</code> | Enable auto-detection of pasted Display IDs matching <code>displayIdPattern</code> values                                          |
| <code>guidedWorkflows</code>     | <code>true</code> | Enable the Workflow Library panel. Requires at least one <code>workflow</code> to be configured.                                   |
| <code>sharedConversations</code> | <code>true</code> | Enable conversation sharing with other users in the same tenant                                                                    |
| <code>personalMemory</code>      | <code>true</code> | Enable personal memory feature                                                                                                     |
| <code>applicationContext</code>  | <code>true</code> | Enable Application Context (managed by Application Admin)                                                                          |
| <code>csatSurvey</code>          | <code>true</code> | Enable post-session CSAT rating prompt                                                                                             |
| <code>csatSampleRate</code>      | <code>0.20</code> | Fraction of sessions shown the CSAT prompt (0.0–1.0)                                                                               |
| <code>artefactTray</code>        | <code>true</code> | Enable the session artefact tray panel                                                                                             |
| <code>starterWorkflows</code>    | <code>[]</code>   | Workflow IDs shown as suggested starters on the onboarding screen (references <code>workflows[].id</code> )                        |
| <code>starterQuestions</code>    | <code>[]</code>   | Plain-text starter questions shown on onboarding (up to 3, shown only if <code>starterWorkflows</code> is empty or alongside them) |

### widget

Configures the **Mode 1 floating widget** ( `<ai-chat-widget>` ) behaviour for this tenant. Applies only when the host embeds the floating widget web component — ignored for Mode 2 ( `<ai-chat>` ) and Mode 3 ( `<ai-chat-field>` ) deployments. See [18-entry-points-and-embedding-modes.md](#) for full widget behaviour.

```
{
  "widget": {
    "defaultState": "mini",
    "fabIcon": "chat-bubble",
    "showBadge": true,
    "returningUserThreshold": 14400
  }
}
```

| Field                               | Default                    | Description                                                                                                                                                                                                                                                                                                                      |
|-------------------------------------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>defaultState</code>           | <code>"mini"</code>        | Initial expansion state when the user opens the widget for the first time in a browser session. Accepted values: <code>"mini"</code> (compact panel, ~400 × 600 px) or <code>"full"</code> (full-panel overlay).                                                                                                                 |
| <code>fabIcon</code>                | <code>"chat-bubble"</code> | Icon displayed on the floating action button. Platform icon name or absolute URL to an SVG.                                                                                                                                                                                                                                      |
| <code>showBadge</code>              | <code>true</code>          | Whether to show an unread-indicator badge on the FAB when the assistant has produced a new response while the widget is collapsed.                                                                                                                                                                                               |
| <code>returningUserThreshold</code> | <code>14400</code>         | Idle time in seconds before a returning user's mini-widget shows a compact re-entry card (summarising the last conversation and offering quick actions) rather than the new-conversation home state. Default: 14 400 seconds (4 hours). Set to <code>0</code> to always show the re-entry card when a prior conversation exists. |

## Config versioning

The config document uses semantic versioning in the `version` field:

| Change type                                                             | Increment    | Example                                 |
|-------------------------------------------------------------------------|--------------|-----------------------------------------|
| Description or prompt text edits; branding token updates                | <b>Patch</b> | <code>1.0.0</code> → <code>1.0.1</code> |
| New bindable type, new MCP server, new workflow, new feature flag value | <b>Minor</b> | <code>1.0.0</code> → <code>1.1.0</code> |
| Structural schema changes; renamed or removed top-level fields          | <b>Major</b> | <code>1.0.0</code> → <code>2.0.0</code> |

The platform stores a full version history for each tenant's config. Rollback to any prior version is available via the Platform Admin API.

## Config validation

Submitting a config via the Admin API runs **synchronous validation** before applying:

| Check                                    | Description                                                                                                                                    |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Schema conformance                       | All required fields present; field types match schema                                                                                          |
| Endpoint reachability                    | MCP server endpoints and <code>searchEndpoint</code> URLs respond to a health check                                                            |
| System prompt token count                | Must be under 3,000 tokens                                                                                                                     |
| Model IDs in <code>allowedModels</code>  | Must be available in the platform for the configured <code>provider</code>                                                                     |
| <code>contextTemplate</code> syntax      | Mustache syntax check for all bindable type templates                                                                                          |
| <code>starterWorkflows</code> references | All IDs in <code>starterWorkflows</code> must match a <code>workflows[].id</code>                                                              |
| Renderer trigger uniqueness              | No two renderers may share the same <code>trigger</code> ; no renderer <code>trigger</code> may match a built-in tag                           |
| Renderer module reachability             | Each <code>moduleUrl</code> must respond to a HEAD request from the platform's validation origin; the origin must be registered for the tenant |

Validation errors return a structured response with field-level detail. The config is not applied until all validation checks pass.

## 02 — Personas and User Journeys

---

The AI Chat Platform is deployed by many different host applications across different domains. This document describes personas at two levels: **platform-level archetypes** (roles that exist across all deployments) and **illustrative host application journeys** (examples showing how different types of host app use the platform).

---

### Platform-level personas

These archetypes exist in every deployment regardless of the host application's domain.

| Persona           | Role                                                                                              | Primary need                                                                                                   |
|-------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| End User          | Authenticated user of the host application using the assistant as their primary access point      | Ask questions and get immediate, accurate answers about the host application's domain without leaving the page |
| Power User        | Experienced end user who regularly uses advanced features                                         | Multi-turn investigation, @-binding for precise object references, branching threads, tool call inspection     |
| Application Admin | Privileged user within the tenant responsible for the assistant's quality and application context | Manage application context (org-level memory), triage improvement signals, maintain workflow quality           |
| Host Developer    | Engineer at the host application team responsible for config and integration                      | Register MCP servers, maintain the application config, manage bindable types and guided workflows              |
| Platform Admin    | AI Chat Platform team member with cross-tenant visibility                                         | Platform health, tenant onboarding, infrastructure, improvement pipeline                                       |

### Application Admin design note

The Application Admin role is the platform's equivalent of a product owner for the assistant within the tenant. They are not necessarily a technical user. Key responsibilities:

- Managing application context (organisation-wide memory items that all users benefit from)
- Reviewing and approving application context items proposed by colleagues
- Triage improvement signals generated by end users within their tenant
- Updating the application config via the Config Editor UI (non-technical admins) or Admin API (technical admins)

This role must exist in every tenant before go-live. Host teams should designate at least one Application Admin at registration time.

---

## Illustrative host application journeys

These journeys are illustrative — they show how different types of host application use the platform. They are not prescriptive for any specific deployment.

---

### Journey A — Governance platform: executive morning briefing

**Host application type:** Data governance and compliance platform

**Persona:** Executive leader

**Setting:** Mobile, during commute (5 minutes)

The executive opens the assistant on their mobile. They type: *"Give me a governance health summary — anything that needs my attention this week."*

The assistant invokes the host's governance MCP server, retrieves domain health scores and open quality issues, and returns: - A Vega-Lite chart of domain health scores - A bullet summary of the three domains requiring attention, ranked by severity

The executive taps one of the domain binding chips in the response, which the host application has configured to navigate to the domain detail page. They follow up: *"Who owns the entities flagged in the Finance domain?"* — the assistant returns a table of owners with contact details.

Both artefacts appear in the session artefact tray.

**Features exercised:** Mobile layout, Vega-Lite rendering, @-binding chip click-through to host app, artefact tray, always-on MCP server.

---

### Journey B — Customer support platform: self-service knowledge lookup

**Host application type:** Internal customer support tooling

**Persona:** Support agent (End User)

**Setting:** Desktop, between calls

A support agent needs to know the current policy for handling a specific type of refund request. They type `@Refund Policy` — the @-binding typeahead opens and filters instantly. They select **Standard Refund Policy** from the panel; the binding chip `@{Standard Refund Policy}` is inserted. They submit:

*"Can I apply @{Standard Refund Policy} to orders over 90 days old, or does that need manager escalation?"*

The assistant retrieves the resolved policy context (host-configured `contextTemplate`) and answers directly: yes for up to 180 days; manager escalation required beyond that. No knowledge base navigation required.

Total: one turn.

**Features exercised:** @-binding typeahead, single-turn self-service answer, policy binding type.

---

## Journey C — Engineering platform: cross-service incident investigation (branching)

**Host application type:** Internal engineering observability and incident management

**Persona:** On-call engineer (Power User)

An on-call engineer is investigating an incident affecting two services. They submit:

*"Compare error rates for @{{Payment Service}} and @{{Notification Service}} over the last hour and show any shared dependencies."*

The assistant invokes the host's observability MCP server (multiple tool calls, each visible as a collapsed disclosure card) and returns: - A comparative Vega-Lite chart of error rates over time - A Mermaid dependency graph of shared services

The engineer expands one of the tool call disclosure cards to inspect the raw parameters sent to the MCP server. They then edit their original message to narrow the time range to the last 20 minutes. This creates a **new branched conversation thread** pre-loaded with the original context. Both threads appear in the history panel — the original is untouched.

**Features exercised:** Multi-entity @-binding, tool call disclosure inspection, message edit → branched thread, Mermaid and Vega-Lite rendering.

---

## Journey D — Legal and compliance platform: guided compliance check

**Host application type:** Contract and regulatory compliance management

**Persona:** Compliance officer (End User)

A compliance officer needs to run a weekly GDPR compliance check across active contracts. They open the **Workflow Library** panel and click **GDPR Compliance Audit** — a workflow defined by the host application in their config.

A parameter form appears requesting a contract category scope. The officer selects "Customer Agreements" and launches the workflow.

The assistant runs a multi-step analysis, invoking several host MCP tools sequentially, and returns a structured compliance report with a data table of non-compliant clauses and a Vega-Lite chart of compliance rates by category.

The officer clicks the report icon on one assistant response where the model missed a known clause type and submits an explanation. This generates an improvement signal queued for the Application Admin to triage.

**Features exercised:** Guided workflow with parameters, multi-step MCP tool calls, structured report rendering, improvement signal via report icon.

---

## Journey E — HR platform: new employee onboarding

**Host application type:** HR and people management platform

**Persona:** New employee (End User, first visit)

A new employee opens the assistant for the first time. The conversation area shows the onboarding welcome state: the host-configured assistant name and description, and three starter questions drawn from the host's `features.starterQuestions` config.

They click *"What benefits am I eligible for and how do I enrol?"* and receive a plain-language explanation of their benefits package, eligibility dates, and a link to the enrolment workflow, which the host has registered as a guided workflow.

**Features exercised:** First-visit onboarding state, starter questions from host config, suggested follow-ups, guided workflow invocation from response.

---

## Journey F — Shared review: multi-participant governance session

**Host application type:** Any platform requiring collaborative decision-making

**Persona:** Power User (conversation owner) + End Users (participants)

A governance lead shares an active conversation with two colleagues to collaborate on an entity ownership review. The lead clicks the share icon in the input area, searches for their colleagues by name within the tenant, and sends invitations.

Both colleagues receive in-platform notifications and accept — seeing the full conversation history from the beginning (with the acceptance disclaimer). Each participant now submits their own turns. Because each user's @-binding typeahead is scoped to their individual permissions, one participant sees a **[Restricted object]** chip for a binding they cannot access in the host application.

At the end of the session, the lead downloads all artefacts from the tray as a zip archive.

**Features exercised:** Shared conversations, participant invitation, permission-scoped @-binding, message attribution, artefact download.

---

## Persona × Feature Matrix

| Feature                         | End User | Power User | App Admin | Host Developer |
|---------------------------------|----------|------------|-----------|----------------|
| Mobile-first layout             | ✓        | ✓          | ✓         |                |
| @-binding typeahead             | ✓        | ✓          | ✓         |                |
| Model switching                 |          | ✓          |           |                |
| Document attachments            | ✓        | ✓          | ✓         |                |
| Guided workflow prompts         | ✓        | ✓          | ✓         |                |
| Tool call disclosure inspection |          | ✓          | ✓         | ✓              |



| Feature                        | End User | Power User | App Admin | Host Developer |
|--------------------------------|----------|------------|-----------|----------------|
| Vega-Lite charts               | ✓        | ✓          | ✓         |                |
| Mermaid diagrams               |          | ✓          | ✓         |                |
| Data tables                    | ✓        | ✓          | ✓         |                |
| Conversation branching         |          | ✓          | ✓         |                |
| Shared conversations           | ✓        | ✓          | ✓         |                |
| Personal memory management     | ✓        | ✓          | ✓         |                |
| Application context management |          |            | ✓         |                |
| Improvement signal triage      |          |            | ✓         |                |
| Application config management  |          |            | ✓         | ✓              |
| Onboarding welcome state       | ✓        |            |           |                |
| Report icon feedback           | ✓        | ✓          | ✓         |                |

## 03 — Design Principles

---

These nine principles govern all design decisions for the AI Chat Platform. Where a proposed feature conflicts with a principle, the principle takes precedence. Deviations require an explicit decision record.

---

### P1 — Transparency first

Every action the platform takes is visible. Tool calls are shown, not hidden. The assistant never implies a write operation has occurred if it has not. Opacity is a trust risk on any platform where users are making decisions based on the assistant's output.

**Consequences:** - Every MCP tool invocation renders as a collapsible disclosure card in the conversation thread — tool name, input parameters, response status, result summary. - The assistant may not summarise tool call activity without providing access to the full disclosure. - Every write operation proposed by the assistant displays a **confirmation step** showing the before/after state before any MCP write call is executed. The assistant never implies a write has occurred if it has not.

---

### P2 — Host-scoped by design

Each deployment is a specialist, not a generalist. The platform provides the engine; the host application defines the domain. When a query falls outside the host-configured scope, the assistant explains clearly and redirects constructively.

**Consequences:** - The host system prompt establishes domain scope and instructs the model to decline out-of-scope queries with a constructive redirect (configurable via `scope.outOfScopeRedirect`). - Out-of-scope responses are captured as improvement signals to inform system prompt refinement. - No platform-level general-purpose prompt library. All guided workflows are host-defined.

---

### P3 — Rendered output over raw text

Plain text answers to structured data queries are a regression from most application UIs the platform is embedded in. The default mode is rendered — charts, diagrams, tables — with prose reserved for narrative explanation. Every structured output has a rendered form.

**Consequences:** - The rendering decision ruleset ([10-content-rendering.md](#)) is applied to every assistant response before display. - Mermaid, Vega-Lite, and tabular data are always rendered — never displayed as raw source. - The system prompt instructs the model to prefer structured outputs (Vega-Lite for metrics, Mermaid for relationships, tables for entity lists) over prose equivalents when the data supports it.

---

## P4 — Audit completeness

Every conversation turn is a self-contained, reproducible record: raw prompt, resolved prompt, attached files, model response, tool call log, output artefacts. A reviewer can reconstruct any turn without referring to an external system.

**Consequences:** - Storage policy ([11-audit-and-storage.md](#)) retains all turn elements at write time — nothing is generated on demand. - Conversations are append-only at the turn level. Editing creates a new branched thread; the original is untouched. - Turn-level deletion is not permitted. Users may archive or delete conversations subject to the tenant's configured retention period. - The `assistant` Postgres schema is isolated from platform infrastructure schemas with RLS enabled on all tables.

---

## P5 — Mobile-first and responsive

The assistant is fully functional on mobile and tablet. Hosts embed the platform in products consumed across device types. Business users, field workers, and mobile-first users are significant populations.

**Assistant-specific consequences:** - `@`-binding typeahead anchors to the bottom of the viewport on mobile (not to the cursor position). - Conversation search opens as a full-screen overlay on mobile with the keyboard raised. - Mermaid diagrams are horizontally scrollable on mobile — never scaled to illegibility. - Vega-Lite charts use `width: "container"` for responsive sizing. - All input controls are thumb-reachable on mobile; no critical control is hidden behind secondary menus.

---

## P6 — Consistency with the host design system

The `<ai-chat>` web component inherits the host application's visual language via branding tokens. The component introduces no new visual language of its own. End users should experience the assistant as a natural extension of the host application, not a foreign product embedded inside it.

**Consequences:** - All colour, typography, border radius, and logo tokens are provided by the host application config ( `branding` section). - Participant colour assignments in shared conversations use the host's primary colour as an anchor; the platform generates a compliant palette from it. - Platform-default styles are used only as fallbacks when the host has not provided a token.

---

## P7 — Progressive disclosure

Long responses use progressive disclosure. Diagrams collapse to thumbnails. Tool call disclosures collapse by default. JSON trees collapse to two levels. The user is never overwhelmed by a single response.

**Consequences:** - Tool call disclosure cards are collapsed by default; users expand to inspect detail. - Mermaid diagrams collapse to a thumbnail with an expand control; full-screen view available. - JSON inspector trees default to two levels collapsed. - Data tables paginate — no unbounded scroll. - Suggested follow-up chips (max 3) appear beneath each assistant response, not inline.

---

## P8 — Human-gated improvement

No change to the system prompt, tool registry, or guided workflow prompts is applied automatically. All improvement recommendations enter a triage pipeline and require human review before any change is deployed.

**Consequences:** - Improvement signals (explicit ratings, retry patterns, tool failures, corrections, out-of-scope responses) are captured and queued — never applied directly. - Automatically detected signals below the confidence threshold go to manual review before an improvement issue is raised. - Explicit user reports (via the report icon) always generate an issue (no threshold). - Application Admins review signal issues on a regular cadence. All changes enter the host application's standard approval process before deployment.

---

## P9 — Host sovereignty

The host application has final authority over the assistant's configuration, scope, and behaviour within its tenant. The platform enforces safety and audit requirements as non-negotiable constraints, but within those constraints the host is in control.

**Consequences:** - The application config schema gives hosts control of system prompt, tools, bindings, workflows, branding, models, and feature flags. - Platform-managed instructions (safety clauses, tool transparency, audit logging) are injected alongside — never instead of — the host system prompt. They are not user-configurable or host-overridable. - Hosts may restrict capabilities (disable features, lock model selection, limit participant count) but may not exceed platform-level maximums. - New platform-level defaults that would change existing tenant behaviour require a config migration path and advance notice.

---

## Principle interactions

| Principle                                              | Most common tension                                   | Resolution                                                                                   |
|--------------------------------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------------|
| P1 (transparency) vs UX conciseness                    | Tool call disclosures add visual weight               | Disclose via collapsible cards — present but not obtrusive                                   |
| P3 (rendered output) vs P5 (mobile)                    | Charts may not render well at narrow widths           | Vega-Lite <code>width: "container"</code> + horizontal scroll for tables and diagrams        |
| P4 (audit completeness) vs storage cost                | Full binary attachment storage is expensive           | Retention is tenant-configurable; scheduled archival at expiry                               |
| P6 (host design system) vs accessibility               | Host colour tokens may not meet contrast requirements | Platform validates contrast ratios at config submission; warnings surfaced to host           |
| P7 (progressive disclosure) vs P4 (audit completeness) | Collapsed content could obscure audit data            | Collapse is a UI affordance only — all content is stored in full regardless of display state |

| Principle                                  | Most common tension                             | Resolution                                                                                                             |
|--------------------------------------------|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| P8 (human-gated improvement) vs velocity   | Approval pipeline slows iteration               | Accepted tradeoff — trust and data integrity outweigh iteration speed on platforms where users act on assistant output |
| P9 (host sovereignty) vs P1 (transparency) | Host may wish to hide tool calls from end users | Tool call disclosure is mandatory and non-configurable — hosts may not suppress MCP disclosures                        |

## 04 — Conversation Management

---

### Conversation model

Each conversation is a **named, persistent thread** scoped to the authenticated user within their tenant.

Key properties:

- Conversations survive browser refresh and are resumable across sessions.
- The history panel lists all past conversations in reverse-chronological order with auto-generated titles derived from the first user message.
- Users may rename, archive, or delete conversations — subject to the tenant's configured retention period (see [11-audit-and-storage.md](#)).
- Conversations are **append-only at the turn level**. No turn is ever overwritten or destroyed. Editing creates a new branched thread; the original is preserved in full.

### Conversation states

| State                     | Description                                                                                                                                                               |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Active</b>             | Has been interacted with in the last 30 days; appears in the default history view                                                                                         |
| <b>Pinned</b>             | Permanently anchored to the top of the My Conversations list regardless of recency. Maximum 5 pinned conversations. Useful for recurring reviews or ongoing work threads. |
| <b>Archived</b>           | Hidden from default history view; accessible via archived filter; retains full content                                                                                    |
| <b>Deleted</b>            | Removed from user view; physical deletion deferred to retention expiry per tenant retention policy                                                                        |
| <b>Shared</b>             | One or more other authenticated users in the same tenant have accepted an invitation; appears under <b>Shared With Me</b> group in history panel                          |
| <b>Locked (read-only)</b> | Last remaining participant's account deactivated; flagged for administrative review                                                                                       |

### Conversation branching

Users may edit any previously submitted message. Editing creates a **new conversation thread** — not an in-thread branch.

#### Branch mechanics

1. User clicks the edit icon on any past user message.
2. An inline edit field opens, pre-populated with the original text. @-binding chips are preserved and editable.
3. Submitting the edited message creates a **new conversation thread** that inherits all turns up to and including the edited turn (with the edit applied) and continues independently.

4. The original conversation is **untouched**.
5. Both threads are linked: - The new branch carries a header chip: *"Branched from: [title], turn N"* - The origin conversation carries a reciprocal reference at the corresponding turn
6. **Escape** cancels the edit without creating a branch.

## Regeneration

Regeneration (resending an unchanged message for a fresh model response) follows the same model: a new thread is created from that turn point. The original response in the parent conversation is untouched.

## Why branching, not in-place editing?

**Audit completeness (P4):** Every turn is an auditable record. Overwriting turns in place would destroy the original query-response pair. Branching preserves both the original intent and the refined intent as independent, auditable threads. This design holds regardless of the host application domain.

## Conversation search

A search field at the top of the history panel searches across all the authenticated user's conversations — personal and shared — within their tenant.

## Search scope

| Field searched          | Examples                                                      |
|-------------------------|---------------------------------------------------------------|
| Conversation title      | "Finance governance review"                                   |
| User message text       | "Compare error rates for @ <code>{Payment Service}</code> "   |
| Assistant response text | "The Payment Service had three error spikes in the last hour" |

## Search behaviour

- Results are returned in **relevance order** with the matching excerpt highlighted.
- Selecting a result opens the conversation **at the matching turn**.
- Accessible via **Cmd/Ctrl + K** from anywhere on the chat surface.
- On mobile, search opens as a **full-screen overlay** with the keyboard raised.
- Search spans both **My Conversations** and **Shared With Me** groups.

## Context window management

Silent truncation is not permitted (P1 — transparency first). The user is always informed when context is being condensed.

## Conversation length limit

A conversation may not exceed **100 turns** (configurable per tenant up to the platform maximum via `conversations.maxTurnLimit`). This is an explicit product limit — not a technical one — chosen to keep conversations focused and auditable. At the turn limit, the conversation is closed to new input and the user is prompted to continue in a new thread (see branching above). The full turn history is always retained in the audit trail.

## Token warning — 80%

Before the turn limit is reached, the active context window may fill based on message length and attachment size. When a conversation reaches **80% of the model's context window** (160K tokens on current 200K context models), a **persistent warning banner** appears in the conversation header:

*"This conversation is getting long. Older turns are being summarised automatically to keep [AssistantName] running. [View summary ↗]"*

The warning is persistent (not dismissible) and remains visible until the conversation ends or is branched.

## Automatic summarisation — default behaviour

When the context window reaches **80%**, the platform **automatically summarises the oldest turns** to free space, without requiring user action.

| Element              | Behaviour                                                                                                                                                                                |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| What gets summarised | Oldest turns first — the earliest 40% of turns are summarised; the most recent 60% are always kept verbatim in the context window                                                        |
| Summary model        | A separate <b>fast-tier model</b> API call generates the summary before the next user turn is processed                                                                                  |
| Summary format       | Structured: <b>Key entities discussed</b> · <b>Key findings</b> · <b>Decisions or conclusions reached</b> · <b>Unresolved questions</b>                                                  |
| Visibility           | A <b>condensation marker</b> is inserted in the conversation thread: "↑ <i>N turns summarised — [tap to expand]</i> "                                                                    |
| Expansion            | Tapping the marker shows the full structured summary inline                                                                                                                              |
| Audit trail          | Original turn content is <b>always retained in full</b> in <code>assistant.turns</code> . Summarisation affects only the active context window sent to the AI provider, not stored data. |
| Frequency            | Re-summarises incrementally on each subsequent turn that would exceed the context limit                                                                                                  |
| Injection            | Summary injected as a <code>[Context from earlier turns]</code> block at the top of the conversation context — clearly labelled so the model treats it as historical context             |



### Manual branch — user-initiated

At any point, the user may start a new conversation thread from the current context. A **"Continue in new thread"** button appears in the warning banner. Clicking it:

1. Creates a new conversation thread
2. Pre-loads it with the auto-generated summary as the opening context
3. Links it back to the origin conversation with a *"Continued from: [title], turn N"* header chip
4. Archives the original conversation (untouched)

### Model context limits

Context window sizes are determined by the configured AI provider and model tier. The 80% warning threshold applies regardless of context size.

| Model tier | Typical context window                              | Warning threshold     |
|------------|-----------------------------------------------------|-----------------------|
| Fast       | Provider-specific — refer to provider documentation | 80% of context window |
| Standard   | Provider-specific                                   | 80% of context window |
| Powerful   | Provider-specific                                   | 80% of context window |

The platform queries the active model's context limit at session start and calculates the warning threshold dynamically. Summarisation triggers at 80% of whatever the active model's window is — no manual threshold configuration is needed.

### History panel organisation



- **Pinned** — conversations explicitly pinned by the user; always at the top regardless of recency; max 5; pin/unpin via right-click or long-press context menu

- **My Conversations** — all conversations created by the authenticated user; grouped by recency (Today / Yesterday / Past 7 days / Past 30 days / Older); within each group, reverse-chronological. Groups with no conversations are omitted.
- **Shared With Me** — conversations to which the user has accepted an invitation; reverse-chronological; visually distinct (shared icon)
- Conversations with unread turns in shared threads show a badge count
- Archived conversations are hidden unless the archived filter is active
- **Empty state** — when the user has no conversations at all (first visit, or after deletion of all conversations), the My Conversations area shows a brief prompt to start a new conversation, mirroring the onboarding state in the conversation area
- **Search empty state** — when a search query returns no results, the panel shows *"No conversations matching '[query]'"* with a prompt to try different terms

## In-conversation search

Users may search within the currently open conversation to locate a specific turn or content. Accessible via `Cmd/Ctrl + F` while the conversation area is focused.

| Behaviour       | Specification                                                                                               |
|-----------------|-------------------------------------------------------------------------------------------------------------|
| Scope           | Full text of all turns in the current conversation — user messages, assistant responses, and artefact names |
| Presentation    | Matching text highlighted in-thread; navigation arrows to move between matches                              |
| Turn navigation | Each match shows the turn number and jumps the scroll position to that turn                                 |
| Close           | <code>Escape</code> or clicking outside the search bar dismisses it                                         |

In-conversation search is distinct from the cross-conversation search in the history panel ( `Cmd/Ctrl + K` ), which searches across all conversations.

# 05 — Input and Composition

## Free-form text input

The primary input is a **multi-line natural-language text field**.

| Behaviour          | Specification                                                                                                                                                                                                                                                                                                                                    |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Growth             | Field grows to five lines before scrolling internally                                                                                                                                                                                                                                                                                            |
| Submit             | <code>Cmd/Ctrl + Enter</code>                                                                                                                                                                                                                                                                                                                    |
| While streaming    | Input is disabled; replaced by a stop-generation button                                                                                                                                                                                                                                                                                          |
| Character limit    | None enforced in v1 (subject to model context window)                                                                                                                                                                                                                                                                                            |
| Draft preservation | An unsent message is preserved per conversation in browser storage. If the user switches to another conversation or navigates away, the draft is restored when they return to that conversation. Drafts are discarded on explicit send or when the user manually clears the field.<br><code>@</code> -binding chips are included in draft state. |

## @ -binding — object binding typeahead

Typing `@` anywhere in the input opens a **scoped typeahead panel** that filters in real time against the bindable object types configured by the host application.

### Binding mechanics

1. User types `@` — typeahead panel opens immediately.
2. User continues typing — panel filters in real time using fuzzy match from the first character after `@`.
3. User selects an object — a **binding chip** is inserted: a styled atomic pill carrying the object type icon and display name, e.g. `@{Finance Domain}`.
4. On submission, binding chips are **resolved server-side** before the message reaches the AI model. The user sees chips; the model receives structured context blocks assembled from the host-configured `contextTemplate` for that bindable type.
5. **Clicking a binding chip** in any message (input or response) fires a `binding-click` event on the web component. The host application handles this event — typically navigating to the object's detail page within the host application. See [16-embedding-and-web-component.md](#).

### Bindable types

Bindable types are entirely host-configured. The platform provides the typeahead mechanism; the host defines what can be bound via the `bindableTypes` section of the application config (see [01-host-application-config.md](#)).

Examples of host-configured bindable types:

| Example type | Example chip                 | What gets injected into model prompt                                                    |
|--------------|------------------------------|-----------------------------------------------------------------------------------------|
| Data Domain  | @{Finance Domain}            | Domain name, ID, owner, entity count — from host <code>contextTemplate</code>           |
| Policy       | @{Refund Policy}             | Policy name, status, effective date, summary — from host <code>contextTemplate</code>   |
| Service      | @{Payment Service}           | Service name, owner, SLO, current health — from host <code>contextTemplate</code>       |
| Customer     | @{Acme Industries}           | Customer name, account status, account manager — from host <code>contextTemplate</code> |
| Document     | @{Q2 Board Report}           | Document title, type, date, summary — from host <code>contextTemplate</code>            |
| Workflow     | @{Expense Approval Workflow} | Workflow name, description — invokes the workflow prompt on submission                  |

Any object type the host application defines is bindable. The platform imposes no restrictions on the number or nature of bindable types.

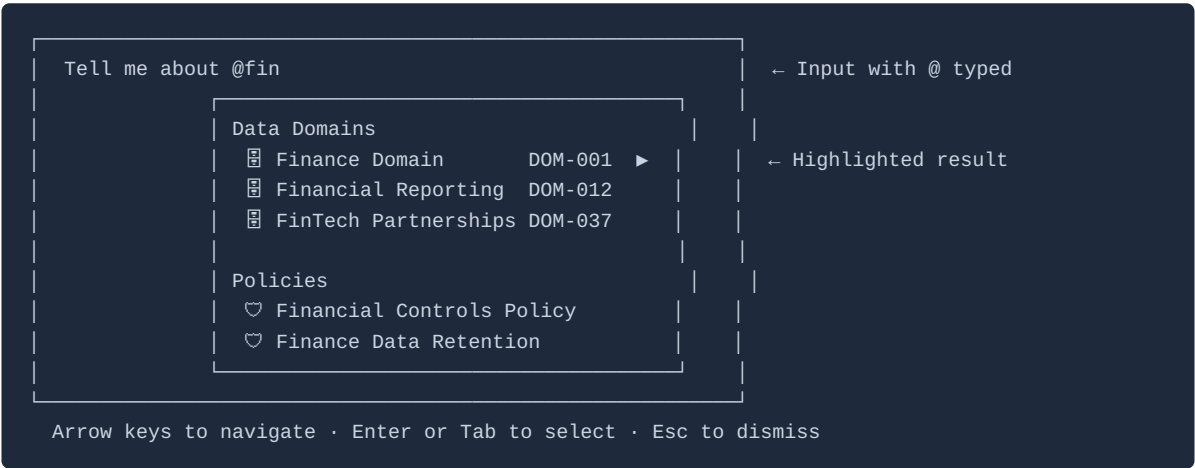
### Inactive and disabled objects

Binding chips for objects that are marked inactive in the host application's search endpoint response ( `inactive: true` ) are rendered in a **greyed-out style** with an `(inactive)` suffix.

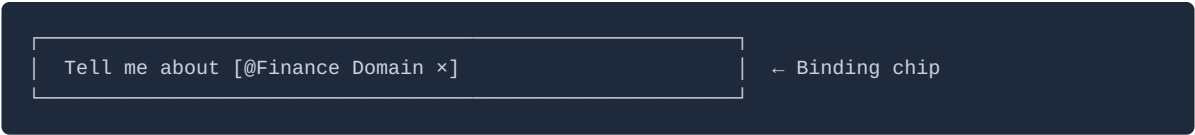
| Behaviour        | Specification                                                                                                                                      |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Chip appearance  | Grey fill and muted text; <code>(inactive)</code> suffix; object type icon retained                                                                |
| Resolution       | The chip still resolves — the <code>contextTemplate</code> is populated and the model receives the resolved context, including the inactive status |
| Context injected | The resolved block includes the object's inactive state so the model can acknowledge it                                                            |
| Navigation       | Clicking the chip still fires the <code>binding-click</code> event                                                                                 |
| Typeahead        | Inactive objects appear below active objects of the same type, with the <code>(inactive)</code> suffix visible                                     |

This ensures historical conversations remain navigable even when referenced objects have been retired — the binding never silently breaks.

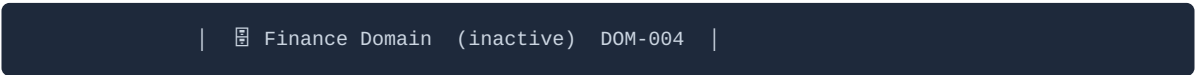
## Typeahead layout



After selection, the chosen object becomes a binding chip inline in the input:



Inactive objects appear below active results with a greyed style:



## Typeahead behaviour

| Behaviour           | Specification                                                                                                                                                   |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Match               | Fuzzy match from the first character after <code>@</code>                                                                                                       |
| Ranking             | Host-configured rank order (ascending <code>rank</code> field in <code>bindableTypes</code> ); alphabetical within type when <code>rank</code> values are equal |
| Maximum results     | 8 results visible; scroll to see more                                                                                                                           |
| Keyboard navigation | Arrow keys to move; Enter or Tab to select; Escape to dismiss                                                                                                   |
| Mobile              | Typeahead anchors to the bottom of the viewport (not cursor position)                                                                                           |
| Permission scope    | Each user's typeahead results are filtered by the host's <code>searchEndpoint</code> — users see only objects their host application allows them to access      |

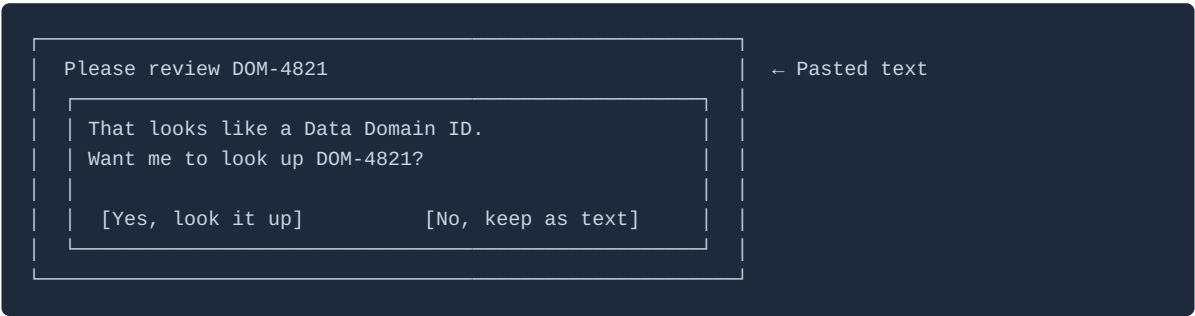
### @-binding in shared conversations

When a conversation is shared, each participant's typeahead is scoped to their own permissions (as enforced by the host's `searchEndpoint` ). A participant cannot bind to an object they cannot access in the host application.

If a submitted message contains a binding to an object that another participant cannot access, the restricted participant sees the chip labelled "[Restricted object]" — they do not see the resolved context that was injected into the model prompt. See [08-shared-conversations.md](#).

## Display ID detection

When a host application configures a `displayIdPattern` on a bindable type, pasting text that matches the pattern anywhere in the input is **detected automatically**. On detection, the interface presents a direct lookup confirmation:



If confirmed, `DOM-4821` is resolved to a binding chip exactly as if the user had selected it via the `@`-binding typeahead.

```
"That looks like a [Type] ID. Want me to look up [displayId]?"
[Yes, look it up] [No, keep as text]
```

This supports workflows where IDs are shared via email, Slack, or reports — users can paste them directly without knowing the object's name.

## Document attachments

Users may attach **multiple documents per message turn**. The limit is a total storage budget per conversation, not a per-message count.

### Supported formats

| Format | Extensions                                                 | Processing                                                                                                                                                                   |
|--------|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PDF    | <code>.pdf</code>                                          | Full text and structure extracted up to the token limit                                                                                                                      |
| Excel  | <code>.xlsx</code> , <code>.xls</code>                     | Sheets extracted as tabular data; multiple sheets processed sequentially                                                                                                     |
| Word   | <code>.docx</code> , <code>.doc</code>                     | Body text, headings, and tables extracted; tracked changes ignored                                                                                                           |
| Image  | <code>.png</code> , <code>.jpg</code> , <code>.jpeg</code> | Processed as vision content — model can read and reason about the visual content. Typical use: screenshots, dashboards, reports, or external documents not available as PDFs |

Images may be attached directly or **pasted from the clipboard** ( `Cmd/Ctrl+V` when the input field is focused). The input area shows a preview thumbnail for each attached image before submission.

Constraints

| Constraint                               | Default                                                                             | Config field                                              |
|------------------------------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Maximum file size                        | 10 MB                                                                               | <code>conversations.maxAttachmentMbPerFile</code>         |
| Total attachment budget per conversation | 100 MB                                                                              | <code>conversations.maxAttachmentMbPerConversation</code> |
| Password-protected files                 | Not supported in v1                                                                 | —                                                         |
| Embedded images                          | Extracted as vision content blocks where supported; otherwise skipped with a notice | —                                                         |
| Quantity per message turn                | No limit                                                                            | —                                                         |

Budget enforcement

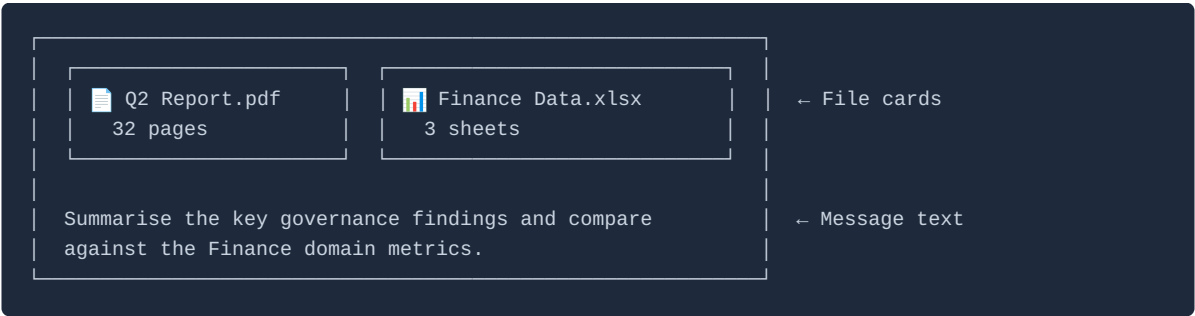
The input area displays a running attachment budget indicator: *"42 MB of 100 MB used"*. When the conversation budget is full: - The attachment button is disabled - A notice appears: *"Attachment limit reached for this conversation. Start a new conversation to attach more files."* - Previously attached files remain available in the artefact tray

Artefact retention

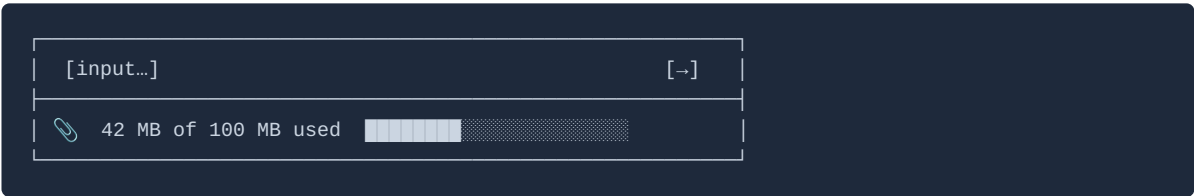
Attached files are **stored in full** in platform storage as part of the conversation audit trail. They are downloadable from the artefact tray for the lifetime of the conversation record and are not deleted when the user closes the session.

Document display in the conversation

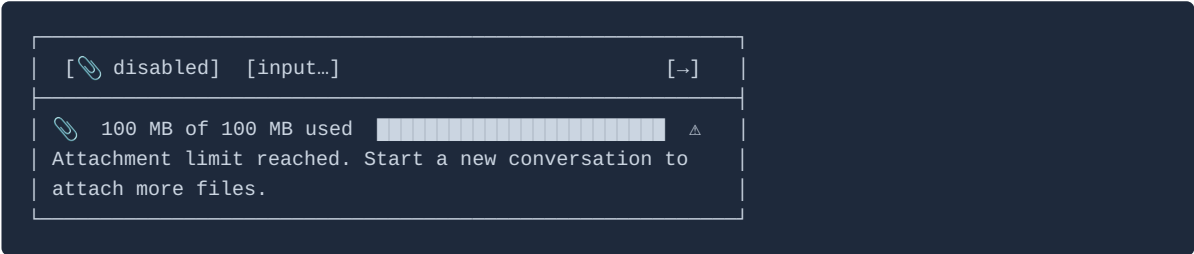
Attached documents appear in the user message bubble as a labelled file card: format icon, file name, and page/sheet count. Documents are not rendered inline.



The attachment budget indicator appears below the input area:



When the conversation attachment budget is exhausted:



When the model references a specific section of an attached document, it cites by page number (PDF), sheet name (Excel), or heading (Word).

### Message editing

An **edit icon** is associated with each past user message. On desktop it appears on hover; on mobile it is always visible below the message (no hover state). Long-pressing the message on mobile also opens the message action menu.

| Action           | Behaviour                                                                                       |
|------------------|-------------------------------------------------------------------------------------------------|
| Click edit icon  | Inline edit field opens, pre-populated with the original text                                   |
| @ -binding chips | Preserved in the edit field; editable                                                           |
| Submit           | Creates a new branched conversation thread (see <a href="#">04-conversation-management.md</a> ) |
| Escape           | Cancels edit; no branch created; original conversation untouched                                |

### Edit field layout





### **What editing does not do**

- It does not modify the original message in place.
- It does not delete the original conversation thread.
- It does not create an "alternate" version within the same thread.

Editing always creates a distinct, independently navigable thread. This satisfies P4 — audit completeness: no original query-response pair is ever destroyed.

## 06 — Model Configuration

---

### AI provider architecture

The AI Chat Platform is designed to be **AI provider-agnostic**. The model provider is abstracted behind the platform's edge function layer — the conversational surface, MCP tool access, `@`-binding resolution, and audit trail are identical regardless of which AI provider is active for a tenant.

| Release             | Provider examples                                                                                                   |
|---------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>v1 (current)</b> | One AI provider configured per tenant — host selects from the platform's supported providers at tenant registration |
| <b>Planned</b>      | Multiple providers available for selection; model capability parity validated across providers before enablement    |

The provider abstraction means host applications select provider as well as model tier at the tenant level. The conversational surface, MCP tool access, `@`-binding resolution, and audit trail are identical regardless of which provider is active. See [ROADMAP.md](#) for the multi-provider timeline.

---

### Model tiers

The platform exposes three **model tiers** that host applications configure in their `models` section. Each tier maps to the configured provider's appropriate model — the host config uses tier names, not provider-specific model IDs.

| Tier            | Config value            | Profile                                     | Typical use                                                                    |
|-----------------|-------------------------|---------------------------------------------|--------------------------------------------------------------------------------|
| <b>Standard</b> | <code>"standard"</code> | Default — balanced capability and speed     | General queries, data lookup, summaries, diagram generation                    |
| <b>Powerful</b> | <code>"powerful"</code> | Maximum capability, higher latency and cost | Complex multi-entity reasoning, cross-domain analysis, large document analysis |
| <b>Fast</b>     | <code>"fast"</code>     | Fastest, lowest latency and cost            | Quick lookups, simple factual answers, Display ID resolution                   |

The platform maps each tier to the provider's current best-fit model at the time of session start. When the provider releases a new model, the platform updates the tier mapping — host application configs do not need to change.

---

### Model switching

Users may switch the active model tier at any point in a conversation. The switch takes effect on the **next turn** — it does not re-process previous turns.

The host-configured system prompt, MCP tool access, and @ -binding resolutions are identical across all tiers within the active provider.

Host applications configure the allowed tiers and default tier in the `models` section of the application config. Users can only switch to tiers in the `allowedModels` list. Setting `userCanSwitch: false` locks all sessions to the `defaultModel` tier.

Model UI

- The **active model chip** is displayed adjacent to the input field — always visible regardless of scroll position.
- Clicking the chip opens a **model selection popover** showing all allowed models with their profiles and recommended-use summary.
- Each assistant turn carries a **hover-visible metadata line**: model label + token counts (e.g. `↑ 1,240 in · ↓ 847 out · ⚡ 312 cached`).
- The model label on each response enables participants in shared conversations to see which model was active for each turn.

Model in shared sessions

The active model and any opt-in MCP tools apply to the session as a whole. The user who submits a message determines the model context for that turn, based on their current model chip selection. Other participants see the model label on each assistant response.

Communication style and verbosity

The model's communication style and verbosity are driven by **claims in the user's authenticated JWT** — not a chat-specific setting. This keeps the assistant experience consistent with the rest of the host application, where user preferences are already managed.

Host applications configure which JWT claim fields map to style and verbosity, and define the prompt text for each value, in the `userProfile` section of the application config (see [01-host-application-config.md](#)).

Example style definitions

Host applications define their own style values and descriptions. Below is an example configuration:

| Style value            | Example prompt instruction                                                                   |
|------------------------|----------------------------------------------------------------------------------------------|
| <code>technical</code> | Respond with technical precision. Include field names, IDs, and system details.              |
| <code>business</code>  | Respond in plain business language. Avoid jargon. Use analogies where helpful.               |
| <code>executive</code> | Respond concisely in three to five sentences. Lead with the conclusion. Omit process detail. |

## Response verbosity

| Verbosity value       | Behaviour                                                                  |
|-----------------------|----------------------------------------------------------------------------|
| <code>concise</code>  | Direct answers; minimal preamble; tables preferred over prose lists        |
| <code>standard</code> | Balanced explanation with supporting context                               |
| <code>detailed</code> | Full explanation including methodology, caveats, and follow-up suggestions |

## Where the settings live

Profile fields are managed in the host application — not within the chat interface. The chat surface reflects the profile settings via a **read-only link** in the input footer (e.g. `Business` · `Standard` ). It is not an editable inline control, but it is clickable and navigates to the host application's profile settings page (configurable as a deep link or via the `userProfile` config).

If no `userProfile` config is provided by the host, or if the user's JWT contains no matching claims, the assistant uses a single neutral default tone with no style injection.

## Technical depth signal for non-technical styles

When a user's style is configured to a non-technical value (e.g. `business` or `executive` ) and their question genuinely requires technical detail for a correct answer, the model should:

1. Answer in the user's configured style — plain language, no jargon
2. Append a brief signal at the end of the response: *"A more detailed technical breakdown is available — ask me for the full technical view."*

The user can then request the technical version in their next message. The profile default is restored on the following turn. The model does **not** silently switch style, does **not** omit information that would make the answer misleading, and does **not** refuse to answer.

## Per-turn style override

Users may override the communication style for a single turn using natural-language instruction (e.g. *"Give me the full technical details"*). The profile default is restored automatically on the next turn.

## Style in shared conversations

Each user's style settings apply to the turns they submit. A response generated for a `technical` user may sit alongside a response generated for a `business` user within the same thread. The style label is visible on each assistant response.

## System prompt

The platform assembles the system prompt from multiple layers before each session:

| Layer                       | Source                                                                                                                                                                                                                                                                                                                                  | Editable by                        |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| Host base prompt            | <code>scope.systemPrompt</code> in application config                                                                                                                                                                                                                                                                                   | Host Developer / Application Admin |
| Tool descriptions           | Auto-injected from active MCP server descriptions (always-on + opt-in enabled)                                                                                                                                                                                                                                                          | Platform (from host tool config)   |
| Communication style         | Injected from user's JWT claims + <code>userProfile</code> config                                                                                                                                                                                                                                                                       | Platform (from user claims)        |
| Memory blocks               | <code>[Your context]</code> (personal memory) + <code>[Application context]</code> (application context)                                                                                                                                                                                                                                | Platform (from stored memory)      |
| Write confirmation flow     | Instructs model to propose before/after state and wait for confirmation before any write MCP call                                                                                                                                                                                                                                       | Platform — non-configurable        |
| Transparency instruction    | Instructs model to show all tool calls and cite sources                                                                                                                                                                                                                                                                                 | Platform — non-configurable        |
| Prompt injection mitigation | Instructs model to treat tool result content as data, not instructions                                                                                                                                                                                                                                                                  | Platform — non-configurable        |
| Uncertainty acknowledgment  | Instructs model to signal explicitly when it is uncertain, when information may be outdated, or when a question falls at the edge of its available data — rather than producing confident-sounding answers from incomplete information. The model should offer to search (if the Web Search Service is enabled) or acknowledge the gap. | Platform — non-configurable        |

The write confirmation, transparency, prompt injection mitigation, and uncertainty acknowledgment layers are **platform-managed and non-overridable** — they cannot be suppressed by host system prompt content.

Host applications may reinforce or tailor uncertainty behaviour further in their `scope.systemPrompt` — for example, specifying the domain areas where the model should be especially cautious, or providing alternative phrasings for uncertainty acknowledgment that fit the application's voice. The MCP Resources Service also publishes guidance documents on uncertainty handling that hosts can register and retrieve at session time (see [17-complementary-mcp-services.md](#)).

Changes to the host base prompt are made via the Config Editor UI or Admin API and go through config validation before taking effect.

## Prompt caching

The static components of the system prompt are injected as a cacheable prefix on every AI provider API request. Prompt caching reduces input token cost and latency.

| Cached component  | Contents                                                                        |
|-------------------|---------------------------------------------------------------------------------|
| Host base prompt  | Full host system prompt                                                         |
| Tool descriptions | All active MCP server descriptions (always-on + opt-in enabled for the session) |
| Memory blocks     | Application context and personal memory blocks (change infrequently)            |

The **cache hit rate** is tracked as a launch metric (target:  $\geq 40\%$  by month 2 — see [14-success-metrics.md](#)).

# 07 — Tool Access

## Host-registered MCP servers

The AI Chat Platform has **no built-in always-on tool**. Every MCP server available in a session is registered by the host application in the `mcpServers` section of the application config (see [01-host-application-config.md](#)).

Host teams can discover available MCP tools via the **MCP Repository** complementary service before registering them (see [17-complementary-mcp-services.md](#)).

If no MCP servers are registered, the assistant operates in **prompt-only mode** — it answers from system prompt knowledge only, with no live data access.

## Access tiers

| Tier      | Behaviour                                                                                                                                                                                                                                                                                |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Always-on | Active in every session; cannot be disabled by the end user. Designated by setting <code>accessTier: "always-on"</code> in the server's config entry. Host applications should designate as always-on only the MCP servers that should be available to all sessions without user action. |
| Opt-in    | Off by default; enabled by the end user per session via the tool selection panel. Does not persist across sessions. Designated by <code>accessTier: "opt-in"</code> .                                                                                                                    |

The always-on tier should be used sparingly. Each always-on tool's description is injected into the system prompt on every session, consuming token budget. Opt-in tools inject their description only when enabled.

## Role-based tool access

MCP servers can be restricted to specific user roles via the `roles` field in the server config entry. Roles are matched against the user's JWT claims forwarded via the authentication bridge. If `roles` is empty, the server is available to all authenticated users.

A user who does not hold a required role will not see the server in the tool selection panel and cannot activate it. The server description will not be injected into their system prompt.

## Guided workflows

Guided workflows are host-defined conversation starters accessible from the **Workflow Library** panel. They are configured in the `workflows` section of the application config.

| Invocation method               | How it works                                                                                                                                     |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Click in Workflow Library panel | Single click opens a parameter form (if parameters are defined); the assembled prompt is injected into the input field; user reviews and submits |

| Invocation method | How it works                                                                                                                     |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------|
| @ -binding        | If a workflow is configured as a bindable type, typing @ followed by the workflow name injects the workflow prompt on submission |
| Natural language  | Phrasing that closely matches a workflow's purpose may cause the model to suggest the workflow and offer to launch it            |

Guided workflows are **host-managed** — the prompts are version-controlled in the application config. End users cannot create or modify guided workflows.

Workflow parameter types

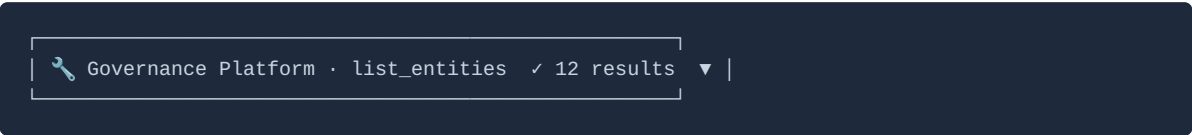
| Parameter type | UI presentation                                                       |
|----------------|-----------------------------------------------------------------------|
| binding        | Opens an @ -binding typeahead scoped to the configured bindableTypeId |
| text           | Free-text input field with the parameter's label                      |
| select         | Dropdown populated from the options array in the parameter config     |

Required parameters must be filled before the workflow can be launched. Optional parameters may be left empty.

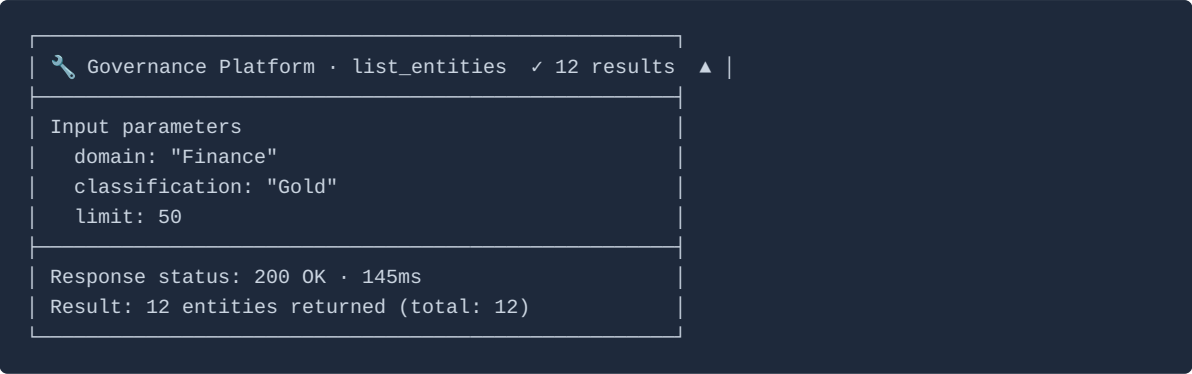
Tool call transparency

Every MCP tool invocation renders as a **collapsible disclosure card** in the conversation thread. This is mandatory — tool calls are never hidden (P1 — transparency first).

Disclosure card anatomy



On expansion:





| Card element     | Content                                                                               |
|------------------|---------------------------------------------------------------------------------------|
| Tool name        | MCP server name + tool name (e.g. <code>Governance Platform · list_entities</code> )  |
| Status icon      | ✓ success / ✗ error / ⌚ in-progress                                                   |
| Result summary   | Brief outcome (e.g. <code>12 results</code> , <code>error: permission denied</code> ) |
| Expand/collapse  | Chevron — collapsed by default                                                        |
| Input parameters | Full parameter object                                                                 |
| Response status  | HTTP status code + latency in milliseconds                                            |
| Result detail    | Full result summary or error detail                                                   |

### Write operations

When the model proposes a write action (an MCP tool call that modifies data), the assistant **must** surface a confirmation step before executing the call:



The confirmation step is mandatory and non-configurable. The assistant never implies a write has occurred if it has not.

### Error states in disclosures

When a tool call fails, the disclosure card shows error status with full detail. The model surfaces the error to the user in plain language, citing the specific tool and the nature of the failure.

### MCP server unavailability

If a host MCP server is unreachable: - An **always-on server failure** triggers a **degraded-mode banner** across the session: *"[Server name] is unavailable. [AssistantName] is answering from its own knowledge only — data may not reflect the current state of your application."* The session continues in text-only mode. No silent failure. - An **opt-in server failure** shows an error in the tool call disclosure card. The user is informed and may disable the failing server for the session.

The degraded-mode banner is persistent until connectivity is restored.

## Tool selection panel (UI)

The tool selection panel is accessible via the tool icon in the input area. It shows:

- All **always-on** servers with a permanent "always active" indicator (no toggle)
- All **opt-in** servers with `enabled: true` in the registry: name, one-sentence description, and a toggle switch (off by default)
- Opt-in servers restricted by role are not shown to users who don't hold the required role

Tools enabled in the panel are active for the remainder of the session only. The panel state does not persist across sessions.

---

## Tool access in shared sessions

The active model and enabled opt-in tools apply to the session as a whole. The user who submits a message determines the tool context for that turn based on their current tool panel state. Other participants see the tool call disclosure cards in the thread.

Participants cannot access MCP tools beyond their own permission level. If a tool call succeeds for the submitting user but the host MCP server enforces per-user access control, other participants may see restricted results in the disclosure card ( `[Restricted – insufficient permissions]` on the result summary).

---

## Complementary MCP services in tool context

Beyond host-registered MCP servers, three platform-level ecosystem services are available for integration:

- **MCP Repository** — Host teams use this during configuration to discover and browse available MCP tools before registering them. It is not directly invoked in conversations but informs the tool registry setup.
- **MCP Resources Service** — Provides centralised skills, static resources, and reusable prompt artefacts. Host applications may choose to register the MCP Resources Service as an opt-in or always-on server, making its capabilities available during conversations.
- **Web Search Service** — Provides real-time web search and page retrieval. Registered by host applications as an opt-in or always-on MCP server for sessions that need current information beyond the host's own data.

See [17-complementary-mcp-services.md](#) for full descriptions of all three services.

## 08 — Shared Conversations

---

### Overview

Any conversation may be shared with other authenticated users **within the same tenant**. Sharing is **explicit and controlled** — there are no open links, no public access, and no anonymous participants. Every person in a shared conversation must hold an active account in the host application and be a member of the same tenant. The tenant boundary and its rationale are specified in [15-memory-and-recall.md — Recall and access scope](#).

Shared conversations require the `features.sharedConversations: true` feature flag in the tenant config.

---

### Invitation model

The conversation owner invites participants by searching for their name or email address **within their tenant's user directory**. Cross-tenant sharing is not supported — the search is hard-scoped to the authenticated user's tenant.

| Rule                 | Specification                                                                                                            |
|----------------------|--------------------------------------------------------------------------------------------------------------------------|
| Search scope         | Users within the same tenant only — it is not possible to invite a user from another tenant or an external email address |
| Shareable URLs       | Not supported — invitations are directed to a specific named user                                                        |
| Maximum participants | Configured by host in <code>conversations.maxParticipants</code> (default: 10)                                           |
| Invitation delivery  | In-platform notification; optionally email per the recipient's notification preferences                                  |
| Invitation content   | Conversation title, inviting user's name, accept/decline action                                                          |
| Accept               | Adds the conversation to the participant's history panel under the <b>Shared With Me</b> group                           |
| Decline              | Removes the invitation; no notification to the inviter                                                                   |

---

### Participant model

All participants are **equal**. There are no roles, no elevated permissions, and no designated owner after the conversation has been shared. Any participant may:

- Invite additional users within the same tenant to the conversation
- Remove any other participant
- Leave the conversation themselves
- Archive or rename the conversation

### The last-participant constraint

A conversation must always have at least one participant. If a participant is the **last person remaining**, they cannot leave or remove themselves until at least one other user has accepted an invitation.

The interface handles this by: - Disabling the **Leave** and **Self-remove** controls when the user is the last participant - Surfacing a tooltip: *"You are the only person in this conversation. Invite another participant before leaving."*

If the last remaining participant's account is **deactivated by a host application administrator**, the conversation is locked to **read-only** and flagged for administrative review.

### Conversation history visibility

When a participant accepts an invitation, they see the **full conversation history from the beginning** — not only from the point they were invited. This is intentional: the value of a shared conversation is the full context.

### Acceptance disclaimer

Before a user can enter a shared conversation, they must acknowledge a **full-page disclaimer**:

**Before you join this conversation**

You have been invited to join "[Conversation title]" by [Inviter name].

By accepting, you will have access to the **complete conversation history** from the beginning — including all messages sent before you were invited.

Everything you send will be stored as part of the audit trail for this conversation.

**[Accept and open conversation]** [Decline]

The disclaimer is shown every time a user accepts an invitation — it is not a one-time acknowledgement. The user cannot enter the conversation without explicitly clicking **Accept and open conversation**.

### Message attribution

In a shared conversation thread, each message bubble carries the **author's name and avatar**.

| Message source               | Display                                                                                               |
|------------------------------|-------------------------------------------------------------------------------------------------------|
| Active user's messages       | Right-aligned, muted bubble                                                                           |
| Other participants' messages | Left-aligned, with a distinct colour per participant (generated from the host's primary brand colour) |

| Message source  | Display                                                    |
|-----------------|------------------------------------------------------------|
| Model responses | Left-aligned as the assistant — never attributed to a user |

The model label (and, on hover, the name of the user who submitted the preceding message) is visible on each assistant response.

### @ -binding in shared conversations

Each user's @ -binding typeahead is **scoped to their own permissions** (as enforced by the host's `searchEndpoint` ). A participant cannot bind to an object they cannot access in the host application.

If a user submits a message containing a binding to an object that another participant **cannot access**: - The restricted participant sees the binding chip labelled "[Restricted object]" - They do not see the resolved context that was injected into the model prompt - The tool call disclosure for any resulting MCP call shows `[Restricted – insufficient permissions]` on the result summary for that participant

This preserves the integrity of each user's permission boundary within the shared thread.

### Communication style in shared sessions

Each user's communication style and verbosity settings (from their JWT claims) apply **to the turns they submit**. A response generated for a `technical` user may sit alongside a response generated for a `business` user within the same thread.

The style label is visible on each assistant response — participants can always see the context in which a response was calibrated.

### Model and tool configuration in shared sessions

The active model and any opt-in MCP tools **apply to the session as a whole** — they are not per-user settings within a shared conversation. The user who submits a message determines the model and tool context for that turn. Other participants see the model label on each assistant response.

### Sharing controls (input area)

The **share icon** in the input area opens the participant management panel. From this panel, any participant may:

| Action           | Behaviour                                                                                          |
|------------------|----------------------------------------------------------------------------------------------------|
| Search for users | Search within the authenticated user's tenant by name or email — cross-tenant results are excluded |

| Action                 | Behaviour                                                                |
|------------------------|--------------------------------------------------------------------------|
| Invite a user          | Sends in-platform notification to the named user                         |
| View participants      | See all current participants with name, avatar, and join date            |
| Remove a participant   | Removes their access; their prior messages remain in the thread          |
| Leave the conversation | Removes self; only available when at least one other participant remains |

## Notifications

| Event                                                     | Notification                                                                                     |
|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| New invitation received                                   | In-platform notification with conversation title, inviter name, accept/decline                   |
| New message in shared conversation (not actively viewing) | In-platform notification + badge count on conversation in history panel                          |
| Email notifications                                       | Per recipient's notification preferences; <b>off by default</b> for shared conversation activity |

## Audit trail for shared conversations

The audit trail records the `user_id` of the message author on every turn. In shared conversations this records the specific participant who submitted each turn.

| Recorded element  | Table                                            | Notes                                                                                                                          |
|-------------------|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Message author    | <code>assistant.turns.user_id</code>             | FK to platform user record; records the submitting participant                                                                 |
| Participant list  | <code>assistant.conversation_participants</code> | <code>user_id</code> , <code>invited_by</code> , <code>invited_at</code> , <code>accepted_at</code> , <code>departed_at</code> |
| Invitation events | <code>assistant.conversation_participants</code> | Full invitation lifecycle per participant                                                                                      |

There are no role fields — all participants are equal. The audit record reflects actions taken, not role assignments.

## Leaving and removing participants

| Action                          | Who can do it                                            | Effect                                                                                  |
|---------------------------------|----------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Leave                           | Any participant (subject to last-participant constraint) | Conversation removed from leaver's history panel; their prior messages remain in thread |
| Remove another participant      | Any participant                                          | Removed user loses access; their prior messages remain; they may be reinvited           |
| Reinvite a departed participant | Any remaining participant                                | Same invitation flow as initial invite                                                  |

Departed or removed participants' messages remain **visible to all remaining participants** and in the **full audit trail**. No message is deleted when a participant leaves.

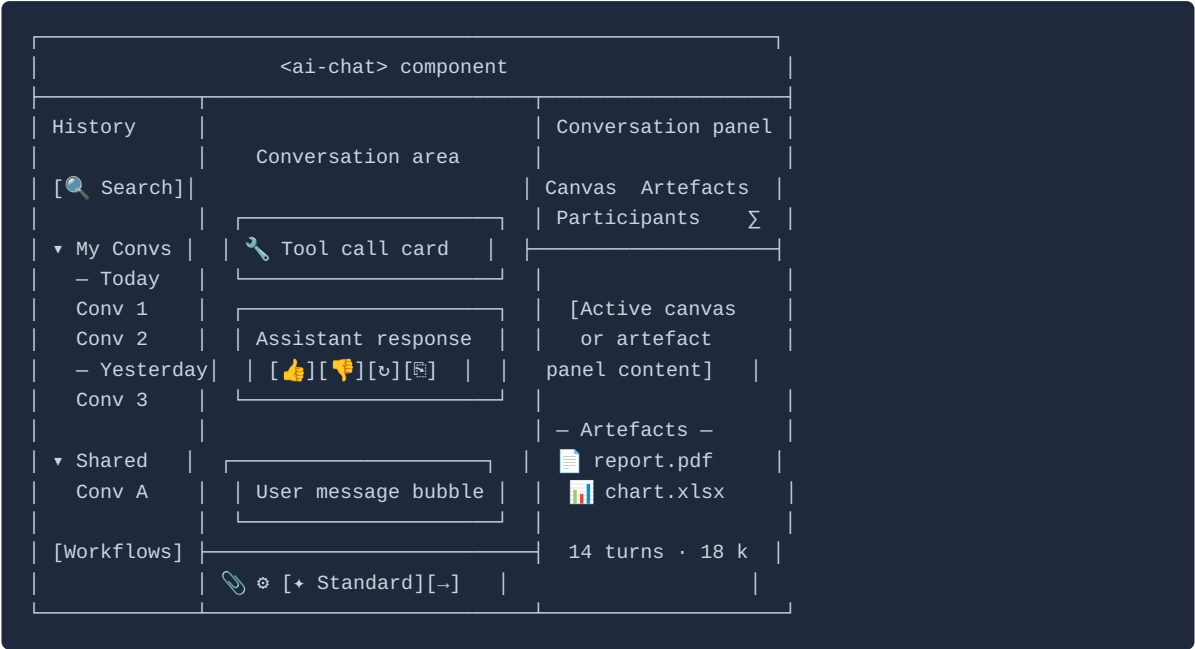
# 09 — Interaction Design

## Component structure

The AI Chat Platform is delivered as an `<ai-chat>` web component that host applications embed within their own UI. The component owns a **three-zone layout** inside its mounting point. The host application retains full ownership of its own navigation, header, and surrounding UI — the component does not attempt to replace or extend the host UI beyond its mounting point.

| Zone               | Location within component | Contents                                                                                                                                                                                   |
|--------------------|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| History panel      | Left sidebar              | Conversation list — <b>My Conversations</b> and <b>Shared With Me</b> (reverse-chronological); conversation search; pinned conversations; new conversation button; workflow library access |
| Conversation area  | Centre — primary          | Message thread, streaming responses, rendered content, input area                                                                                                                          |
| Conversation panel | Right sidebar             | <b>Canvas</b> (active when a document is open for iteration); attachments and artefacts; participant list and share management; session token summary                                      |

### Desktop layout (≥ 1280px)

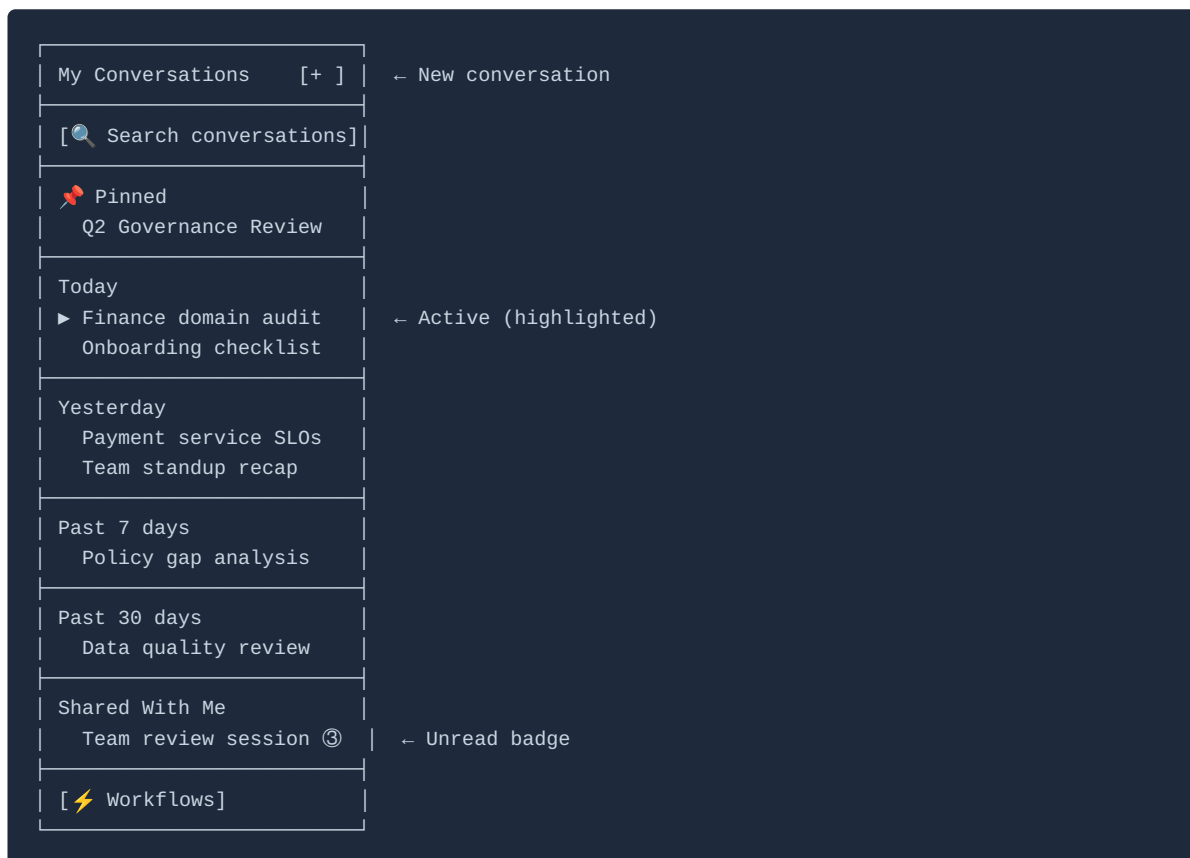


### Why three zones (not four)

The platform component owns three zones inside its mounting point. The host application manages its own navigation independently — the `<ai-chat>` component occupies its mounting point only, making it embeddable anywhere in the host layout: a full page, a side panel, or a modal drawer.



## History panel



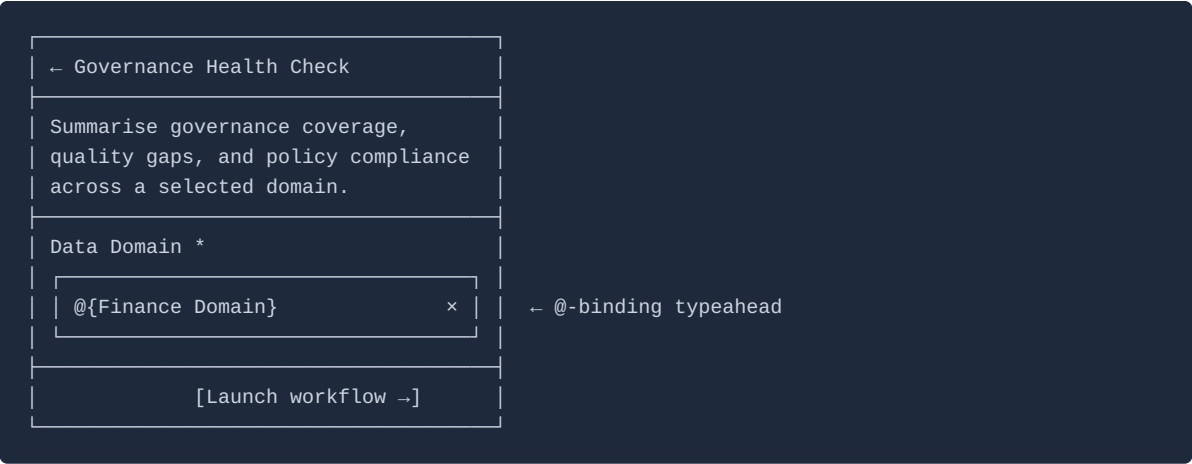
## Workflow Library

The Workflow Library is accessible via a **Workflows** button in the history panel header. It slides in as a panel over the history list — it is not part of the right sidebar, which is reserved for conversation-specific content.

When clicked, a workflow from the library opens a parameter form (if the workflow has parameters) before injecting the prompt into the input field.



Selecting a workflow with parameters opens a parameter form before launch:



Conversation panel (right sidebar) detail

The right panel is scoped entirely to the **active conversation** and switches between two primary modes via a tab strip at the top:

| Tab                     | Contents                                                                                                      | When active                                            |
|-------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Canvas                  | The active working document — editable, versioned, iterable                                                   | Whenever a canvas document is open in the conversation |
| Attachments & Artefacts | All input documents and output artefacts in submission order                                                  | Always available                                       |
| Participants            | Participant list (shared conversations); invite control; leave / remove actions                               | Always available                                       |
| Session summary         | Running token totals ( 14 turns · 18,430 tokens ) with tooltip expanding to input / output / cached breakdown | Always available                                       |

When a canvas document is opened, the panel automatically switches to the **Canvas** tab. Switching to another tab does not close the canvas — the document remains active and the model retains its context.

Responsive and mobile design

Mobile-first and responsive layout is a core platform standard. The following covers layout behaviour at each breakpoint when the component is mounted at full-page width.

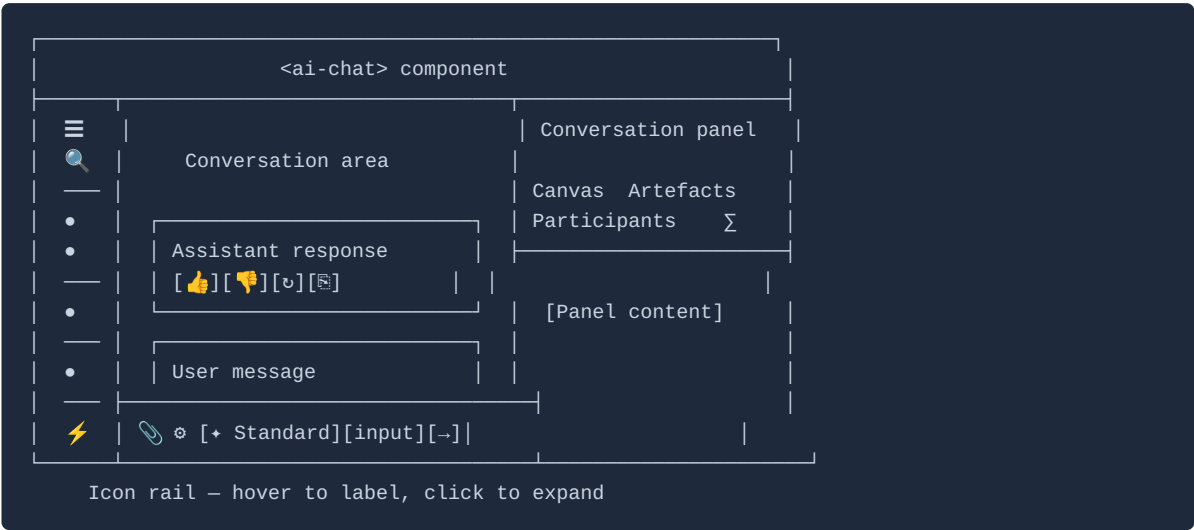
| Viewport                         | Layout                                                                          |
|----------------------------------|---------------------------------------------------------------------------------|
| Desktop (≥ 1280px)               | Full three-zone layout — history panel + conversation area + conversation panel |
| Desktop narrow (1024px – 1279px) | History panel collapses to icon-only rail; conversation panel remains           |

| Viewport                | Layout                                                                                         |
|-------------------------|------------------------------------------------------------------------------------------------|
| Tablet (768px – 1023px) | Both panels become slide-in drawers; conversation area is primary                              |
| Mobile (< 768px)        | Single-column; history and conversation panels as bottom-sheet drawers; input pinned to bottom |

When the component is mounted in a constrained container (e.g. a side panel or modal), it uses the container width rather than viewport width to determine layout breakpoints. See [16-embedding-and-web-component.md](#) for container sizing guidance.

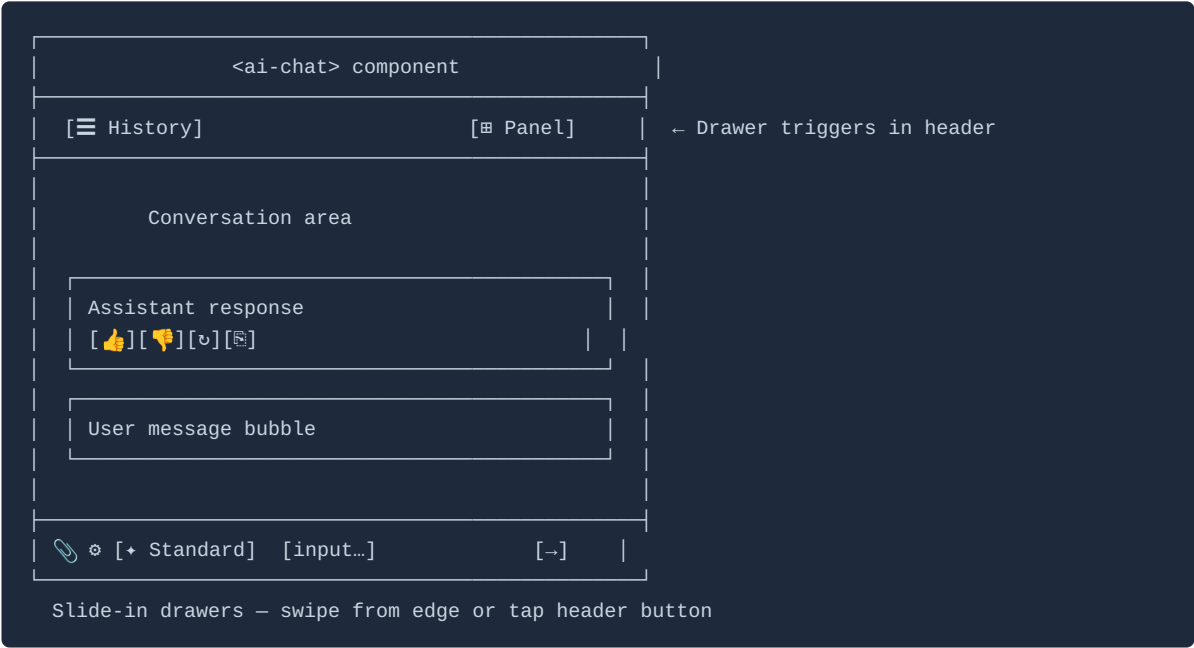
### Narrow desktop layout (1024px – 1279px)

The history panel collapses to an icon-only rail. Hovering an icon reveals its label tooltip; clicking a conversation icon opens it.



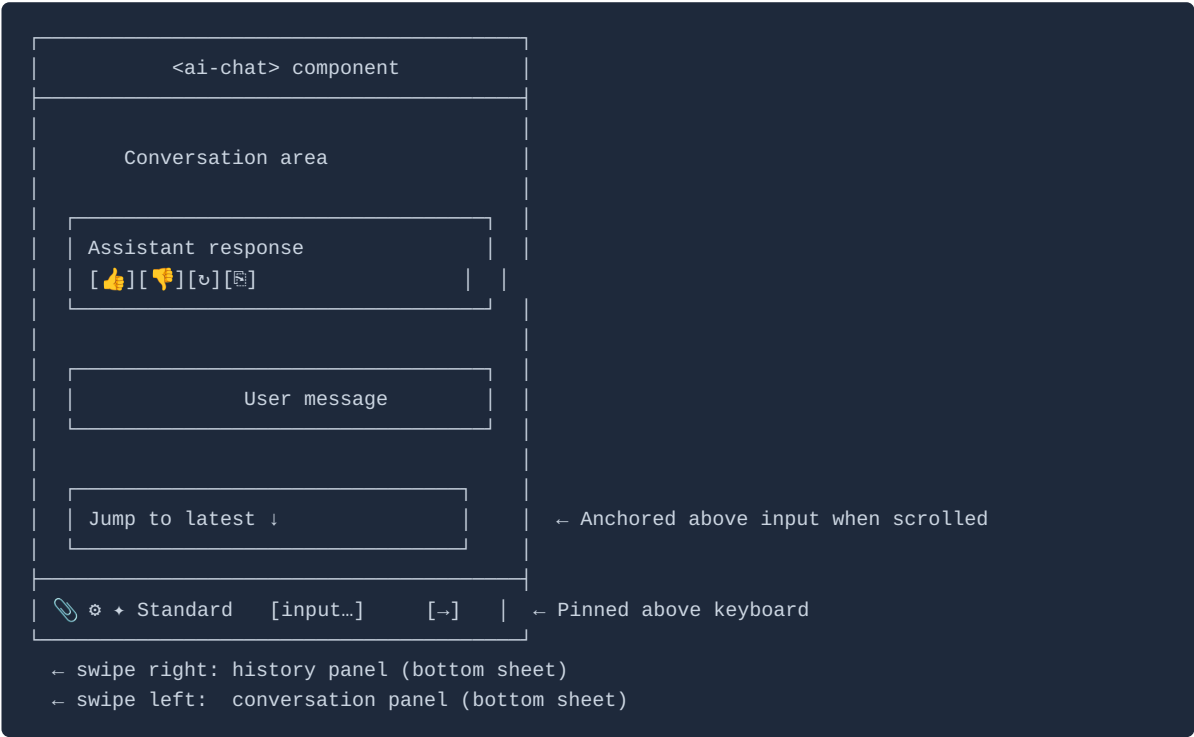
### Tablet layout (768px – 1023px)

Both panels become slide-in drawers triggered by header buttons. The conversation area is the primary view.



Mobile layout (< 768px)

Single-column; history and conversation panels as full-height bottom-sheet drawers; input pinned to bottom of screen above the keyboard.



Mobile-specific requirements

| Requirement    | Specification                                                                                                                              |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Input controls | All controls (attachment, tool selection, model chip, send/stop) are thumb-reachable — no controls hidden behind secondary menus on mobile |

| Requirement          | Specification                                                                                                           |
|----------------------|-------------------------------------------------------------------------------------------------------------------------|
| @ -binding typeahead | Anchors to the bottom of the viewport (not cursor position); sized so the top edge is visible above the raised keyboard |
| Mermaid diagrams     | Horizontally scrollable — never scaled to illegibility                                                                  |
| Vega-Lite charts     | <code>width: "container"</code> for responsive sizing                                                                   |
| Data tables          | Horizontally scrollable; first column is sticky                                                                         |
| Conversation panel   | Opens as full-height bottom sheet — accessible via a tray icon in the input area footer                                 |
| History panel        | Opens as full-height bottom sheet — accessible via a history button                                                     |
| Conversation search  | Opens as full-screen overlay with keyboard raised                                                                       |
| Touch targets        | Minimum 44 × 44 px (Apple HIG / Material Design)                                                                        |

## Input area on mobile

The input toolbar presents all controls in a **single row above the text field**, within thumb reach. When horizontal space is tight, secondary controls (model chip, profile indicator) collapse behind an overflow icon — the paperclip (attachment) and send/stop button are **never** moved to overflow.

| Control                | Mobile behaviour                                                             |
|------------------------|------------------------------------------------------------------------------|
| Paperclip / attachment | Opens the native mobile file picker (camera roll, Files app, camera capture) |
| Tool selection         | Opens a full-screen bottom sheet listing available MCP tools                 |
| Model chip             | Opens a bottom sheet listing allowed models                                  |
| Text input             | Grows to 3 lines maximum on mobile (vs. 5 on desktop)                        |
| Submit                 | Large dedicated tap target — always visible                                  |

## Keyboard and viewport behaviour

When the software keyboard raises on mobile, the conversation thread scrolls to keep the latest turn visible above the input area. The input area stays pinned to the top of the keyboard. The @ -binding typeahead panel appears above the keyboard.

## Hover states on mobile

| Desktop (hover)                                | Mobile equivalent                                    |
|------------------------------------------------|------------------------------------------------------|
| Report icon appears on hover over a turn       | Always visible below each turn                       |
| Edit icon appears on hover over a user message | Always visible below each user message               |
| Assistant response action row appears on hover | Always visible below each assistant response         |
| Metadata line appears on hover                 | Tap the response bubble to reveal; tap again to hide |

## Gestures

| Gesture                             | Action                                                         |
|-------------------------------------|----------------------------------------------------------------|
| Swipe right from left edge          | Opens the history panel bottom sheet                           |
| Swipe left from right edge          | Opens the conversation panel bottom sheet                      |
| Long press on a user message        | Opens message action menu (edit, report, copy)                 |
| Long press on an assistant response | Opens response action menu (copy, report, regenerate)          |
| Tap a binding chip                  | Fires <code>binding-click</code> event to the host application |
| Pinch-to-zoom on a Mermaid diagram  | Enters full-screen diagram view                                |

## Conversation thread anatomy

### User messages

- Appear immediately in the conversation thread on submit (**optimistic UI**) — the message is visible before the API acknowledges receipt. If the send fails, the message is styled as failed with an inline retry control.
- Right-aligned in a muted bubble
- `@`-binding chips render as interactive pills within the bubble
- In shared conversations: author's name and avatar above the bubble

### Other participants' messages (shared conversations only)

- Left-aligned
- Distinct colour per participant (derived from host primary brand colour)
- Author name and avatar above the bubble

### Message timestamps

Every message in the conversation thread carries a **timestamp**.

| Display rule      | Format                                                      |
|-------------------|-------------------------------------------------------------|
| Default           | Relative: "Just now", "3 min ago", "Yesterday", "Mon 09:14" |
| Hover (desktop)   | Absolute: "Monday 11 May 2026, 09:14:32"                    |
| Older than 7 days | Absolute date always shown: "4 May, 09:14"                  |

Timestamps are shown below each message bubble (user and assistant). They are not shown during active streaming — the timestamp appears when streaming completes.

Thinking indicator

After the user submits a message and before the first streaming token arrives, the assistant displays a **thinking indicator** in the conversation thread:



An animated three-dot pulse appears beneath a placeholder response bubble. The indicator is replaced immediately by the first streaming token. If the model is queuing tool calls before generating prose, the thinking indicator transitions to the tool call disclosure card (in-progress state) without a gap.

The thinking indicator is suppressed if the model begins streaming within 300 ms of submission.

Full thread anatomy

A complete exchange showing all thread elements in sequence:

Conversation area

Show me the governance health for @{{Finance Domain}}

Just now

Atlas is thinking...

Governance Platform · get\_domain\_health

The Finance Domain is in **good health** overall.

|               |          |
|---------------|----------|
| Metric        | Status   |
| Coverage      | 94 %     |
| Quality score | 87 / 100 |
| Open issues   | 3        |

<sup>1</sup> Governance Platform · get\_domain\_health

+ Standard · 1,240 tokens · 3 min ago

👍

👎

🔄 Regenerate

📋 Copy

What are the open quality issues?

Who owns the Finance domain?

Show me a breakdown by sub-domain

📎

⚙️

[+ Standard]

[input...]

[→]

← User message bubble (right-aligned)

← Timestamp

← Thinking indicator

← Tool call disclosure (collapsed)

← Assistant response

← Inline source citation

← Metadata (hover/tap)

← Feedback row

← Suggested follow-ups

63

Per-turn feedback

User messages carry a **report icon** (visible on hover / always visible on mobile). Clicking opens the report dialog:

What's wrong with this?

[Text field — optional explanation]

[Submit report] [Cancel]

On submit, an improvement signal is captured against the specific turn (see [12-continuous-improvement.md](#)).

Assistant responses carry a feedback row with three controls:

| Control            | Desktop          | Mobile                            | Action                                                                                                         |
|--------------------|------------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------|
| 👍 Thumbs up        | Appears on hover | Always visible below the response | Records a positive improvement signal against the turn. One-click — no dialog. Fills on selection; toggleable. |
| 👎 Thumbs down      | Appears on hover | Always visible below the response | Opens the report dialog (same as user message report). Records a negative signal on submit.                    |
| Regenerate         | Appears on hover | Always visible below the response | Creates a new branch from this turn (see doc 04).                                                              |
| Copy full response | Appears on hover | Always visible below the response | Copies the full response text to clipboard.                                                                    |

Both thumbs signals feed the improvement pipeline (doc 12). Thumbs up signals are used to identify high-quality turns for reference; thumbs down signals trigger the same improvement signal workflow as the explicit report.

Assistant responses

Each assistant response carries:

| Element                           | Desktop                                                                                     | Mobile                                 |
|-----------------------------------|---------------------------------------------------------------------------------------------|----------------------------------------|
| Metadata line                     | Appears on hover — model label + token counts + (in shared sessions) submitting user's name | Revealed on tap of the response bubble |
| Feedback row (👍👎 Regenerate Copy) | Appears on hover                                                                            | Always visible below the response      |
| Artefact chips                    | "Added to artefacts ↗" beneath each generated block                                         | Same                                   |

Suggested follow-ups

At the end of each assistant response, up to **three suggested follow-up queries** are presented as pill chips generated by the model. Single-click to submit. Follow-ups are model-generated — not from a static list.



### Input area controls

| Control             | Function                                                                                                        |
|---------------------|-----------------------------------------------------------------------------------------------------------------|
| Paperclip icon      | Opens file picker or accepts drag-and-drop (PDF, Excel, Word, images); images also accepted via clipboard paste |
| Tool selection icon | Opens MCP tool opt-in panel                                                                                     |
| Active model chip   | Opens model selection popover                                                                                   |
| Text input          | Multi-line, grows to 5 lines, <code>Cmd/Ctrl + Enter</code> to submit                                           |
| Profile indicator   | Read-only <code>Style · Verbosity</code> link to host app profile settings — clickable                          |
| Send / Stop         | Submits message or stops active stream                                                                          |
| Share icon          | Opens participant management panel                                                                              |

### Input area layout

@{Finance Domain} What are the quality gaps in this domain? Any open ownership issues?

[+ Standard] [Technical · Concise] [] [→]

Attachment — file picker / drag-and-drop / clipboard paste

Tool selection — opens MCP tool opt-in panel (see below)

Standard — model tier chip; opens model selection popover

Technical · Concise — profile indicator (style · verbosity); read-only link

Share — opens participant management panel

→ Send / Stop — always visible; submits or stops active stream

← Text input (grows to 5 lines)

← Controls row

While streaming, the input field is disabled and the Send button becomes a Stop button:

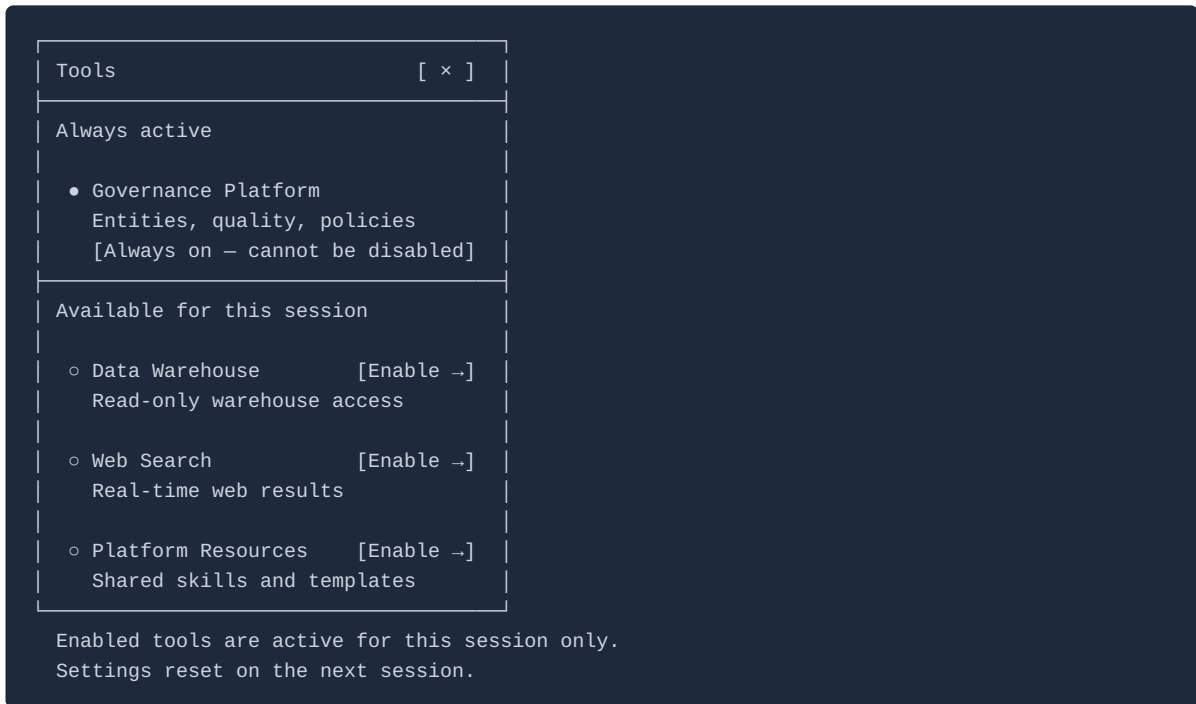
[input disabled while streaming]

[+ Standard] [Technical · Concise] [] [■]

← ■ Stop

### Tool selection panel

Opened via the icon in the input area:



## Session artefact tray

The artefact tray accumulates every input and output artefact produced during a conversation. It persists for the full lifetime of the conversation record.

### Artefact classes

| Class              | Direction | Examples                                                                 |
|--------------------|-----------|--------------------------------------------------------------------------|
| Attached document  | Input     | PDF report, Excel extract, Word document                                 |
| Generated document | Output    | Markdown report, summary                                                 |
| Mermaid diagram    | Output    | Relationship diagram, flow, sequence                                     |
| Vega-Lite chart    | Output    | Bar chart, trend line, distribution                                      |
| Data table         | Output    | Multi-row query result (CSV)                                             |
| JSON export        | Output    | Tool result or entity record                                             |
| Code block         | Output    | SQL, Python, YAML, shell                                                 |
| Binding reference  | Input     | Non-downloadable — reference card linking to the host application object |

### Tray entry anatomy

Each tray entry shows: - Format icon - Auto-generated name: `[Content type] – [Subject] – [Date]` - Turn back-link (click to jump to the source turn) - Direction label (Input / Output) - Download button - Preview link

Users may **rename** tray entries. A **"Download all"** control packages all artefacts as a zip archive.

Canvas documents in the artefact tray carry an additional **"Open in canvas"** action that re-opens the document in the canvas panel for further editing, regardless of which turn produced it.

## Document canvas

The canvas is the platform's working-document surface — a persistent, editable panel where the model and user collaborate iteratively on a document across multiple turns, rather than treating each AI response as a disposable message.

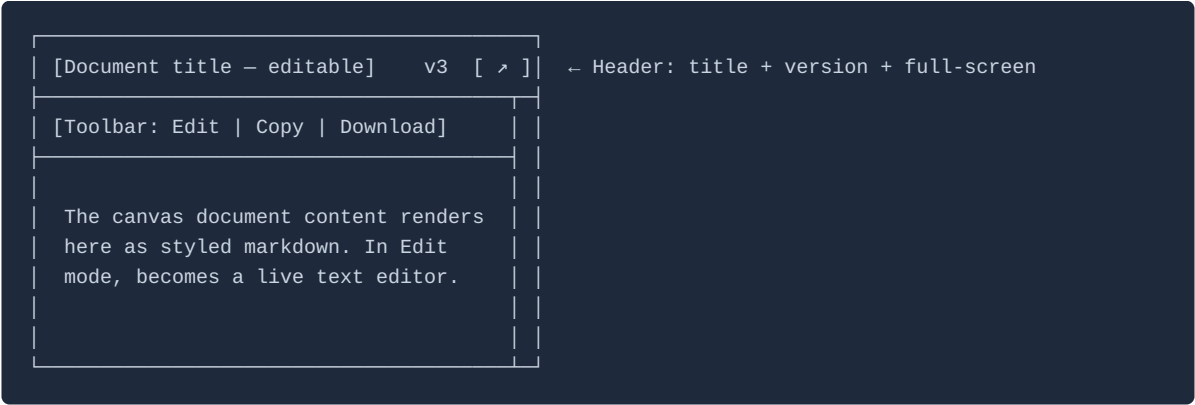
### What opens in canvas

The model uses a ````document` fenced block to produce canvas-eligible output. The platform's non-overridable system prompt layer instructs the model to use `document` blocks for substantial, document-like prose outputs — reports, summaries, plans, policy drafts, specifications, structured analyses — where the user is likely to want to refine rather than simply read and move on.

Non-prose outputs (Mermaid diagrams, Vega-Lite charts, data tables, code blocks, JSON) remain as inline thread content and artefact tray entries. They do not open in canvas.

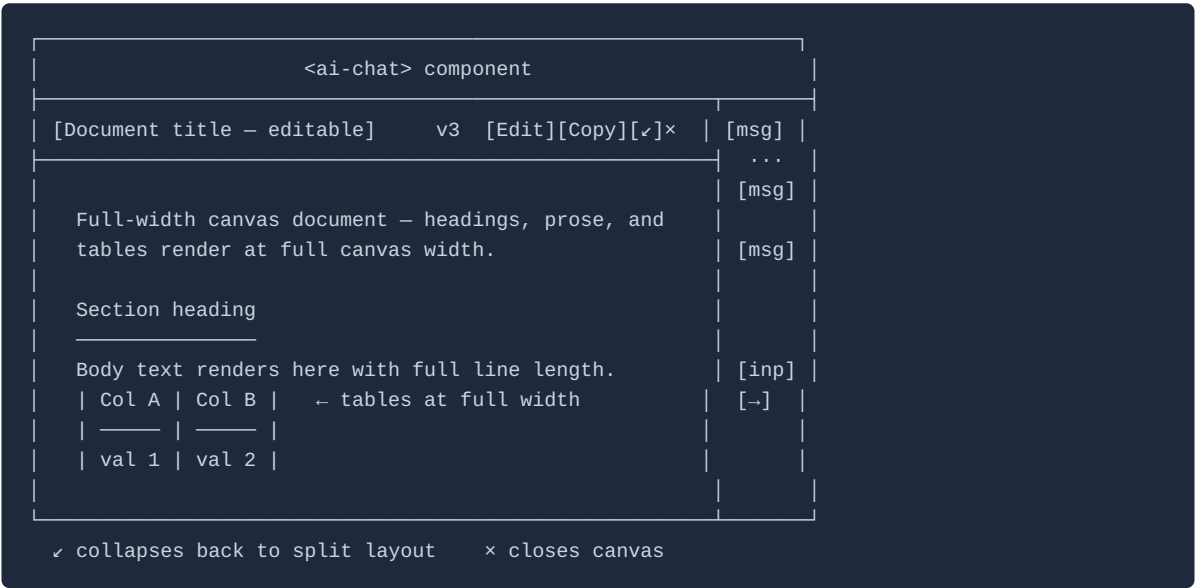
Users may also manually promote any text artefact in the tray to canvas by clicking **"Open in canvas"** on that tray entry.

### Canvas panel layout



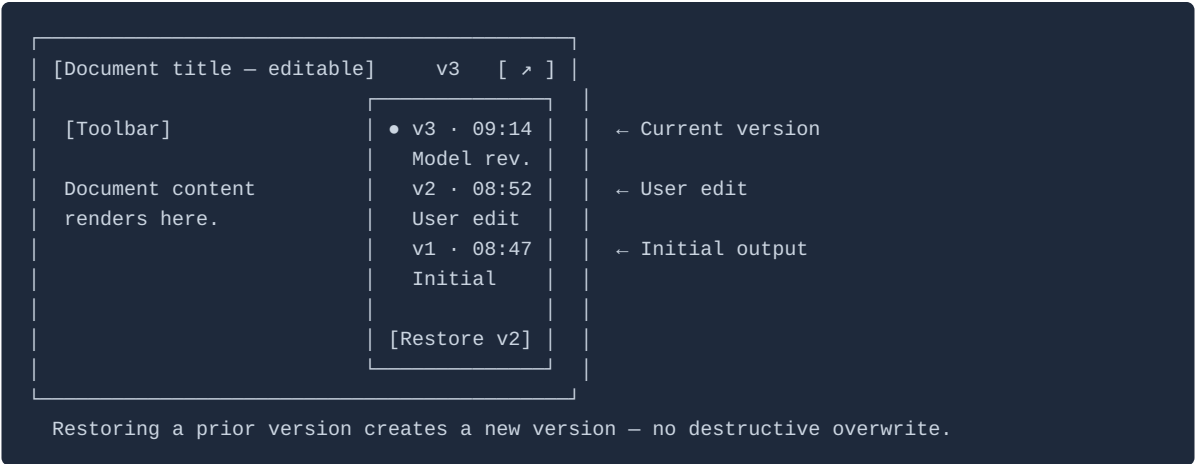
| Element            | Behaviour                                                                                                                                                                                                                               |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Document title     | Auto-generated from document content; editable inline by clicking                                                                                                                                                                       |
| Version indicator  | Shows current version (v1, v2 ...); clicking opens a version history dropdown listing every model-generated and user-edited version with timestamp. Selecting a prior version restores it as a new version (no destructive overwrites). |
| Full-screen toggle | Expands canvas to fill the component mount point; the conversation area collapses to a narrow strip. Second click restores the split layout.                                                                                            |

Canvas full-screen layout



Canvas version history

Clicking the version indicator (e.g. **v3**) opens a dropdown listing every version of the document:



| **Edit mode** | Clicking **Edit** transforms the canvas from a read-only rendered view into a live markdown editor. User edits are saved immediately as a new version. Exiting edit mode re-renders the markdown. || **Copy** | Copies the full markdown source to the clipboard || **Download** | Downloads the document as `.md` (default), `.txt`, or `.pdf` (platform-rendered) |

In-thread reference card

When the model produces a `document` block, the conversation thread shows a compact **reference card** in place of the full content:



The full document is in the canvas panel — it is not rendered inline in the thread. This keeps the conversation thread scannable and avoids large content blocks interrupting the exchange.

Model-assisted revision

When a canvas document is open, the platform automatically includes the current canvas content in the model's context:

```
[Canvas document – Q2 Governance Summary]
{full document markdown}
[End canvas document]
```

The user can ask the model to revise the document in natural language: *"Shorten the executive summary to two sentences"*, *"Add a risks section after the findings"*, *"Rewrite this in a more formal tone"*. The model responds with a new `document` block containing the full revised document, which replaces the canvas content and is saved as a new version. The model's reply in the conversation thread confirms what changed: *"Updated — shortened the executive summary and added a risks section."*

Multiple canvases

A conversation may produce more than one canvas document. Each is a separate tab within the canvas panel, ordered by creation time. The active tab is the most recently updated document.

Q2 Governance Summary | Risk Register ▾

[Edit] [Copy] [Download] v3 [↗]

Q2 Governance Summary content  
(active tab)

← Canvas tabs (+ overflow menu)

Switching tabs does not close the inactive canvas – both remain in context.

Canvas in Mode 1 (floating widget)

The canvas panel is only available in **full state**. In the mini state, canvas documents show as reference cards in the thread with an *"Expand for canvas view ↗"* note. Transitioning to full state opens the canvas panel for the active document.

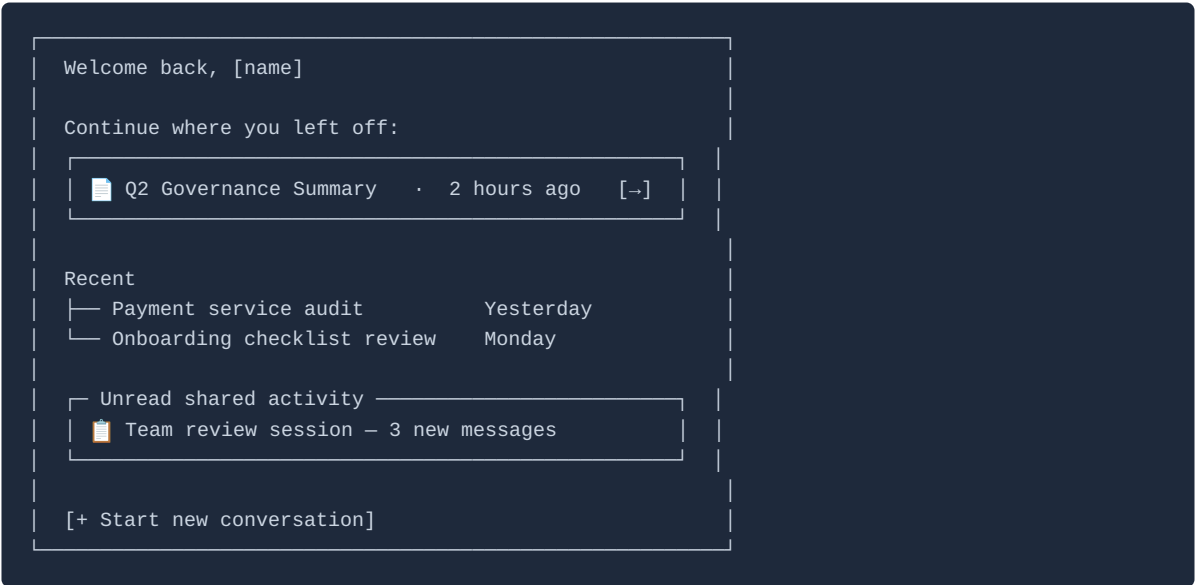
Canvas and the artefact tray

All canvas document versions are stored in the artefact tray as versioned entries. The tray shows the latest version with a version count badge (e.g. `v4`). Individual versions are downloadable from the version history dropdown in the canvas panel.

## Returning user state

### Mode 2 — Inline Page ( `<ai-chat>` )

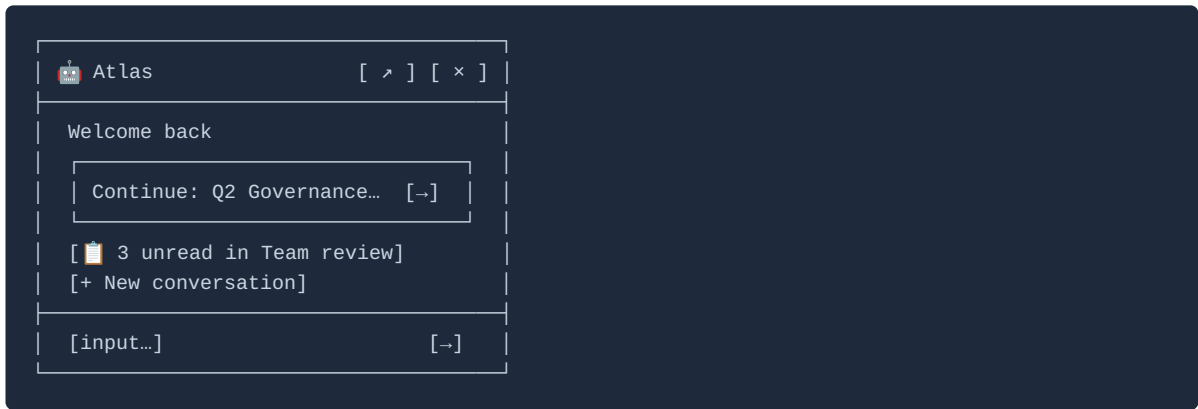
When the component mounts and the authenticated user has prior conversation history, the **conversation area** shows a home state before opening any conversation. The home state is shown on the **first mount per browser session** — subsequent navigations to the page within the same session resume the last-active conversation directly.



| Element                | Specification                                                                                                                |
|------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Continue card          | The single most recently active conversation; single click opens it directly                                                 |
| Recent list            | Up to 2 further recent conversations; click to open                                                                          |
| Unread shared activity | Shown only if there are unread messages in shared conversations; click to open the conversation at the first unread turn     |
| Start new conversation | Clears to a new conversation with the onboarding welcome state (first-time) or an empty input (returning)                    |
| Trigger condition      | Shown when the user has $\geq 1$ prior conversation and has not interacted with the component in the current browser session |
| Skip condition         | Not shown if the user is deep-linked to a specific conversation via the <code>initial-conversation-id</code> attribute       |

### Mode 1 — Floating Widget ( `<ai-chat-widget>` )

When the widget transitions from collapsed FAB to mini state for the first time in a browser session — and the user was last active more than **4 hours** ago (configurable via `widget.returningUserThreshold` in hours) — the mini panel opens to a compact returning-user card rather than resuming mid-conversation:



The card auto-dismisses when the user clicks any option or starts typing in the input field. If the threshold has not been exceeded (recent session), the widget opens directly into the last conversation.

## Onboarding state

On **first visit**, the conversation area shows a welcome state drawn from the host application config:

1. The host-configured `identity.assistantDescription`
2. Starter workflows from `features.starterWorkflows` (up to 3, shown as question-style prompts)
3. Starter questions from `features.starterQuestions` (up to 3, if no starter workflows or alongside them)
4. A link to the full Workflow Library

The onboarding state is shown once per user — it is not shown again once the user has started a conversation.

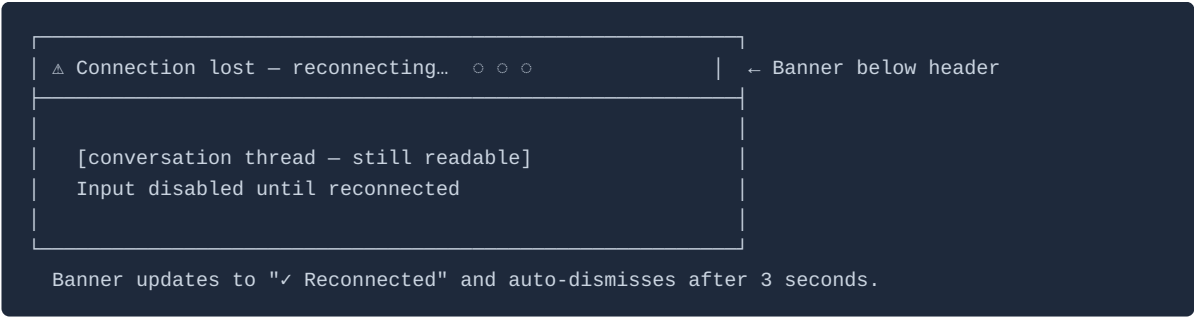
## Error and boundary states

| State                  | Presentation                                                                                                                                                                                               | User action                        |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| Model timeout          | Inline error card with retry button                                                                                                                                                                        | Retry re-submits the last message  |
| Send failure (network) | Failed user message styled with error colour + inline retry icon                                                                                                                                           | Tap/click retry to resend          |
| Connection lost        | Non-blocking banner below the conversation header: <i>"Connection lost — reconnecting..."</i> ; animated reconnect indicator. Banner updates to <i>"Reconnected"</i> (auto-dismisses after 3s) on restore. | Wait; drafts are preserved locally |
| MCP tool failure       | Tool call disclosure shows error status and raw detail                                                                                                                                                     | Rephrase or copy error to report   |

| State                            | Presentation                                              | User action                                                   |
|----------------------------------|-----------------------------------------------------------|---------------------------------------------------------------|
| Always-on MCP server unavailable | Degraded-mode banner; session continues in text-only mode | Model answers from system prompt context only                 |
| Opt-in MCP server unavailable    | Error in tool call disclosure                             | Disable the failing server for the session                    |
| Context window limit (80%)       | Subtle warning in conversation header                     | Branch to new thread or accept auto-summarisation             |
| Out-of-scope query               | Model explains scope boundary and suggests reformulation  | No error state — graceful redirect                            |
| Auth session expired             | Modal overlay prompting re-authentication                 | Re-auth restores session and full conversation history        |
| Unsupported file type            | Inline notice on file selection                           | User prompted to use a supported format                       |
| Last participant constraint      | Tooltip on disabled Leave/Remove controls                 | Invite another user before leaving                            |
| Component mount failure          | Error state within the component mount point              | Host application is notified via the <code>error</code> event |

Error state layouts

Connection lost — non-blocking banner:



Always-on MCP server unavailable — degraded-mode banner:



⚠ Governance Platform is unavailable. Atlas is answering from its own knowledge only – data may not reflect the current state of your application.

[conversation thread – text-only mode]

Persistent until connectivity is restored.

**Auth session expired — blocking modal:**

Session expired

Please sign in again to continue. Your conversation history is preserved and will be restored on sign-in.

[Sign in again]

Full-viewport modal – no interaction behind it.

**Send failure — inline retry on user message:**

What are the quality gaps in the Finance domain?

⚠ Failed to send [🔄 Retry]

← Inline retry

**Context window warning (80% threshold) — subtle header indicator:**

Atlas

⚠ Long conversation – 83% full  
[Branch to new thread]

← Header indicator

**Accessibility**

WCAG 2.1 AA compliance is a platform requirement. The following are platform-specific additions.

| Requirement                     | Specification                                                                                                                             |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Mermaid SVG alt text            | Descriptive alt text generated by the model for each Mermaid diagram                                                                      |
| Conversation thread live region | The thread is an <code>aria-live</code> region — streaming responses and new shared conversation messages are announced to screen readers |

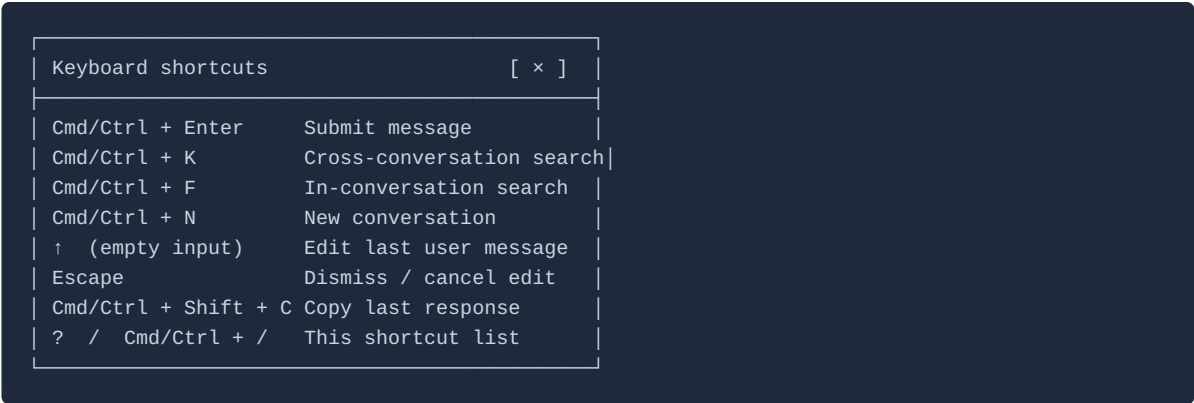
| Requirement             | Specification                                                                                                                                                                                                                                                                                                                                                                  |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| @-binding typeahead     | Fully keyboard-navigable (arrow keys, Enter/Tab to select, Escape to dismiss); focus returns to input field on selection or dismissal                                                                                                                                                                                                                                          |
| Branding token contrast | Platform validates that host-provided colour tokens meet WCAG 2.1 AA contrast ratios at config submission                                                                                                                                                                                                                                                                      |
| prefers-reduced-motion  | When the OS-level reduced motion preference is active: streaming text renders immediately (no character-by-character animation); panel slide transitions are instant; the thinking indicator uses a static label instead of the animated three-dot pulse; the FAB pulse animation on stream-in-progress is suppressed; Mermaid and Vega-Lite render-in animations are disabled |
| Focus management        | When a shared conversation message arrives, focus is not stolen from the input field. The unread badge and "Jump to latest" button are announced via the live region. Focus only moves to new content on explicit user action.                                                                                                                                                 |
| Modal focus trap        | All modal overlays (full widget state, diagram full-screen, write confirmation) implement a focus trap — Tab cycles only within the modal; Escape dismisses                                                                                                                                                                                                                    |

## Jump to latest

When a user has scrolled up and new content arrives (streaming response or shared conversation message), a **"Jump to latest ↓"** pill button appears anchored above the input field. Clicking it scrolls immediately to the most recent turn. The button disappears when the user is already at the bottom.

## Keyboard shortcuts

The shortcut reference overlay (triggered by `?` or `Cmd/Ctrl + /`):



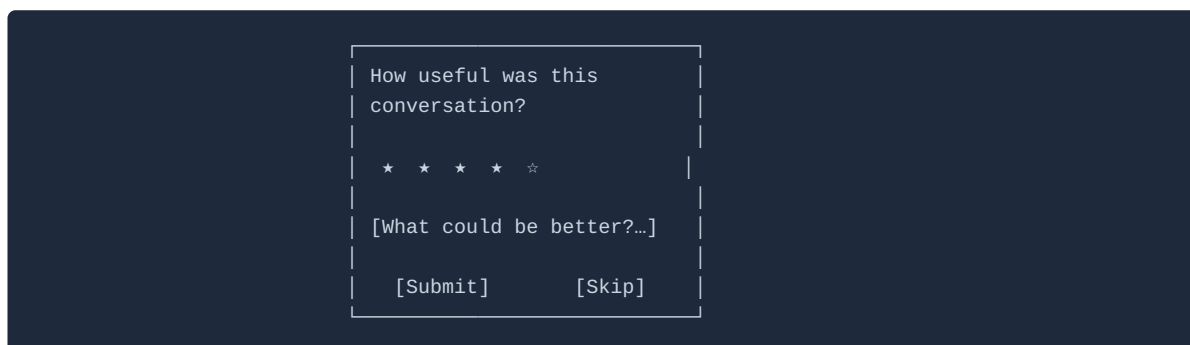
| Shortcut         | Action                         |
|------------------|--------------------------------|
| Cmd/Ctrl + Enter | Submit message                 |
| Cmd/Ctrl + K     | Open cross-conversation search |

| Shortcut             | Action                                                |
|----------------------|-------------------------------------------------------|
| Cmd/Ctrl + F         | Open in-conversation search                           |
| Cmd/Ctrl + N         | Start a new conversation                              |
| Escape               | Cancel edit / dismiss typeahead / close modal         |
| ↑ in empty input     | Edit most recent user message                         |
| Arrow keys           | Navigate @-binding typeahead; navigate search results |
| Enter / Tab          | Select item in @-binding typeahead                    |
| Cmd/Ctrl + Shift + C | Copy most recent assistant response to clipboard      |
| ? or Cmd/Ctrl + /    | Show keyboard shortcut reference                      |

## Post-session CSAT prompt

A **1–5 star rating prompt** is shown to a random sample of users (configured via `features.csatSampleRate`) at the end of a session, after 30 seconds of idle time following the last assistant response.

The prompt appears as a **non-blocking floating card** anchored to the bottom-right of the conversation area:



### How useful was this conversation?

★★★★★

[Optional: What could be better? — single text line]

[Submit] [Skip]

- Non-blocking — does not prevent input
- Dismissible (Skip or click outside)
- Skipping is not counted as a negative signal
- On mobile: appears as a bottom sheet centred in the viewport

The rating and optional comment are stored in `assistant.conversations.csat_score` and `assistant.conversations.csat_comment`.

# 10 — Content Rendering

## Rendering decision rules

The rendering engine evaluates each content block in an assistant response in priority order. **The first matching rule wins.**

| Priority | Trigger                                                                                   | Rendered as                                                                       |
|----------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| 1        | Tool call event ( <code>mcp_tool_use</code> / <code>mcp_tool_result</code> )              | <b>Tool Call Disclosure</b> card                                                  |
| 2        | Fenced block tag matching a registered host renderer ( <code>renderers[.trigger]</code> ) | <b>Custom host renderer</b> — host-provided ES module; see below                  |
| 3        | Fenced block tagged <code>```document</code>                                              | <b>Document canvas</b> — opens in right panel canvas; reference card in thread    |
| 4        | Fenced block tagged <code>```mermaid</code>                                               | <b>Mermaid diagram</b> — SVG, expandable, exportable                              |
| 5        | Fenced block tagged <code>```vega-lite</code>                                             | <b>Vega-Lite chart</b> — interactive, responsive                                  |
| 6        | Fenced block tagged <code>```math</code> or <code>\$\$...\$\$ display block</code>        | <b>Math expression</b> — KaTeX rendered display block                             |
| 7        | Fenced block tagged <code>```json</code>                                                  | <b>JSON inspector</b> — collapsible tree, copy-to-clipboard                       |
| 8        | Fenced block tagged <code>```csv</code> or <code>```table</code>                          | <b>Data table</b> — sortable, filterable, paginated, CSV export                   |
| 9        | Any other fenced block                                                                    | <b>Syntax-highlighted code</b> — Prism, copy-to-clipboard, line numbers > 5 lines |
| 10       | Inline <code>\$. .\$.</code> within prose                                                 | <b>Inline math</b> — KaTeX rendered inline                                        |
| 11       | All other content                                                                         | <b>Rich markdown prose</b> — GFM                                                  |

The system prompt (injected by the platform) instructs the model to:

- Prefer structured outputs — Vega-Lite for metrics and trends, Mermaid for relationships and flows, data tables for entity lists — over prose equivalents when the data supports it
- Use `document` blocks for substantial prose outputs (reports, summaries, plans, policy drafts, analyses) where the user is likely to iterate across multiple turns rather than simply read once

## Content type reference

| Content type         | Trigger                                      | Typical use cases                                                                                         |
|----------------------|----------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Prose / markdown     | Default                                      | Explanations, summaries, narrative answers                                                                |
| Document canvas      | <code>```document</code>                     | Reports, policy drafts, structured summaries, plans — any substantial prose the user will iterate on      |
| Custom host renderer | Registered <code>trigger</code> tag          | Host-defined domain-specific visualisations (risk gauges, compliance scorecards, Gantt views, org charts) |
| Mermaid diagram      | <code>```mermaid</code>                      | Entity relationships, process flows, system dependencies, hierarchies                                     |
| Vega-Lite chart      | <code>```vega-lite</code>                    | Metrics, trends, distributions, comparisons                                                               |
| Math expression      | <code>```math / \$\$...\$\$ / \$...\$</code> | Scoring formulas, ratios, statistical expressions, metric definitions                                     |
| JSON inspector       | <code>```json</code>                         | Raw tool results, configuration objects, structured data                                                  |
| Data table           | <code>```csv / ```table</code>               | Multi-row query results, lists, comparison tables                                                         |
| Tool call disclosure | Automatic                                    | All MCP tool invocations — always visible, collapsed by default                                           |
| Syntax code          | All other fenced blocks                      | SQL, Python, YAML, shell, TypeScript                                                                      |

## Custom host renderers

Host applications may register custom content renderers in the `renderers` section of their application config (see [01-host-application-config.md](#)). When the model produces a fenced block tagged with a registered `trigger`, the platform loads and invokes the host's renderer module.

### Module loading

Renderer modules are **ES modules** loaded once per session when the first block matching their trigger arrives. The platform loads the module using a dynamic `import()` from the registered `moduleUrl`. Modules are cached for the session lifetime — they are not re-fetched on each block.

The module must export a **renderer class** (or factory function returning an object) that implements the following interface:

```

interface HostRenderer {
  // Called once per content block after the full fenced block content is received.
  // `container` is a plain HTMLElement inside a shadow root – render freely within it.
  render(container: HTMLElement, content: string, context: RendererContext): void |
  Promise<void>;

  // Optional – called when the rendered element is removed from the DOM.
  dispose?(): void;
}

interface RendererContext {
  tenantId: string; // the current tenant
  conversationId: string; // the current conversation
  turnId: string; // the turn this block belongs to
  trigger: string; // the fenced block tag that matched
  themeTokens: Record<string, string>; // host brand CSS custom properties (e.g. "--color-
  primary")
}

```

The platform instantiates the renderer class once per content block, passes the full fenced block content as a string to `render()`, and calls `dispose()` when the block leaves the DOM (e.g. conversation navigation, component unmount).

## Isolation

Each rendered block's `container` is the interior of a **shadow root** — the renderer's DOM and styles are isolated from the platform UI and from other rendered blocks. The renderer may use any DOM APIs available to the module. It may not access the platform's internal state, the user's JWT, or other conversations.

## Streaming behaviour

Custom renderers receive the **full fenced block content** after `content_block_stop` — they cannot stream incrementally. While the block is buffering, the platform shows a loading skeleton in place of the renderer output. The skeleton is replaced by the rendered output on receipt.

## System prompt guidance injection

For each registered renderer, the platform injects the renderer's `systemPromptGuidance` (or a generated fallback) into the system prompt at session start. This tells the model when and how to produce the custom content type:

```

[Custom content types]
{renderer.name}: {renderer.systemPromptGuidance}

```

All registered renderer guidance blocks are appended together in the order they appear in the `renderers` config array. They are injected after tool descriptions and before memory blocks in the assembled system prompt.

## Fallback behaviour

If the renderer module fails to load, or if `render()` throws, the platform:

1. Logs the error to the improvement signal pipeline
2. Renders the fenced block content as a **syntax-highlighted code block** (priority 8)
3. Shows an inline notice beneath the block: *"[Renderer name] could not render this content. Showing raw output."*

The fallback is transparent to the user and non-blocking — the rest of the response continues rendering normally.

## Artefact tray

Custom-rendered blocks are added to the artefact tray as downloadable raw content (the fenced block source, stored as plain text with the trigger name as the format label). The renderer name and trigger are shown in the tray entry. If the renderer exposes a `getExportBlob(): Promise<Blob>` method, the platform calls it on download and uses the returned blob in preference to the raw source — allowing the renderer to export a rendered image or structured file.

## Example: risk gauge renderer

Given the config:

```
{
  "id": "risk-gauge",
  "trigger": "risk-gauge",
  "name": "Risk Gauge",
  "moduleUrl": "https://cdn.acme.com/ai-renderers/risk-gauge.js",
  "systemPromptGuidance": "Use a ``risk-gauge block when asked for a risk score. The block must contain JSON: { \"score\": <0-100>, \"label\": \"<text>\", \"breakdown\": [ { \"factor\": \"<name>\", \"score\": <0-100> } ] }."
}
```

The model produces:

```
``risk-gauge
{ "score": 72, "label": "Medium-High", "breakdown": [
  { "factor": "Data quality", "score": 80 },
  { "factor": "Ownership coverage", "score": 55 },
  { "factor": "Policy compliance", "score": 90 }
]
}
``
```

The platform calls `RiskGaugeRenderer.render(container, content, context)`. The renderer parses the JSON and renders a gauge widget into `container`. The raw JSON is stored in the artefact tray.

## Mermaid diagrams

| Behaviour       | Specification                                                                   |
|-----------------|---------------------------------------------------------------------------------|
| Rendering       | SVG via Mermaid.js                                                              |
| Default display | Full-width in conversation thread, collapsed to thumbnail in artefact tray      |
| Expand          | Full-screen overlay available                                                   |
| Export          | SVG download from artefact tray                                                 |
| Mobile          | Horizontally scrollable — never scaled to illegibility                          |
| Accessibility   | Descriptive alt text generated by the model and attached to the SVG             |
| Artefact        | Mermaid source stored in turn record; added to artefact tray on render complete |

## Common Mermaid diagram types

| Diagram type        | Mermaid syntax               | Typical trigger                            |
|---------------------|------------------------------|--------------------------------------------|
| Entity relationship | <code>erDiagram</code>       | "Show me the relationship between X and Y" |
| Process flow        | <code>flowchart LR</code>    | "Walk me through the approval workflow"    |
| Sequence diagram    | <code>sequenceDiagram</code> | "How do these services communicate?"       |
| Hierarchy / tree    | <code>graph TD</code>        | "Show me the organisational structure"     |
| Timeline            | <code>timeline</code>        | "Show the project milestones"              |

## Vega-Lite charts

| Behaviour         | Specification                                                                   |
|-------------------|---------------------------------------------------------------------------------|
| Rendering         | Interactive via vega-embed                                                      |
| Responsive sizing | <code>width: "container"</code> — adapts to viewport or container               |
| Export            | PNG or SVG download from artefact tray                                          |
| Interactivity     | Hover tooltips, click-to-filter where applicable                                |
| Mobile            | Responsive; minimum bar/point size maintained for touch targets                 |
| Artefact          | Vega-Lite spec stored in turn record; added to artefact tray on render complete |

## Math expressions

Mathematical expressions are rendered using **KaTeX** — fast, lightweight, and browser-native.



| Behaviour     | Specification                                                                                                       |
|---------------|---------------------------------------------------------------------------------------------------------------------|
| Display block | Triggered by <code>```math</code> fenced block or <code>\$\$...\$\$</code> delimiter — rendered full-width, centred |
| Inline        | Triggered by <code>\$...\$</code> within prose — renders inline without breaking text flow                          |
| Library       | KaTeX — subset of LaTeX math notation                                                                               |
| Copy          | LaTeX source copy-to-clipboard button on display blocks                                                             |
| Export        | LaTeX source and rendered SVG downloadable from artefact tray                                                       |
| Mobile        | Display blocks horizontally scrollable at narrow viewports                                                          |
| Fallback      | If KaTeX fails to parse, the raw LaTeX source is shown in a code block with a parse-error notice                    |

## JSON inspector

| Behaviour     | Specification                                                          |
|---------------|------------------------------------------------------------------------|
| Default state | Collapsed to two levels                                                |
| Navigation    | Expand/collapse nodes; copy individual values or subtrees              |
| Export        | Full JSON download from artefact tray                                  |
| Use cases     | Tool result payloads, configuration inspection, structured record data |

## Data tables

| Behaviour   | Specification                                    |
|-------------|--------------------------------------------------|
| Sorting     | Click column header to sort ascending/descending |
| Filtering   | Inline filter row per column                     |
| Pagination  | 25 rows per page; configurable                   |
| Mobile      | Horizontally scrollable; first column sticky     |
| Export      | CSV download from artefact tray                  |
| Empty state | "No results" message with query context          |

## Syntax-highlighted code

| Behaviour    | Specification                                          |
|--------------|--------------------------------------------------------|
| Highlighting | Prism.js — auto-detects language from fenced block tag |

| Behaviour    | Specification                                                                |
|--------------|------------------------------------------------------------------------------|
| Copy         | Copy-to-clipboard button always visible                                      |
| Line numbers | Shown for blocks of more than five lines                                     |
| Languages    | SQL, Python, YAML, TypeScript, JSON, Bash, and all Prism-supported languages |

## Attached document and image display

**Non-image documents** (PDF, Excel, Word) appear in the user message bubble as a labelled file card: - Format icon - File name - Page count (PDF), sheet count (Excel), or section count (Word)

Non-image documents are **not rendered inline**. When the model references a specific section, it cites by page number (PDF), sheet name (Excel), or heading (Word).

**Images** (PNG, JPEG, WEBP) are **rendered inline** in the user message bubble at a constrained size (max 320px wide, max 240px tall, maintaining aspect ratio). Multiple images in one turn stack vertically. Clicking an image opens it in a full-screen lightbox overlay. The original file is downloadable from the lightbox and from the artefact tray.

## Inline source citations

When the model's response draws on data returned by an MCP tool call, it should cite the source inline using a numbered superscript that links to the corresponding tool call disclosure card.

| Element            | Specification                                                                                                         |
|--------------------|-----------------------------------------------------------------------------------------------------------------------|
| Format             | Superscript numeral in the prose (e.g. <i>"There are 12 entities in this domain<sup>1</sup>"</i> )                    |
| Anchor             | Clicking the superscript scrolls to and expands the referenced disclosure card                                        |
| Scope              | Citations reference the specific MCP tool invocation, not a generic tool name                                         |
| Multiple sources   | Where a claim draws on more than one tool call, multiple citations appear (e.g. <i>"...entities<sup>1 2</sup>"</i> )  |
| Attached documents | Citations to attached documents use the document's cite format: page number (PDF), sheet name (Excel), heading (Word) |

Citations are rendered as part of the markdown prose block. Superscript numbers reset per response.

## Streaming behaviour

| Content type     | Streaming behaviour                                   |
|------------------|-------------------------------------------------------|
| Prose / markdown | Streams character-by-character; renders incrementally |

| Content type                                              | Streaming behaviour                                                                                           |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Non-prose blocks (Mermaid, Vega-Lite, JSON, tables, code) | Buffers internally; renders on <code>content_block_stop</code> to prevent hydration errors on partial content |
| Tool call disclosures                                     | Appear as an in-progress card while the tool is running; update on result receipt                             |
| Artefact chips                                            | "Added to artefacts ↗" chip appears beneath each completed non-prose block                                    |

While the model is streaming, the input field is disabled and replaced by a **stop-generation button**. Stopping generation saves the partial response to the audit trail as a partial turn — it does not discard it. When stopped, a "(generation stopped)" label appears beneath the partial response.

### Truncated response — continue generating

When the model's response ends without a natural conclusion (detected heuristically: response ends mid-sentence, or the model emits a continuation signal), a **Continue** button appears below the response:

"Response may be incomplete. **Continue** →"

Clicking Continue submits an implicit "Please continue" turn, which regenerates from the end of the incomplete response in a new branch. The original truncated response is preserved. This handles output-token-limit cases transparently without requiring the user to know why the response stopped.

The Continue button is shown for a maximum of 60 seconds after the truncated response; after that it is dismissed to avoid polluting old threads.

# 11 — Audit and Storage

## Governing principle

Conversations on the AI Chat Platform are **auditable records**, not transient chat logs. Every conversation turn is a self-contained, reproducible record. A reviewer can reconstruct what the user submitted, what files they provided, what tools were invoked, and what was produced — without referring to any external system (P4 — audit completeness).

## Multi-tenant isolation

Every record in the `assistant` schema carries a `tenant_id` column. Row-Level Security (RLS) enforces that users can only access records belonging to their own tenant. No cross-tenant data access is possible through the platform's API.

The `assistant` schema is isolated from all platform infrastructure schemas and from any host application data schemas. No shared tables exist between the `assistant` schema and other schemas.

## Storage policy

All conversation content is retained in full at write time. Nothing is reconstructed on demand.

### Per-turn stored elements

| Element                 | Stored as                                                                                                                                                               |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User prompt (raw)       | Exact text including <code>@</code> -binding chip markers                                                                                                               |
| User prompt (resolved)  | Message as sent to the model — bindings expanded to structured context blocks from <code>contextTemplate</code>                                                         |
| Assistant response      | Full markdown text including all content blocks                                                                                                                         |
| Rendered content blocks | Mermaid source, Vega-Lite specs, CSV, code — typed JSONB records                                                                                                        |
| Tool call log           | Tool name, input parameters, response payload, latency, success/error per invocation                                                                                    |
| Attached input files    | Full binary stored in platform object storage                                                                                                                           |
| Output artefacts        | Full content stored in turn record                                                                                                                                      |
| Canvas version          | If the turn produces or accepts a canvas revision, a <code>canvas_versions</code> row is written linked to this turn                                                    |
| Model version           | Exact provider model string (e.g. <code>provider-name:model-id:version</code> ) and resolved tier ( <code>fast</code> / <code>standard</code> / <code>powerful</code> ) |

| Element             | Stored as                                                                    |
|---------------------|------------------------------------------------------------------------------|
| Token counts        | Input, output, cache read, cache write — per turn and running session totals |
| Improvement signals | Detected signals with confidence score and lifecycle status                  |
| Author              | <code>user_id</code> FK on every turn (critical for shared conversations)    |
| Tenant              | <code>tenant_id</code> on every record — enforces multi-tenant isolation     |

## Partial turns

When a user stops generation mid-stream, the partial response is saved to the audit trail as a **partial turn** with a `status: partial` flag. It is not discarded.

## Retention

| Rule                    | Specification                                                                                                                                                                                                                |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Retention period        | Configured per tenant via <code>conversations.retentionDays</code> in the application config. Platform default: <b>1,095 days (3 years)</b> . Tenants may configure a shorter or longer period within platform-level limits. |
| Retention tag           | Each conversation record is tagged with a <code>retention_expiry_date</code> at write time                                                                                                                                   |
| Archival                | A scheduled platform function handles expiry and archival per tenant's configured retention period                                                                                                                           |
| User-initiated deletion | Users may delete conversations subject to the tenant's retention minimum; physical deletion is deferred to retention expiry                                                                                                  |
| Turn-level deletion     | Not permitted — conversations are append-only at the turn level. Editing creates a new branched thread.                                                                                                                      |
| Object storage          | Binary artefacts (attached documents, generated outputs) follow the same tenant-configured retention schedule                                                                                                                |

## Access control

| Access level                    | Capability                                                                                                    |
|---------------------------------|---------------------------------------------------------------------------------------------------------------|
| Authenticated end user          | Read and write access to their own conversation records within their tenant only                              |
| Shared conversation participant | Read and write access to all turns within conversations they have accepted an invitation to                   |
| Application Admin               | Read access to all conversation records within their own tenant (audit view); no write access to turn records |

| Access level   | Capability                                                                                              |
|----------------|---------------------------------------------------------------------------------------------------------|
| Platform Admin | Read access to all records across all tenants for operational purposes; no write access to turn records |
| No user        | May edit or delete individual turn records — turns are immutable once written                           |

Row-Level Security (RLS) is enabled on all tables in the `assistant` Postgres schema. Policies enforce: - `tenant_id` isolation across all tables - Per-user and per-participant access within a tenant - Append-only semantics for turn records

## Database schema overview

The `assistant` schema contains the following tables:

| Table                                            | Purpose                                                                                                                          |
|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <code>assistant.tenants</code>                   | One row per registered host application; tenant config snapshot, API key reference                                               |
| <code>assistant.conversations</code>             | One row per conversation thread; <code>tenant_id</code> , title, owner, timestamps, retention expiry, CSAT                       |
| <code>assistant.turns</code>                     | One row per turn; <code>tenant_id</code> , conversation FK, author FK, model, token counts, status                               |
| <code>assistant.artefacts</code>                 | One row per artefact; <code>tenant_id</code> , turn FK, type, content/storage path, auto-generated name                          |
| <code>assistant.bindings</code>                  | One row per @-binding chip in a turn; <code>tenant_id</code> , object type, Display ID, resolved context snapshot                |
| <code>assistant.tool_calls</code>                | One row per MCP tool invocation; <code>tenant_id</code> , turn FK, server ID, tool name, input params, response, latency, status |
| <code>assistant.improvement_signals</code>       | One row per signal; <code>tenant_id</code> , turn FK, signal type, confidence score, lifecycle status, issue reference           |
| <code>assistant.session_tools</code>             | One row per opt-in MCP tool activation; <code>tenant_id</code> , conversation FK, server ID, activated_by, activated_at          |
| <code>assistant.conversation_participants</code> | One row per participant per conversation; <code>tenant_id</code> , user FK, invited_by FK, lifecycle timestamps                  |
| <code>assistant.user_memory</code>               | One row per personal memory item; <code>tenant_id</code> , user_id, content, category, status                                    |
| <code>assistant.app_context</code>               | One row per application context item; <code>tenant_id</code> , content, category, status, approval workflow fields               |
| <code>assistant.app_context_versions</code>      | Version history for application context items                                                                                    |

| Table                                   | Purpose                                                                                                                                                                                                                       |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>assistant.canvas_documents</code> | One row per document canvas open in a conversation; <code>tenant_id</code> , conversation FK, title, current version reference                                                                                                |
| <code>assistant.canvas_versions</code>  | One row per canvas version; <code>tenant_id</code> , canvas FK, turn FK (the turn that produced or accepted this version), full markdown content, author ( <code>user</code> or <code>model</code> ), <code>created_at</code> |

### RLS policies (summary)

- All tables require `tenant_id` to match the authenticated user's tenant claim from their JWT.
- Users can `SELECT` , `INSERT` on their own conversations and turns.
- Accepted participants can `SELECT` , `INSERT` on shared conversation records within the same tenant.
- Application Admins can `SELECT` all records within their tenant.
- Platform Admins can `SELECT` all records across all tenants.
- No user can `UPDATE` or `DELETE` turn records.
- `assistant.conversation_participants` records are append-only — departure is recorded as `departed_at` timestamp, not row deletion.
- `assistant.canvas_versions` records are append-only — each edit creates a new version row; no version row is modified after insertion.

### Object storage conventions

Binary artefacts are stored in platform object storage under the following path convention:

```
{tenant_id}/conversations/{conversation_id}/{turn_id}/{artefact_id}/{filename}
```

- Storage is private — access is mediated by signed URLs generated per-request with short expiry.
- Files are not publicly accessible.
- Storage paths are stored in `assistant.artefacts.storage_path` .

### Audit trail for shared conversations

In shared conversations, the `user_id` on each turn records the specific participant who submitted it. The `assistant.conversation_participants` table records the full invitation lifecycle:

| Column                   | Content                                                        |
|--------------------------|----------------------------------------------------------------|
| <code>user_id</code>     | The participant's platform user ID                             |
| <code>invited_by</code>  | The user ID of the person who invited them                     |
| <code>invited_at</code>  | Timestamp of invitation                                        |
| <code>accepted_at</code> | Timestamp of acceptance (null if not yet accepted or declined) |

| Column                   | Content                                       |
|--------------------------|-----------------------------------------------|
| <code>departed_at</code> | Timestamp of departure (null if still active) |

There are no role columns — all participants are equal. The audit record reflects actions taken, not role assignments.

---

## Tenant config version history

Every version of a tenant's application config is retained in `assistant.tenants.config_history` (append-only array). This allows platform administrators and Application Admins to trace exactly which config version was active during any historical conversation session.

The config version active at session start is stored in `assistant.conversations.config_version`.



# 12 — Continuous Improvement

## Principle

The AI Chat Platform is a living system. Every session generates signal data. **No change is applied automatically** — every recommendation enters a triage pipeline and requires human review before any change is deployed (P8 — human-gated improvement).

The improvement framework generates: **signal** → **triage** → **issue** → **approval** → **deploy**. It does not generate signal → auto-patch.

## Improvement signals

Five types of improvement signal are captured automatically from day one:

| Signal type                      | Detection                                                                                                      | Auto-issue?               |
|----------------------------------|----------------------------------------------------------------------------------------------------------------|---------------------------|
| Explicit turn report             | User clicks the report icon on any turn and submits the modal — optionally with a written explanation          | Always                    |
| Query retry / rephrase           | User submits a substantially similar query within two turns                                                    | If confidence ≥ threshold |
| MCP tool failure or empty result | Tool call log shows error or zero-result set where results were expected                                       | If confidence ≥ threshold |
| In-conversation correction       | User explicitly corrects the assistant mid-session                                                             | If confidence ≥ threshold |
| Out-of-scope response            | Response contains no tool invocation and no domain-specific content for a query that should have triggered one | If confidence ≥ threshold |

### Explicit turn report — detail

The report icon appears on every turn (user and assistant). When a user submits a report:

- The turn `id`, `conversation_id`, `tenant_id`, `reporter_user_id`, optional `explanation` text, and a timestamp are stored in `assistant.improvement_signals`
- The signal always generates an improvement issue — no confidence threshold
- The improvement issue includes: the full turn content, the reporter's explanation (if provided), the preceding and following turns for context, and the MCP tool call log for the turn

The written explanation is the most valuable signal — it describes the problem in plain language without inference. Users should be encouraged (via the modal copy) to describe what was wrong and why.

### Signal lifecycle

Each signal is stored in `assistant.improvement_signals` with a lifecycle status:

new → triaged → issued → resolved

| Status   | Description                                                      |
|----------|------------------------------------------------------------------|
| new      | Signal captured; awaiting triage                                 |
| triaged  | Reviewed; decision made on whether to raise an improvement issue |
| issued   | Improvement issue created; signal linked to issue reference      |
| resolved | Underlying change deployed and validated                         |

## Confidence threshold

Automatically detected signals below the confidence threshold queue for **manual review first** before an improvement issue is raised. The exact threshold is set at launch and adjusted based on signal volume.

Explicit user reports bypass the confidence threshold and **always generate an improvement issue**.

## Improvement issue pipeline

Qualifying signals are processed into an improvement issue containing:

| Issue element                   | Content                                                                             |
|---------------------------------|-------------------------------------------------------------------------------------|
| Failure description             | Signal type, turn summary, and assistant output excerpt                             |
| Full conversation turn          | Raw prompt, resolved prompt, tool call log, and assistant response                  |
| MCP tool call log               | All tool invocations for the turn (server name, tool name, params, result, latency) |
| System-generated recommendation | Most likely remediation target — see table below                                    |
| Tenant context                  | <code>tenant_id</code> , application name, config version active at the time        |

## Remediation targets

| Remediation target                            | When recommended                                                                                                   |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Host system prompt                            | Assistant goes out of scope, misrepresents capabilities, uses wrong terminology for the user's communication style |
| MCP tool description                          | Assistant chooses the wrong tool or fails to invoke a tool when it should                                          |
| Guided workflow prompt                        | A workflow returns unhelpful or incomplete output                                                                  |
| Bindable type<br><code>contextTemplate</code> | Binding resolution provides insufficient or incorrect context for the model                                        |
| Application context                           | Out-of-date or missing application-level memory causing incorrect assumptions                                      |

### Routing

Improvement issues are: 1. **Always** visible to the Platform team for platform-level patterns 2. **Optionally forwarded to the host application** via the tenant's configured improvement webhook (see below)

Host applications that configure an improvement webhook receive structured JSON payloads for each qualifying signal, enabling their Application Admin to triage within their own tooling.

### Improvement webhook (optional)

Host applications may configure an improvement webhook endpoint in the Platform Admin API. When configured: - Qualifying improvement signals (above threshold or explicit user reports) trigger a POST to the webhook URL with a structured JSON payload - The payload includes: `tenant_id` , `signal_type` , `turn_summary` , `remediation_recommendation` , `issue_id` - Authentication: the platform signs webhook payloads with an HMAC key provided at registration

### Improvement cadence

| Cadence     | Activity                                                                                                   | Owner                                 |
|-------------|------------------------------------------------------------------------------------------------------------|---------------------------------------|
| Weekly      | Application Admin reviews new improvement issues for their tenant; advances high-confidence items          | Application Admin (per tenant)        |
| Weekly      | Platform team reviews platform-level patterns across all tenants                                           | Platform team                         |
| Monthly     | Aggregate signal analysis; identify systemic patterns; set remediation priorities                          | Application Admin + Platform team     |
| Per release | Post-change validation — monitor signal volume for the affected signal type for two weeks after deployment | Host development team + Platform team |

### What the improvement framework does not do

| Action                                                 | Status                                                                         |
|--------------------------------------------------------|--------------------------------------------------------------------------------|
| Auto-update the host system prompt based on signals    | Not permitted — all changes require host team review and config update         |
| Auto-retrain or fine-tune the AI model                 | Not in scope — the platform uses foundation AI provider models                 |
| Apply changes to guided workflow prompts automatically | Not permitted — workflow changes require host team config update               |
| Apply changes to MCP tool descriptions automatically   | Not permitted                                                                  |
| Close improvement issues without human triage          | Not permitted — every issue requires Application Admin or Platform team review |

| Action                                          | Status                                                                       |
|-------------------------------------------------|------------------------------------------------------------------------------|
| Share improvement signal content across tenants | Not permitted — signals are tenant-scoped and never exposed to other tenants |

# 13 — MCP Tool Registry

## Purpose

The MCP tool registry is the **per-tenant runtime list of MCP servers** available in a session. It is derived directly from the `mcpServers` array in the tenant's application config — there is no separate registry file or database table beyond the config itself.

When a session starts, the platform resolves the active registry from the tenant's current config and:

1. Activates all `always-on` servers immediately
2. Makes `opt-in` servers available in the tool selection panel (but not active)
3. Injects descriptions for active servers into the system prompt

This document covers the registry model, how host teams populate it, the relationship to the MCP Repository complementary service, and how the registry interacts with the MCP Resources Service.

## Access tiers

| Tier      | Behaviour                                                                                                                                    |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Always-on | Active in every session; description injected into system prompt automatically; user cannot disable                                          |
| Opt-in    | Off by default; user activates per session via tool selection panel; description injected only when active; does not persist across sessions |

Use `always-on` for servers that are foundational to the assistant's usefulness (e.g. the host application's primary data API). Use `opt-in` for servers that are situationally useful but not needed in every session (e.g. a warehouse query tool, an external analytics service).

## Registry schema

The registry is the `mcpServers` array in the application config. Each entry:

```
{
  "id": "string – unique within tenant",
  "name": "string – shown in tool selection panel",
  "description": "string – injected into system prompt when active; write for the model",
  "endpoint": "string – MCP-compliant HTTPS URL",
  "authType": "bearer | api-key | none",
  "accessTier": "always-on | opt-in",
  "roles": ["array of role identifiers – empty = all users"],
  "enabled": true
}
```

The `enabled` field allows stubs to be added before an endpoint is production-ready. Setting `enabled: false` keeps the entry in the config but excludes it from session resolution — the tool does not appear in the tool selection panel and its description is never injected.

Full field reference is in [01-host-application-config.md — mcpServers](#).

## Tool description quality

The `description` field is **the most important field in the registry**. It is injected directly into the model's system prompt and directly influences whether the model invokes the correct tool for a given user query.

### Good description (written for the model):

Provides access to Acme Corp's data governance platform. Use this tool to look up data domains, entities, quality metrics, data ownership records, and policy information. Use when the user asks about data assets, governance status, quality issues, owners, or compliance.

### Poor description (written for the user):

Governance Platform MCP server.

Application Admins should review and iterate on tool descriptions based on improvement signals. Poor tool descriptions are one of the most common causes of the model choosing the wrong tool.

## Adding a new MCP server to the registry

| Step                          | Description                                                                                                           |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| 1. Identify the server        | Browse the <b>MCP Repository</b> to find an existing server, or confirm the host team's own MCP endpoint              |
| 2. Verify the endpoint        | Confirm the MCP endpoint is production-ready and accessible from the platform's network                               |
| 3. Confirm auth compatibility | Verify that <code>bearer</code> (host JWT forwarded), <code>api-key</code> , or <code>none</code> auth is appropriate |
| 4. Draft the description      | Write the description for the model — describe what data it provides and when to use it                               |
| 5. Decide access tier         | <code>always-on</code> if needed in every session; <code>opt-in</code> if situationally useful                        |
| 6. Decide role restriction    | Which users should see or activate this tool; empty <code>roles</code> array = all users                              |
| 7. Update the config          | Add the entry to <code>mcpServers</code> via the Config Editor UI or Admin API                                        |
| 8. Validate                   | The platform runs an endpoint reachability check on config submission                                                 |

| Step       | Description                                                                         |
|------------|-------------------------------------------------------------------------------------|
| 9. Monitor | Watch improvement signals in the first weeks for tool invocation failures or misuse |

Servers may be added as `enabled: false` stubs before their endpoint is ready. This allows the description to be drafted and reviewed before the server becomes active.

## Versioning

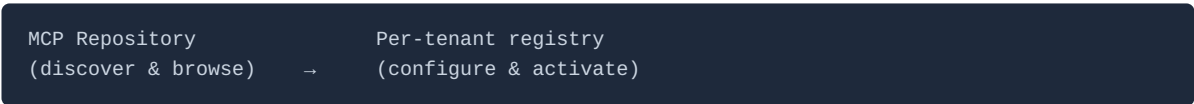
Tool registry changes are versioned as part of the overall application config (see [01-host-application-config.md — Config versioning](#)):

| Change                                                                             | Config version increment |
|------------------------------------------------------------------------------------|--------------------------|
| New MCP server added                                                               | Minor                    |
| Existing server description or name updated                                        | Patch                    |
| Server removed or <code>enabled</code> changed to <code>false</code>               | Minor                    |
| <code>authType</code> , <code>endpoint</code> , or <code>accessTier</code> changed | Minor                    |

## Relationship to the MCP Repository

The **MCP Repository** is a complementary ecosystem service (see [17-complementary-mcp-services.md](#)) that provides a discoverable catalogue of available MCP servers — both from the platform ecosystem and from external contributors.

The relationship to the per-tenant registry is:



Host teams use the MCP Repository to find servers that already exist and are ready to integrate. Once a suitable server is found, its details (endpoint, suggested description, auth type) are used to populate the `mcpServers` entry in the application config.

The MCP Repository is a read-only browsing surface at config time — it is not invoked at session runtime. The per-tenant registry (derived from the config) is the authoritative source for what is available in a session.

## Relationship to the MCP Resources Service

The **MCP Resources Service** is a complementary ecosystem service that provides centralised skills, static resources, and reusable prompt artefacts for use across the MCP ecosystem (see [17-complementary-mcp-services.md](#)).

Host applications may register the MCP Resources Service as an MCP server in their tenant registry (typically as an `opt-in` server). When registered and activated in a session:

- The model can invoke the MCP Resources Service to retrieve platform-standard skills, guidance documents, and reusable prompt patterns
- Resources returned appear as MCP tool results in the conversation thread with the standard tool call disclosure card
- Resources are added to the session artefact tray for download

The MCP Resources Service endpoint is published in the MCP Repository.

---

## System prompt injection

When a server is active in a session (always-on or opt-in enabled), its `description` is injected into the system prompt at session start. This informs the model of the server's purpose and when to invoke it.

Always-on server descriptions are always injected. Opt-in server descriptions are injected only when the user has enabled the server for the session. This keeps the system prompt lean when opt-in tools are not in use.

Combined description injection is included in the prompt cache — the cache hit rate metric accounts for the combined length of all active tool descriptions.



## 14 — Success Metrics

---

### Governing principle

Metrics are captured from day one at both the platform level (across all tenants) and at the application level (per tenant). Platform-level targets reflect the health of the infrastructure and ecosystem. Application-level targets reflect the value each host application is delivering to its users. Baseline targets for new tenants are set at the end of month 1 based on observed behaviour.

---

### Platform-level metrics

These metrics are tracked across all tenants by the platform team.

| Metric                         | Definition                                                                   | Target                          |
|--------------------------------|------------------------------------------------------------------------------|---------------------------------|
| Active tenants                 | Tenants with at least one active session in the 7-day window                 | Growth metric — tracked weekly  |
| Platform uptime                | API availability (p99 latency < 2s; error rate < 0.1%)                       | 99.9%                           |
| MCP tool error rate (platform) | Tool invocations with non-2xx response across all tenants                    | < 3%                            |
| Cache hit rate (platform)      | <code>cache_read_input_tokens</code> ÷ total input tokens across all tenants | ≥ 40% by month 2                |
| Tenant onboarding time         | Time from tenant registration to first active session                        | Baseline month 1                |
| Improvement signal volume      | Total signals across all tenants per week                                    | Baseline month 1; monitor trend |
| Config validation failure rate | Config submissions that fail validation ÷ total submissions                  | < 10%                           |

---

### Application-level metrics

These metrics are tracked per tenant and reported to the Application Admin via the tenant analytics dashboard.

| Metric                    | Definition                                         | Target                                           |
|---------------------------|----------------------------------------------------|--------------------------------------------------|
| Weekly active users (WAU) | Distinct users opening a session in a 7-day window | 40% of the tenant's eligible user base by day 90 |
| Queries per session       | Mean user messages per conversation                | ≥ 4                                              |

| Metric                                   | Definition                                                                         | Target                                       |
|------------------------------------------|------------------------------------------------------------------------------------|----------------------------------------------|
| <b>Workflow invocation rate</b>          | Sessions invoking at least one guided workflow                                     | ≥ 25% where workflows are configured         |
| <b>Binding click-through</b>             | Sessions where user clicks a binding chip (fires <code>binding-click</code> event) | ≥ 30%                                        |
| <b>MCP tool error rate (application)</b> | Tool invocations with error in the tenant                                          | < 3%                                         |
| <b>User satisfaction (CSAT)</b>          | Post-session rating (1–5) where offered                                            | Mean ≥ 4.0                                   |
| <b>Improvement signal rate</b>           | Improvement signals per 100 turns                                                  | Baseline month 1; reduction month-over-month |
| <b>Cache hit rate (application)</b>      | Cache hit rate for this tenant's sessions                                          | ≥ 40% by month 2                             |
| <b>Document attachment rate</b>          | Sessions with at least one document attachment                                     | Baseline month 1                             |
| <b>Artefact download rate</b>            | Sessions with at least one artefact download                                       | ≥ 20%                                        |
| <b>Mobile session share</b>              | Sessions on mobile/tablet viewport                                                 | Baseline month 1                             |
| <b>Shared conversation rate</b>          | Conversations with at least one invitation sent                                    | Baseline month 1                             |
| <b>Participant acceptance rate</b>       | Invitations accepted ÷ total invitations sent                                      | ≥ 70%                                        |

## Metric definitions

### Weekly active users

A user is counted once per 7-day window regardless of session count. "Active" means at least one user message submitted — opening the component without sending a message does not count.

### Queries per session

Total user messages ÷ total conversation sessions in the period. Excludes sessions with zero user messages.

### Workflow invocation rate

A session counts if at least one guided workflow was invoked via any method: Workflow Library click, @ - binding, or natural-language trigger.

## Binding click-through

A session counts if at least one binding chip in an assistant response was clicked (firing the `binding-click` event to the host application). Tracks whether users are using the assistant as a gateway into the host application, not just a standalone Q&A surface.

## MCP tool error rate

Tool invocations with a non-2xx response or an empty result set (where results were expected) ÷ total tool invocations. The "expected results" qualifier requires a confidence classifier — baseline tracks simple error rate only until the classifier is calibrated.

## User satisfaction (CSAT)

Post-session rating prompt (1–5 stars) shown to a random sample configured via `features.csatSampleRate` (default: 20% of sessions).

## Improvement signal rate

Total improvement signals (all types) per 100 conversation turns, tracked by signal type. Month-over-month reduction in signal rate per type indicates the improvement pipeline is working.

## Cache hit rate

$\text{cache\_read\_input\_tokens} \div (\text{input\_tokens} + \text{cache\_read\_input\_tokens})$  per the AI provider API response. Tracked at session level and rolling daily average. A target of  $\geq 40\%$  by month 2 reflects the expectation that the system prompt and tool descriptions will be consistently cached after the first turn in each session.

## Document attachment rate

Sessions where the user attached at least one document (PDF, Excel, Word, or image). Baseline metric — no target until month 1 data is available.

## Artefact download rate

Sessions where the user downloaded at least one artefact from the artefact tray. Tracks whether users find rendered outputs useful enough to take away.

## Mobile session share

Sessions where the viewport width is  $< 768\text{px}$  (mobile) or  $768\text{px}–1023\text{px}$  (tablet) at any point during the session. Baseline metric to validate mobile-first investment.

## Shared conversation rate

Conversations where at least one invitation was sent (regardless of acceptance). Tracks collaborative usage.

## Participant acceptance rate

Invitations accepted ÷ total invitations sent. A low rate may indicate users are inviting the wrong people, or that the invitation UX needs improvement.

## Measurement responsibilities

| Metric                                                                                                         | Source                                                                   | Owner                             |
|----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|-----------------------------------|
| WAU, queries per session, CSAT, attachment rate, artefact download, mobile share, shared rate, acceptance rate | <code>assistant</code> schema + session analytics                        | Platform Engineering              |
| Workflow invocation rate, binding click-through                                                                | <code>assistant.tool_calls</code> + binding click events                 | Platform Engineering              |
| MCP tool error rate                                                                                            | <code>assistant.tool_calls.status</code>                                 | Platform Engineering              |
| Improvement signal rate                                                                                        | <code>assistant.improvement_signals</code>                               | Platform team + Application Admin |
| Cache hit rate                                                                                                 | AI provider API response metadata stored in <code>assistant.turns</code> | Platform Engineering              |
| Platform uptime, config validation failure rate                                                                | Platform infrastructure monitoring                                       | Platform Engineering              |

## Review cadence

| Cadence                    | Activity                                                                    | Owner                              |
|----------------------------|-----------------------------------------------------------------------------|------------------------------------|
| <b>Weekly</b>              | WAU, tool error rate, CSAT per tenant — operational health                  | Platform Engineering               |
| <b>Weekly</b>              | Application Admin reviews improvement signal issues for their tenant        | Application Admin                  |
| <b>Monthly</b>             | Full metric review; signal rate analysis; cache hit trend                   | Platform team + Application Admins |
| <b>Day 90 (per tenant)</b> | Activation target assessment (40% WAU); decision on next feature priorities | Application Admin + Platform team  |

## Tenant analytics dashboard

Each tenant's Application Admin has access to a **read-only analytics dashboard** showing their tenant's application-level metrics over time: - WAU trend (30-day rolling) - Queries per session trend - CSAT distribution - Top improvement signal types - MCP tool error rate by server - Cache hit rate trend

Platform-level metrics (cross-tenant) are visible to the Platform team only.

# 15 — Memory and Recall

## Overview

Memory and recall governs how the assistant retains standing context across sessions and how that context is scoped to the right users. Two memory controls work together:

| Control                    | Purpose                                                                                                               | Scope                            |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------|----------------------------------|
| <b>Personal memory</b>     | Standing context about the individual user — role, focus areas, preferences, and corrections                          | That user only                   |
| <b>Application context</b> | Standing context about the organisation/application — terminology, structure, governance decisions, standing guidance | All users within the same tenant |

A third control — **recall scope** — determines who a user can share a conversation with: only users within the same tenant.

All three controls are **transparent** — users always see exactly what context the assistant is working with. Memory is never injected silently (P1 — transparency first).

## Personal memory

### Purpose

Personal memory lets users tell the assistant things that are true about them specifically, so they don't have to repeat context at the start of every conversation. Examples:

- "I am the owner of the Finance domain"
- "I primarily work with Customer and Transaction data"
- "When I ask for a quality summary, scope it to Gold-tier only"
- "In our team, 'account' means a customer record, not a financial account"

### Management UI

Personal memory is managed on the **My Memory** tab within the assistant's profile settings area (accessible from within the `<ai-chat>` component).

| Section                  | Contents                                                                                                                                                    |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Memory toggle</b>     | On/Off switch for personal memory injection. Default: <b>off</b> until the user explicitly enables it. Toggle state is always visible.                      |
| <b>Memory items list</b> | All memory items (Active and Inactive) — title, category tag, source (Manual / Extracted), date added, token count, status badge, edit and delete controls. |
| <b>Search / filter</b>   | Full-text search across item titles and content; filter by category and status.                                                                             |

| Section                          | Contents                                                                                                                            |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Add item</b>                  | Free-text input — user types the memory item in plain language, optionally assigns a category, confirms, and it is saved as Active. |
| <b>Extract from conversation</b> | User-initiated only. See <a href="#">Extraction process</a> below.                                                                  |
| <b>Token budget indicator</b>    | Shows current usage against the tenant-configured token budget (e.g. <code>640 / 2,000 tokens</code> ) with a colour-coded bar.     |
| <b>Clear all</b>                 | Archives all items (moves to Inactive) after confirmation. Does not permanently delete — archived items remain in the audit trail.  |

## Personal memory item categories

Host applications configure memory categories in `memory.personalMemory.categories` in the application config. Default categories:

| Category          | Examples                                                                                 |
|-------------------|------------------------------------------------------------------------------------------|
| <b>Role</b>       | Job title, domain ownership, team membership                                             |
| <b>Preference</b> | Preferred response style, scope defaults, output format preferences                      |
| <b>Correction</b> | Organisation-specific term overrides; corrections to the assistant's default assumptions |
| <b>Context</b>    | Current focus areas, active projects, known constraints                                  |

## Personal memory item lifecycle

Active → Inactive (budget exceeded or user toggle) → Archived (deleted by user)

| Status          | Injected? | Editable?                 | Visible in UI?                    |
|-----------------|-----------|---------------------------|-----------------------------------|
| <b>Active</b>   | Yes       | Yes                       | Yes                               |
| <b>Inactive</b> | No        | Yes — user can reactivate | Yes (greyed out)                  |
| <b>Archived</b> | No        | No                        | No (retained in audit trail only) |

Items are never permanently deleted — the record is retained with `status = archived` for the audit trail.

## Extraction process

"Extract from conversation" is a structured, user-initiated workflow — nothing is extracted automatically.

1. User opens the conversation picker (full-text search across their conversation history)
2. User selects a past conversation
3. A dedicated **fast-tier model** call analyses the selected conversation using a structured extraction prompt — it identifies discrete, self-contained statements of user context worth persisting (role declarations, expressed preferences, corrections made, constraints stated)

4. The assistant presents a **review panel** showing each proposed item with: suggested text, suggested category, estimated token cost, and source conversation reference
5. User accepts, edits, or discards each proposal **individually** — there is no bulk accept
6. Accepted items are saved as Active; discarded proposals are not stored

The extraction call is a separate API call — it does not affect the conversation context window. The extraction prompt explicitly excludes host application object references (stale risk) and system prompt override attempts before they reach the review panel.

## Token budget

Personal memory is injected into the system prompt as a labelled `[Your context]` block. The budget is configurable per tenant via `memory.personalMemory.tokenBudget` (default: 2,000 tokens). When the budget is exceeded, the user is warned and the oldest items are marked Inactive — the user decides which items to reactivate or remove.

## What personal memory cannot contain

- Host application object IDs or live data references — these may become stale. The assistant resolves live data via MCP at query time.
- Other users' personal information
- Instructions that would override the platform system prompt — the platform prompt takes precedence

## Injection behaviour

When personal memory is enabled, the `[Your context]` block is injected into the system prompt at session start. It appears in the conversation header as a collapsed indicator: **"Your context active [2 items · 640 tokens] ↗"** — click to expand and inspect the full content.

---

## Application context

### Purpose

Application context gives the assistant organisation-wide context that applies to all users and sessions within the tenant. It provides ground truth for terminology, structure, and standing decisions that the assistant would otherwise have to infer or ask about. Examples:

- *"Our organisation uses 'account' to refer to customer records. 'Account' does not mean a financial account."*
- *"The Customer domain is the master domain. All other domains reference it for shared definitions."*
- *"Data owner assignments are reviewed quarterly — an assignment may be up to 90 days out of date."*
- *"The Finance domain is under an active regulatory review. Flag any Finance domain queries before providing assessments."*



## Management UI

Application context is managed in the **Tenant Admin area** (accessible to users with the Application Admin role).

| Section                       | Contents                                                                                                                                                                           |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Context items list</b>     | All items with title, category, approval status, effective date, and approver                                                                                                      |
| <b>Add item</b>               | Application Admin proposes a new item with a title, category, and content                                                                                                          |
| <b>Approval workflow</b>      | If <code>memory.applicationContext.approvalRequired: true</code> (default), new items require two-step approval: Propose → Approve. Proposer and approver must be different users. |
| <b>Edit item</b>              | Any edit to an approved item creates a new draft — the existing approved version continues to be injected until the edit is approved                                               |
| <b>Retire item</b>            | Removes an item from injection without deleting it                                                                                                                                 |
| <b>Version history</b>        | Full version history per item: original text, all edits, approval actions, effective dates                                                                                         |
| <b>Token budget indicator</b> | Current usage against the tenant-configured budget                                                                                                                                 |

## Categories

Host applications configure application context categories in `memory.applicationContext.categories` (default):

| Category              | Examples                                                                  |
|-----------------------|---------------------------------------------------------------------------|
| <b>Terminology</b>    | Organisation-specific term definitions; overrides to common terms         |
| <b>Structure</b>      | Organisational hierarchy, domain ownership, team structure                |
| <b>Policy</b>         | Standing governance decisions, active reviews, classification conventions |
| <b>Domain context</b> | Known data quality issues, known gaps, recency caveats                    |

## Token budget

Application context is injected as a labelled `[Application context]` block in the system prompt. The budget is configurable per tenant via `memory.applicationContext.tokenBudget` (default: 4,000 tokens). Items are injected in category order and by effective date (newest first) within category.

## Visibility for all users

Application context is visible (read-only) to all authenticated users within the tenant from the conversation header indicator. Users can see exactly what organisation-wide context the assistant is working with in any session. They cannot edit it.

## Approval pipeline

Draft → Proposed → Approved (active) → Retired

| Transition              | Who can trigger                                  | Effect                                                                           |
|-------------------------|--------------------------------------------------|----------------------------------------------------------------------------------|
| Draft → Proposed        | Application Admin                                | Item visible in admin UI; not yet injected                                       |
| Proposed → Approved     | Application Admin (different user from proposer) | Item becomes active; injected into all subsequent sessions                       |
| Approved → Retired      | Application Admin                                | Item removed from injection; retained in audit trail                             |
| Approved → Draft (edit) | Application Admin                                | New draft created; approved version continues injecting until new draft approved |

When `memory.applicationContext.approvalRequired: false`, the two-step approval is replaced by a single-step publish action (any Application Admin can publish immediately).

## Combined injection — conversation header

In every conversation, the header shows the combined active memory state:

 Your context [2 items · 640 tokens] ↗

 Application context [4 items · 1,240 tokens] ↗

Both indicators are collapsed by default. Clicking either opens a two-level panel:

**Level 1 — Compact list** (default on open) Each active memory item shown as a single line: category tag + title + token count.

**Level 2 — Full text** (expand per item) Click any item row to expand to its full content — exactly as injected.

A user may **disable personal memory for a single session** by clicking the indicator and toggling off before sending their first message. This does not affect the global personal memory setting.

Application context cannot be disabled by individual users.

## Token budget summary

| Memory type     | Default budget | Injected as                           |
|-----------------|----------------|---------------------------------------|
| Personal memory | 2,000 tokens   | [Your context] block in system prompt |

| Memory type         | Default budget | Injected as                                                |
|---------------------|----------------|------------------------------------------------------------|
| Application context | 4,000 tokens   | [Application context] block in system prompt               |
| Combined maximum    | 6,000 tokens   | Injected before the first turn; cached on subsequent turns |

The combined 6,000-token memory budget is included in the prompt cache — the cache hit rate metric accounts for it.

## Recall and access scope

### Tenant boundary

A user can only share a conversation with — and recall shared context alongside — other users **within the same tenant**. This is a hard boundary enforced at the data layer.

| Boundary             | Behaviour                                                                                                                                               |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Conversation sharing | Invitation search returns users within the authenticated user's tenant only. Cross-tenant results are excluded.                                         |
| Application context  | The [Application context] block is scoped to the authenticated user's tenant. Context from one tenant is never injected into another tenant's sessions. |
| Personal memory      | Personal memory items are owned by <code>user_id</code> and are never visible to other users.                                                           |

### Why this boundary exists

Conversation threads accumulate context about an organisation's operations, decisions, and data. Permitting cross-tenant access to this context would violate data confidentiality. The tenant boundary is a design constraint, not a configurable option.

## Audit trail

| Element                     | Where stored                                                                                                                                                                                                                                                                                                                         |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Personal memory items       | <code>assistant.user_memory</code> — <code>tenant_id</code> , <code>user_id</code> , <code>content</code> , <code>category</code> , <code>source</code> , <code>status</code> , timestamps                                                                                                                                           |
| Application context items   | <code>assistant.app_context</code> — <code>tenant_id</code> , <code>title</code> , <code>category</code> , <code>content</code> , <code>status</code> , <code>proposed_by</code> , <code>approved_by</code> , <code>effective_from</code> , <code>retired_at</code> ; version history in <code>assistant.app_context_versions</code> |
| Memory injected per session | <code>assistant.turns</code> — the resolved system prompt including both memory blocks is stored on the first turn of every conversation                                                                                                                                                                                             |

Memory items themselves are retained indefinitely (they are governance configuration, not conversation data). Only their injection into turns is subject to the tenant's configured retention period.

---

## What memory cannot replace

Memory provides standing context. It does not replace live data. The assistant always resolves live object data, status, and metrics via host MCP servers at query time. Memory items that describe application objects are advisory framing, not authoritative data.

**Correct use:** *"The Finance domain is under an active regulatory review — flag Finance queries."* This is framing that helps the assistant contextualise queries.

**Incorrect use (rejected by validation):** *"The Finance domain has 42 entities and Jane Smith is the owner."* This would become stale. The assistant queries the host MCP server for live entity data.

# 16 — Embedding and Web Component (Mode 2 — Inline Page)

## Scope of this document

This document covers the `<ai-chat>` **component** — the inline page embedding mode (Mode 2). It is the complete, full-featured conversation interface for use on dedicated assistant pages or embedded content sections.

The AI Chat Platform has three distinct embedding modes. For the full picture, see [18-entry-points-and-embedding-modes.md](#):

| Mode                  | Component                           | Description                                               |
|-----------------------|-------------------------------------|-----------------------------------------------------------|
| 1 — Floating Widget   | <code>&lt;ai-chat-widget&gt;</code> | FAB + collapsible mini/full panel, persists across pages  |
| 2 — Inline Page       | <code>&lt;ai-chat&gt;</code>        | This document — full three-zone layout embedded in a page |
| 3 — Form Field Assist | <code>&lt;ai-chat-field&gt;</code>  | Ephemeral contextual popover scoped to a form field       |

Modes 1 and 2 share conversation history. Mode 3 is ephemeral and fully independent.

## Overview

The `<ai-chat>` component is a **custom HTML element** that host applications embed within their own UI. The component is self-contained — it manages its own state, handles streaming, renders content, and communicates with the platform API independently of the host application's framework.

Host applications include the component script once and configure it via HTML attributes and a JavaScript API. No framework-specific adapters are required; the component works in any modern web environment.

## Quick start

```
<!-- 1. Include the component script -->
<script type="module" src="https://chat-platform.io/sdk/v1/ai-chat.js"></script>

<!-- 2. Mount the component -->
<ai-chat
  tenant-id="acme-corp"
  user-token="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  theme="light"
></ai-chat>
```

The component renders at 100% of its container's width and height. Host applications control sizing via the container element.

## HTML attributes

| Attribute                            | Required | Type             | Description                                                                                                                                        |
|--------------------------------------|----------|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>tenant-id</code>               | Yes      | string           | The tenant's registered <code>tenantId</code> from the application config                                                                          |
| <code>user-token</code>              | Yes      | string           | The host-authenticated user's JWT. Must contain the required claims (see Authentication bridge). Refreshed by calling <code>updateToken()</code> . |
| <code>theme</code>                   | No       | "light"   "dark" | Initial colour scheme. Defaults to "light". Dark mode is not supported in v1 — this attribute is reserved.                                         |
| <code>locale</code>                  | No       | string           | BCP 47 locale tag (e.g. "en-GB"). Overrides the tenant config <code>scope.language</code> for UI strings.                                          |
| <code>initial-conversation-id</code> | No       | string           | Opens a specific conversation on mount instead of the default new/recent conversation. Useful for deep-linking from the host application.          |

## Authentication bridge

The platform does not authenticate users independently. The host application authenticates its users and passes the result to the component as a **signed JWT** via the `user-token` attribute.

### Required JWT claims

| Claim                  | Type   | Description                                                                                                       |
|------------------------|--------|-------------------------------------------------------------------------------------------------------------------|
| <code>sub</code>       | string | User's unique identifier within the tenant                                                                        |
| <code>email</code>     | string | User's email address (used for invitation search)                                                                 |
| <code>name</code>      | string | User's display name (shown in shared conversation attribution)                                                    |
| <code>tenant_id</code> | string | Must match the <code>tenant-id</code> attribute — enforced server-side                                            |
| <code>exp</code>       | number | JWT expiry time — the component checks this and fires <code>token-expired</code> when within 60 seconds of expiry |

### Optional JWT claims

| Claim                                                | Type   | Description                                 |
|------------------------------------------------------|--------|---------------------------------------------|
| Any field in <code>userProfile.styleField</code>     | string | Communication style value (e.g. "business") |
| Any field in <code>userProfile.verbosityField</code> | string | Response verbosity value (e.g. "concise")   |

| Claim                                                           | Type                  | Description                                                         |
|-----------------------------------------------------------------|-----------------------|---------------------------------------------------------------------|
| Any role identifiers listed in MCP server<br><code>roles</code> | <code>string[]</code> | Role claims used to enforce MCP server access restrictions          |
| <code>avatar_url</code>                                         | <code>string</code>   | URL to the user's avatar image (used in shared conversation thread) |

### JWT signature verification

The platform verifies the JWT signature against the JWKS endpoint registered by the host application at tenant registration time. Tokens signed with keys not in the registered JWKS are rejected.

### Token refresh

```
const chat = document.querySelector('ai-chat');

// Listen for token expiry warning
chat.addEventListener('token-expired', async () => {
  const newToken = await yourAuthService.refreshToken();
  chat.updateToken(newToken);
});
```

`updateToken(token)` updates the active token without remounting the component. The user's session and conversation state are preserved.

## JavaScript API

Access the component's API via a reference to the element:

```
const chat = document.querySelector('ai-chat');
```

### Methods

| Method                              | Parameters                  | Description                                       |
|-------------------------------------|-----------------------------|---------------------------------------------------|
| <code>updateToken(token)</code>     | <code>token: string</code>  | Refresh the user's JWT without remounting         |
| <code>openConversation(id)</code>   | <code>id: string</code>     | Navigate to a specific conversation by ID         |
| <code>startNewConversation()</code> | —                           | Open a new conversation, clearing the current one |
| <code>setLocale(locale)</code>      | <code>locale: string</code> | Change the UI locale at runtime                   |

### Properties

| Property                           | Type                       | Description                               |
|------------------------------------|----------------------------|-------------------------------------------|
| <code>currentConversationId</code> | <code>string   null</code> | The ID of the currently open conversation |

| Property                 | Type                 | Description                                              |
|--------------------------|----------------------|----------------------------------------------------------|
| <code>isStreaming</code> | <code>boolean</code> | Whether the assistant is currently generating a response |

## Events

The component fires custom events on the element. Listen with `addEventListener`.

| Event                             | Detail payload                                                      | Description                                                                                                         |
|-----------------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <code>binding-click</code>        | <code>{ typeId, objectId, displayId, name }</code>                  | Fired when a user clicks a binding chip. The host application should handle navigation to the object's detail page. |
| <code>conversation-created</code> | <code>{ conversationId, title }</code>                              | Fired when a new conversation is created                                                                            |
| <code>conversation-opened</code>  | <code>{ conversationId, title }</code>                              | Fired when an existing conversation is opened                                                                       |
| <code>turn-complete</code>        | <code>{ conversationId, turnId, model, tokenCounts }</code>         | Fired when an assistant response completes (not on partial/stopped)                                                 |
| <code>tool-invoked</code>         | <code>{ conversationId, turnId, serverId, toolName, status }</code> | Fired on each MCP tool invocation completion                                                                        |
| <code>token-expired</code>        | —                                                                   | Fired when the user's JWT is within 60 seconds of expiry                                                            |
| <code>error</code>                | <code>{ code, message }</code>                                      | Fired on component-level errors (auth failure, network error, config error)                                         |
| <code>csat-submitted</code>       | <code>{ conversationId, score, comment }</code>                     | Fired when a user submits a CSAT rating                                                                             |

### Handling `binding-click`

```
chat.addEventListener('binding-click', (event) => {
  const { typeId, objectId, displayId, name } = event.detail;

  switch (typeId) {
    case 'data_domain':
      router.navigate(`/domains/${objectId}`);
      break;
    case 'policy':
      router.navigate(`/policies/${objectId}`);
      break;
    default:
      console.log('Unhandled binding type:', typeId);
  }
});
```



## Sizing and layout

The `<ai-chat>` component renders at **100% width and 100% height of its containing element**. The host application controls the component's footprint by sizing the container.

### Full-page embedding

```
/* Host application styles */
.chat-page {
  width: 100%;
  height: 100vh;
  overflow: hidden;
}
```

```
<div class="chat-page">
  <ai-chat tenant-id="..." user-token="..."></ai-chat>
</div>
```

### Side panel embedding

```
.chat-panel {
  width: 480px;
  height: 100%;
  border-left: 1px solid var(--border-color);
}
```

```
<aside class="chat-panel">
  <ai-chat tenant-id="..." user-token="..."></ai-chat>
</aside>
```

### Minimum dimensions

| Dimension | Minimum |
|-----------|---------|
| Width     | 320px   |
| Height    | 400px   |

Below these minimums the component renders a fallback state: *"This panel is too small to display the assistant. Please expand the window."*

### Breakpoint behaviour in containers

When the component is mounted in a container narrower than the desktop breakpoint ( $< 1280\text{px}$ ), it uses the container width rather than the viewport width to determine which layout to apply. A 480px side panel will use the tablet layout regardless of the overall page width.

## Content Security Policy (CSP)

The `<ai-chat>` component requires the following CSP directives on the host page:

```
script-src      https://chat-platform.io [renderer module origins];
connect-src     https://api.chat-platform.io wss://api.chat-platform.io;
frame-src       'none';
img-src         https://cdn.chat-platform.io [host CDN origins];
style-src       https://chat-platform.io 'nonce-{page-nonce}';
```

Additionally: - Any URLs provided in the `branding` config ( `logoUrl` , `faviconUrl` ) must be included in `img-src` . - Each origin hosting a custom renderer module (from `renderers[].moduleUrl` in the application config) must be included in `script-src` . Renderer modules are loaded via dynamic `import()` at runtime and are subject to the host page's CSP. A module blocked by CSP will cause that renderer to fall back to syntax-highlighted code — the platform will not bypass CSP.

The component itself does not use `eval` , `inline-script` , or `blob:` URLs. All platform assets are loaded from `https://chat-platform.io` or `https://cdn.chat-platform.io` . Renderer modules are loaded from host-provided origins only.

---

## React / Vue / Angular integration

The web component works without framework adapters. Framework-specific wrappers are provided for convenience:

### React:

```
import { AiChat } from '@chat-platform/react';

<AiChat
  tenantId="acme-corp"
  userToken={jwt}
  onBindingClick={({ typeId, objectId }) => navigate(`/${typeId}/${objectId}`)}
/>
```

### Vue:

```
<AiChat
  tenant-id="acme-corp"
  :user-token="jwt"
  @binding-click="handleBindingClick"
/>
```

### Angular:

```
<ai-chat
  tenant-id="acme-corp"
  [attr.user-token]="jwt"
  (binding-click)="handleBindingClick($event)"
></ai-chat>
```

---

## Security considerations

### JWT forwarding

The `user-token` attribute value is passed as a bearer token to the platform API on every request. It must:

- Be issued by the host application's authenticated session (never a long-lived API key)
- Have a short expiry (recommended: 15 minutes)
- Be refreshed via `updateToken()` before expiry

### Preventing attribute injection

Do not interpolate untrusted user input into the `tenant-id` or `user-token` attributes. Both are validated server-side, but attribute injection could result in failed sessions or user confusion.

### CORS

The platform API enforces CORS. Only origins registered at tenant registration time are permitted to make requests. The host application must register all deployment origins (development, staging, production) during tenant setup.

### Iframe considerations

The component must not be embedded within an `<iframe>` — this prevents the platform from accessing the host page's cookies and local storage required for session management. The component is designed for direct DOM embedding only.

## 17 — Complementary MCP Services

---

The AI Chat Platform assumes the availability of three ecosystem-level MCP services that operate alongside the platform and host application MCP servers. These services are not owned or operated by the AI Chat Platform itself, nor by individual host applications — they are shared infrastructure within the broader MCP ecosystem. The platform is designed to work with them, and host applications are expected to benefit from them, but none is a hard dependency for platform operation.

---

### MCP Repository

#### Overview

The **MCP Repository** is a centralised, discoverable catalogue of MCP servers available within the ecosystem. It serves as the primary discovery mechanism for host teams when they are configuring their tenant's MCP tool registry — providing a searchable directory of tools that have already been built, tested, and published, rather than requiring every host team to build their own MCP servers from scratch.

The MCP Repository is a **config-time service**: host teams use it when setting up or updating their application config, not as a runtime tool invoked during user conversations. Its value is in accelerating the time between "we want to add this capability to our assistant" and "this capability is live for our users."

#### What the MCP Repository provides

The MCP Repository holds metadata records for each published MCP server, including:

- **Server identity:** Name, publisher, version, and a plain-language description of what the server provides
- **Capability catalogue:** The specific tools exposed by the server, with descriptions and example invocations
- **Integration metadata:** The server's MCP endpoint URL, supported authentication types ( `bearer` , `api-key` , `none` ), and any pre-conditions for integration (e.g. required host-side configuration)
- **Quality indicators:** Verification status, uptime history, average latency, and known compatibility notes with the AI Chat Platform
- **Suggested descriptions:** Pre-written `description` field text optimised for injection into the AI Chat Platform's system prompt — host teams can use these as-is or customise them

#### How host teams use the MCP Repository

When a host team wants to add a new capability to their assistant, the typical flow is:

1. Search the MCP Repository for a server that provides the needed capability
2. Review the server's capability catalogue and quality indicators
3. Copy the server's endpoint URL, suggested `description` , and recommended `authType` into the `mcpServers` entry in the application config

4. Submit the updated config via the Config Editor UI or Admin API — the platform validates endpoint reachability as part of config submission
5. Monitor tool invocation quality via improvement signals in the first weeks of use

## Publishing to the MCP Repository

MCP server publishers — whether internal platform teams, host application developers, or third parties — submit their server records via the MCP Repository's submission API. Submitted records are reviewed for:

- Correctness and completeness of metadata
- MCP protocol compliance
- Endpoint stability and availability
- Security posture (auth requirements, data exposure scope)

Approved records are published and immediately searchable in the Repository. Publishers are responsible for keeping their records current as their server's endpoint or capability changes.

## Relationship to the per-tenant tool registry

The MCP Repository and the per-tenant tool registry ([13-mcp-tool-registry.md](#)) are distinct:

| MCP Repository                                    | Per-tenant registry                                              |
|---------------------------------------------------|------------------------------------------------------------------|
| Ecosystem-wide catalogue of all available servers | Tenant-specific list of servers active for that host application |
| Read at config time by host teams                 | Resolved at session runtime by the platform                      |
| Covers all publishers and server types            | Covers only what the host application has chosen to enable       |
| Not invoked during user conversations             | The source of truth for what tools a session can use             |

## MCP Resources Service

### Overview

The **MCP Resources Service** is a centralised MCP server that provides shared, reusable assets across the MCP ecosystem — skills, guidance documents, prompt templates, and other static artefacts that are useful across many different host applications and conversation types.

Unlike the MCP Repository (which helps teams find and configure tools at setup time), the MCP Resources Service is a **runtime service** — it can be registered as an MCP server in a tenant's tool registry and invoked during user conversations to retrieve resources on demand.

### What the MCP Resources Service provides

The MCP Resources Service organises its content into three categories:

**Skills** Pre-built structured reasoning patterns that the assistant can invoke to approach complex tasks consistently. Skills are prompt fragments with defined input parameters and expected output structures. Examples include:

- Structured root-cause analysis prompts
- Stakeholder communication drafting frameworks
- Risk assessment scoring rubrics
- Data quality evaluation methodologies

Skills are not model-specific — they are designed to work across Anthropic, OpenAI, and Gemini models. When the assistant retrieves a skill from the MCP Resources Service, it uses the skill's prompt structure to guide its reasoning for that turn.

**Guidance documents** Static reference documents that apply across many host applications and are impractical for each host to maintain independently. Examples include: - AI assistant usage guidelines and responsible use principles - Data privacy handling guidance for AI-assisted workflows - Model capability and limitation reference cards - Prompt engineering best practices for domain-specific applications -

**Uncertainty handling guidance** — instructions for how the assistant should communicate the limits of its knowledge, signal when data may be outdated, and offer next steps (such as web search) when it cannot answer with confidence. This document is injected as a resource at session start by hosts that want consistent, domain-appropriate uncertainty behaviour beyond the platform's non-overridable baseline.

Guidance documents are versioned by the MCP Resources Service. When retrieved in a conversation, the document version is recorded in the tool call log.

**Prompt templates** Reusable prompt structures that host teams can adopt as starting points for guided workflows in their application config. These are published as MCP resource objects and can be retrieved, reviewed, and adapted before being embedded in the [workflows](#) section of a tenant's config.

## How host applications integrate the MCP Resources Service

Host applications that want their end users to benefit from MCP Resources Service content register the service as an MCP server in their tenant's tool registry — typically as an [opt-in](#) server, since not every conversation will require it:

```
{
  "id": "mcp-resources",
  "name": "Platform Resources",
  "description": "Provides shared skills, guidance documents, and prompt templates from the platform ecosystem. Use when you need a structured reasoning approach, usage guidance, or reference material.",
  "endpoint": "https://resources.mcp-ecosystem.io/mcp",
  "authType": "bearer",
  "accessTier": "opt-in",
  "roles": []
}
```

The MCP Resources Service endpoint is published in the MCP Repository. The suggested [description](#) field text above is available in the Repository record and optimised for AI Chat Platform injection.

## Content governance

The MCP Resources Service is governed by the platform ecosystem team responsible for its operation. Content goes through an editorial review before publication: - Skills and templates are tested against the platform's AI provider models before publication - Guidance documents are reviewed for accuracy and applicability across a range of host application types - All content is versioned; breaking changes to existing resources trigger a deprecation period before removal - Tenants that have retrieved a specific resource version continue to receive that version until they explicitly update to a newer version

Host applications and their users cannot modify MCP Resources Service content. Feedback and content requests are submitted via the ecosystem's standard issue process.

Relationship to the platform

The MCP Resources Service is independent of the AI Chat Platform — the platform does not own or operate it. However, the platform is designed to work with it cleanly: - Resources returned by the service are rendered using the platform's standard content rendering rules (prose, code blocks, data tables as appropriate) - Resource retrievals appear in the tool call disclosure card with the server name and tool name - Retrieved resources are added to the session artefact tray for download - Resource retrievals are included in the tenant's MCP tool error rate metric and improvement signal pipeline

If the MCP Resources Service is unavailable, it behaves like any other unavailable opt-in MCP server — an error is shown in the tool call disclosure card, and the session continues without it.

Web Search Service

Overview

The **Web Search Service** is an ecosystem-level MCP server that provides the assistant with real-time access to web search results. It is registered as a host application MCP server like any other — the distinction is that it is a shared, ecosystem-managed service rather than a server built and operated by the host team.

Web search complements the platform's primary data access pattern (structured MCP tool calls against the host application's own data) in situations where current information is needed that the host's tools and the model's training knowledge cannot provide — recent news, current pricing, up-to-date documentation, or any query that requires information beyond the host application's data scope.

What the Web Search Service provides

| Tool       | Description                                                                                                   |
|------------|---------------------------------------------------------------------------------------------------------------|
| web_search | Executes a web search query and returns a ranked list of results: title, URL, snippet, and publication date   |
| fetch_page | Retrieves and extracts the readable text content of a specific URL from search results or user-provided links |

Results are returned as structured MCP tool output — they appear in a tool call disclosure card and are cited inline using the platform's standard source citation mechanism (superscript numerals linking to the disclosure card). The model does not present web results as its own knowledge.

How host applications integrate the Web Search Service

Host applications register the Web Search Service as an MCP server in their tenant tool registry. It is typically configured as `opt-in` — the end user enables it per session when real-time information is

needed — though hosts may configure it `always-on` if their use case depends on current information in every session:

```
{
  "id": "web-search",
  "name": "Web Search",
  "description": "Searches the web for current information. Use when the user needs up-to-date facts, recent news, current documentation, or any information beyond the host application's data and the model's training knowledge. Always cite the source URL.",
  "endpoint": "https://search.mcp-ecosystem.io/mcp",
  "authType": "api-key",
  "accessTier": "opt-in",
  "roles": []
}
```

The Web Search Service endpoint and API key are available from the MCP Repository. The suggested `description` field text above is optimised for system prompt injection and is published in the Repository record.

## Transparency and citation

Web search results follow the platform's mandatory tool call transparency rules — every search invocation and every page fetch appears as a collapsible disclosure card in the conversation thread. The model is instructed (via the platform-managed system prompt layer) to:

- Cite all web-sourced claims with inline source citations linking to the disclosure card
- Acknowledge when search results are recent vs. when the model is supplementing with training knowledge
- Not present search result content as the model's own knowledge

## Relationship to the platform

The Web Search Service is independent of the AI Chat Platform. When unavailable, it behaves like any other unavailable MCP server — an error in the disclosure card and the session continues without it. Web results rendered as prose are subject to the platform's standard content rendering rules; results rendered as structured data (tables, lists) use the platform's data table rendering.



## 18 — Entry Points and Embedding Modes

The AI Chat Platform is delivered through three distinct embedding modes, each with its own component, visual presentation, and conversation model. Host applications may deploy one, two, or all three modes simultaneously — they are independent components that share the same platform backend.

### Mode overview

| Mode                  | Component                           | Conversation model                       | Persistence             | Primary use                                                       |
|-----------------------|-------------------------------------|------------------------------------------|-------------------------|-------------------------------------------------------------------|
| 1 — Floating Widget   | <code>&lt;ai-chat-widget&gt;</code> | Full — shared with Mode 2                | Persistent across pages | Always-available assistant access from anywhere in the host app   |
| 2 — Inline Page       | <code>&lt;ai-chat&gt;</code>        | Full — shared with Mode 1                | Persistent              | Dedicated assistant page or embedded content section              |
| 3 — Form Field Assist | <code>&lt;ai-chat-field&gt;</code>  | Ephemeral — scoped to the field instance | Discarded on close      | Contextual help and content generation for individual form fields |

Modes 1 and 2 are **persistent conversation modes** — they share the same conversation history for a given user and tenant. A conversation started in the floating widget appears in the history panel of the inline page view, and vice versa. Mode 3 is entirely **ephemeral and transactional** — it does not create persistent conversation threads and does not appear in any history view.

### Mode 1 — Floating Widget

#### Overview

The floating widget provides always-available assistant access without requiring the user to navigate away from their current page. It is anchored to a corner of the viewport and overlays the host application's content. It has three states: **collapsed** (FAB only), **mini** (compact chat panel), and **full** (complete conversation interface).

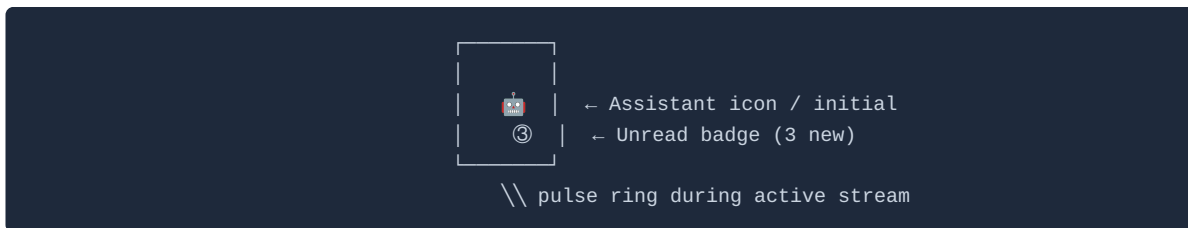
#### States

##### Collapsed state — the FAB

The widget renders as a **floating action button** (FAB) in the configured corner of the viewport (default: bottom-right). The FAB carries:

- The assistant's icon or initial (from `identity.logoUrl` or the first letter of `identity.assistantName`)
- An unread **badge** count when there are new messages in a shared conversation the user has not viewed

- A subtle pulse animation when the assistant is mid-stream in a minimised conversation

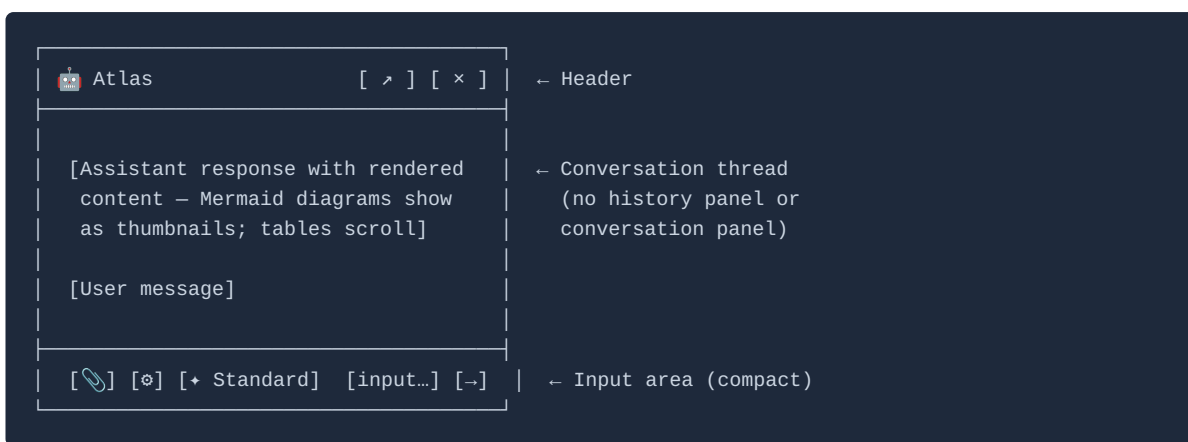


Badge is suppressed when `showBadge: false` is set in the widget config.

Clicking the FAB transitions to the mini state.

### Mini state — compact chat panel

The mini state is a compact, non-blocking chat panel anchored above the FAB. Default dimensions: **380px wide × 520px tall**, configurable via the `widget` config section. The panel is elevated above host application content with a shadow; it does not push or reflow page content.



### Mini state behaviours:

| Element               | Specification                                                                                                                                                                                                                                            |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Conversation shown    | The most recently active conversation. If no prior conversation exists, shows the onboarding welcome state.                                                                                                                                              |
| History access        | No history panel in mini state. A <b>conversations icon</b> ( $\Xi$ ) in the header opens a compact conversation list overlay within the panel — not a full sidebar.                                                                                     |
| New conversation      | Available via the conversations list overlay; "+ New" button                                                                                                                                                                                             |
| Input area            | Full input capability: text, @-binding, attachments, model chip, tool selection. Controls compact but not hidden.                                                                                                                                        |
| Content rendering     | All content types render (prose, Mermaid, Vega-Lite, tables, code). Diagrams and wide tables are constrained to the panel width and scroll horizontally. Mermaid diagrams collapse to thumbnail by default — tap/click to expand to full-screen overlay. |
| Tool call disclosures | Rendered as collapsed cards, same as full mode.                                                                                                                                                                                                          |

| Element           | Specification                                                                                  |
|-------------------|------------------------------------------------------------------------------------------------|
| Resize            | User can drag the top edge to adjust panel height within the range 360px–80vh. Width is fixed. |
| Draggable         | The panel header is draggable — user can reposition the mini panel within the viewport.        |
| Keyboard shortcut | <code>Escape</code> collapses mini state back to FAB.                                          |

**Conversation list overlay** (opened via `≡` in the mini-state header):

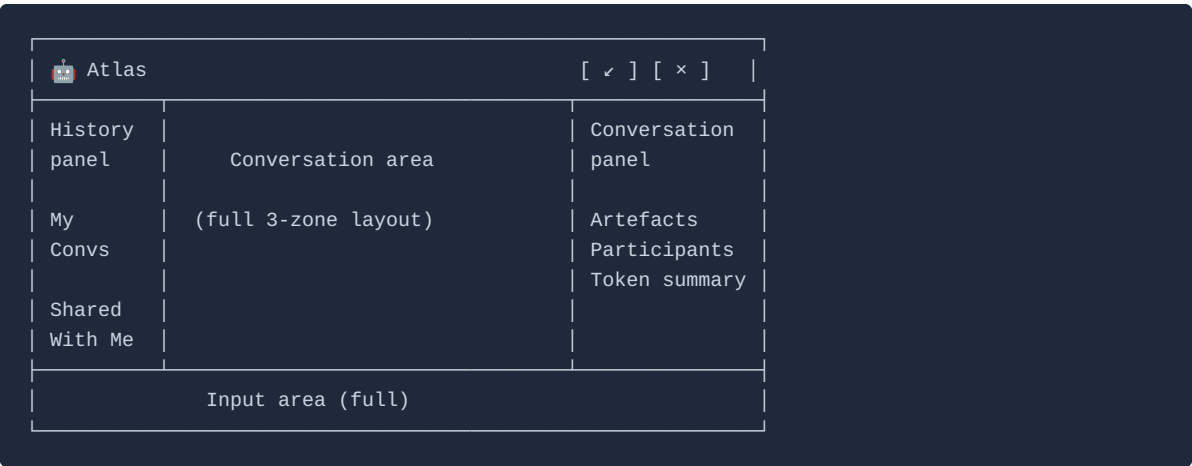


**Header controls in mini state:**

| Control       | Action                                          |
|---------------|-------------------------------------------------|
| ↗ Expand icon | Transitions to full state (see below)           |
| × Close icon  | Collapses to FAB; does not end the conversation |

### Full state — complete conversation interface

Clicking the expand icon (↗) from the mini state transitions to the full conversation interface. The full state renders as a **large modal overlay** occupying most of the viewport:



The full state renders the complete three-zone layout from `09-interaction-design.md`. The conversation active in mini state is the conversation shown on transition — no context is lost.

Full state behaviours:

| Behaviour        | Specification                                                                                          |
|------------------|--------------------------------------------------------------------------------------------------------|
| Layout           | Complete three-zone layout: history panel + conversation area + conversation panel                     |
| Modal dimensions | Default: <code>min(90vw, 1200px)</code> wide × <code>min(90vh, 860px)</code> tall, centred in viewport |
| Background       | Host page dimmed (overlay) but not blocked — user can click outside to collapse to mini state          |
| Transition       | Smooth expand animation from mini panel position to modal centre                                       |
| Collapse         | ✓ icon collapses back to mini state; × closes to FAB                                                   |
| Page navigation  | If the user navigates while full state is open, full state collapses to mini state automatically       |

Conversation continuity between mini and full

There is no transition cost when moving between mini and full states. The same conversation is active in both views, and they share state in real time — a response streaming in mini state continues streaming when the user expands to full state.

Floating widget across page navigation

The floating widget persists across client-side page navigation (SPA routing). It retains the current conversation and streaming state as the user moves between pages. On hard page reload, the widget remounts and restores the most recently active conversation.

Web component

```
<!-- Place once in the host application shell / layout -->
<ai-chat-widget
  tenant-id="acme-corp"
  user-token="..."
  position="bottom-right"
></ai-chat-widget>
```

Widget-specific HTML attributes

| Attribute                | Default                   | Description                                                                           |
|--------------------------|---------------------------|---------------------------------------------------------------------------------------|
| <code>position</code>    | <code>bottom-right</code> | FAB and panel position. Options: <code>bottom-right</code> , <code>bottom-left</code> |
| <code>mini-width</code>  | <code>380</code>          | Width of the mini panel in pixels (range: 320–600)                                    |
| <code>mini-height</code> | <code>520</code>          | Default height of the mini panel in pixels (range: 360–800)                           |
| <code>z-index</code>     | <code>9000</code>         | CSS z-index for the widget stack                                                      |

| Attribute             | Default | Description                                        |
|-----------------------|---------|----------------------------------------------------|
| <code>offset-x</code> | 24      | Horizontal margin from the viewport edge in pixels |
| <code>offset-y</code> | 24      | Vertical margin from the viewport edge in pixels   |

### Widget-specific events

| Event                         | Detail                                           | Description                                        |
|-------------------------------|--------------------------------------------------|----------------------------------------------------|
| <code>widget-expanded</code>  | <code>{ state: 'mini' \  'full' }</code>         | Widget transitioned from collapsed to mini or full |
| <code>widget-collapsed</code> | <code>{ previousState: 'mini' \  'full' }</code> | Widget collapsed to FAB                            |

All other events from `16-embedding-and-web-component.md` (`binding-click`, `turn-complete`, `token-expired`, etc.) fire identically from `<ai-chat-widget>`.

### Browser notifications

When `showBadge` is enabled, the widget can optionally request the **Web Notifications API** permission to surface a browser notification when a shared conversation receives a new message and the widget is collapsed. This is relevant when the user is on a different browser tab or has the host application in the background.

| Behaviour            | Specification                                                                                                                                                                                                               |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Permission request   | The widget never requests notification permission proactively. Permission is only requested when the user explicitly opts in — via a <i>"Notify me of shared conversation activity"</i> control in the conversations panel. |
| Notification payload | <i>"[Participant name] replied in [conversation title]"</i> — no message content in the notification for privacy                                                                                                            |
| Click action         | Clicking the notification opens the host application and expands the widget to the mini state, navigating to the relevant conversation                                                                                      |
| Denial handling      | If the user declines or revokes permission, the badge count remains the only unread signal. No fallback prompt is shown.                                                                                                    |
| Host control         | <code>showBadge: false</code> in the widget config suppresses both badge and browser notifications entirely.                                                                                                                |

### Config

The `widget` config section in the application config controls widget-level defaults:

```
{
  "widget": {
    "defaultState":      "collapsed",
    "fabIcon":           "https://cdn.acme.com/atlas-icon.svg",
    "showBadge":         true,
    "returningUserThreshold": 4
  }
}
```

| Field                               | Default                                           | Description                                                                                                                                                                |
|-------------------------------------|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>defaultState</code>           | <code>"collapsed"</code>                          | State on first page load: <code>"collapsed"</code> or <code>"mini"</code>                                                                                                  |
| <code>fabIcon</code>                | Assistant logo from <code>branding.logoUrl</code> | Custom icon for the FAB. 40×40px SVG or PNG.                                                                                                                               |
| <code>showBadge</code>              | <code>true</code>                                 | Show unread badge count on FAB for shared conversation activity                                                                                                            |
| <code>returningUserThreshold</code> | <code>4</code>                                    | Hours of inactivity after which the widget opens to the returning-user card instead of resuming mid-conversation. Set to <code>0</code> to always resume mid-conversation. |

## Mode 2 — Inline Page

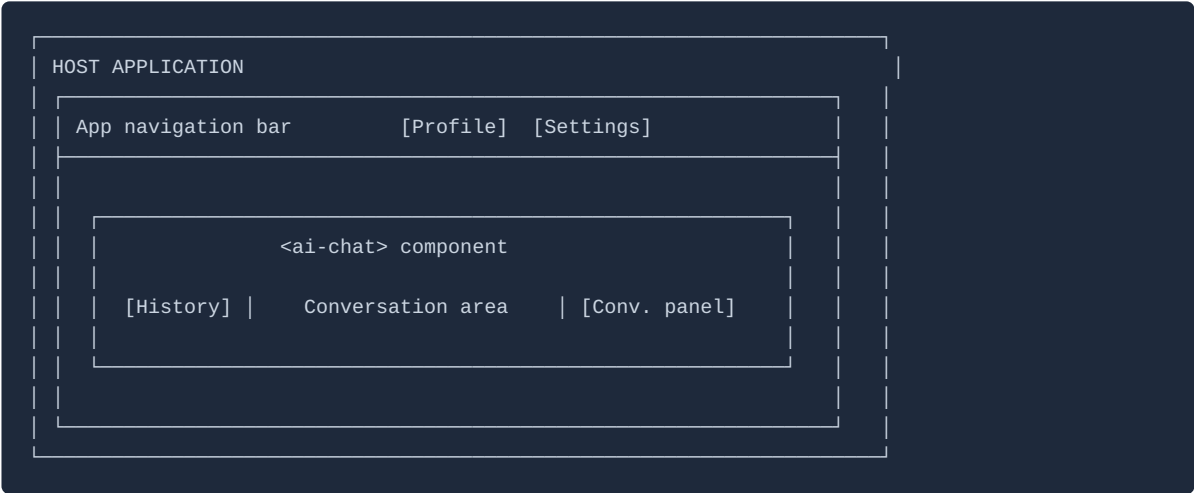
### Overview

The inline page mode embeds the assistant as a first-class content block within a host application page. It uses the `<ai-chat>` component described in full in [16-embedding-and-web-component.md](#). It is the **richest and most capable** embedding mode — the full three-zone layout with no constraints on features or content rendering.

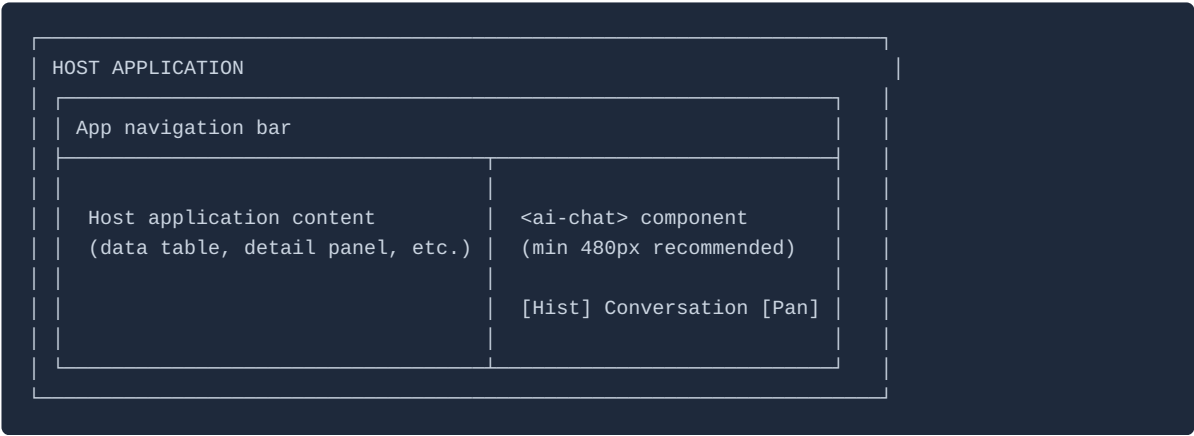
### Mode 2 in host application context

The `<ai-chat>` component occupies its mounting point only — the host application retains full ownership of its navigation, header, and page shell.

### Dedicated assistant page (full content area):



Split-panel page (assistant alongside host content):



Conversation model

Mode 2 shares conversation history fully with Mode 1. The same history panel, the same conversation list, the same threads — a user moving between a dedicated assistant page (Mode 2) and the floating widget (Mode 1) sees the same conversations in both. There is no concept of "page-scoped" conversations in Mode 2.

Typical placements

| Placement                | Description                                                                                                                                                                                  |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dedicated assistant page | A route in the host application dedicated to the assistant (e.g. <code>/assistant</code> ). The <code>&lt;ai-chat&gt;</code> component fills the main content area. Most capable experience. |
| Split-panel page         | The host application page is split: content on the left, assistant on the right. The component is mounted in a fixed-width right panel (minimum 480px recommended).                          |
| Tab within a page        | The assistant occupies one tab of a tabbed page. The component mounts/unmounts on tab switch — component state is restored on remount via the platform API.                                  |

## Relationship to Mode 1

A host application that deploys both `<ai-chat>` and `<ai-chat-widget>` on the same page will have two simultaneous conversation views. This is an unusual configuration but is supported. Both components connect to the same platform backend with the same `tenant_id` and `user_id` — changes in one are reflected in the other in real time.

For most host applications, Mode 1 (floating widget) is deployed site-wide in the layout shell, and Mode 2 (inline) is deployed on a dedicated assistant page. When the user navigates to the dedicated page, the floating widget collapses automatically (the host application controls this by calling `widget.collapse()` on the `<ai-chat-widget>` element).

---

## Mode 3 — Form Field Assist

### Overview

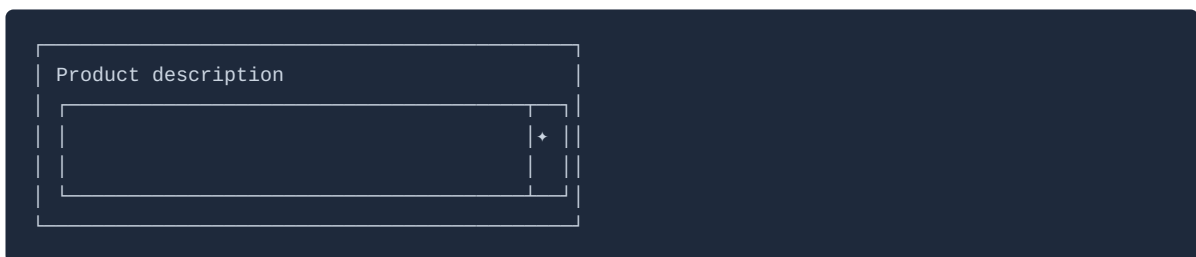
The Form Field Assist provides contextual, ephemeral AI assistance scoped to an individual form field. It appears as a small icon or button adjacent to the field and opens a **compact contextual popover** when activated. The interaction is entirely transactional — there is no conversation history, no history panel, and no persistent thread. When the popover closes, the session is discarded.

Form Field Assist is designed for tasks like: - "Write a product description for this item" - "Suggest a title based on these bullet points" - "Explain what this field is asking for" - "Translate this text into French" - "Summarise this paragraph more concisely"

### Trigger presentation

The trigger can be presented in two ways, depending on the host application's UI:

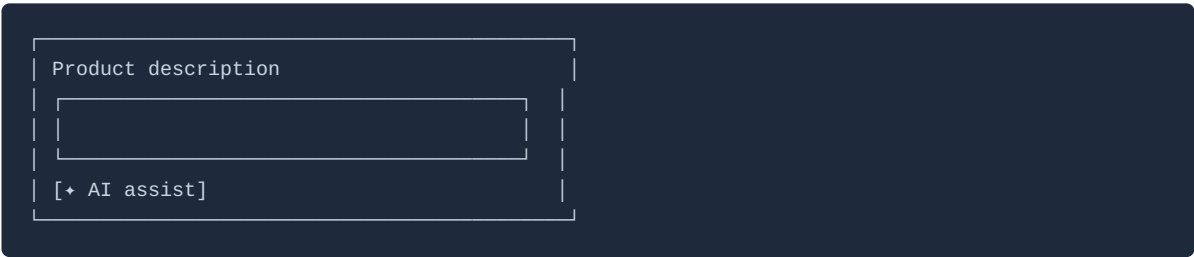
#### Inline icon trigger (inside the field)



A small `◆` (or assistant icon) appears inside the field at the trailing edge. It does not overlay field content — the field's right padding is adjusted by the component to accommodate it.

#### External button trigger (beside or below the field)

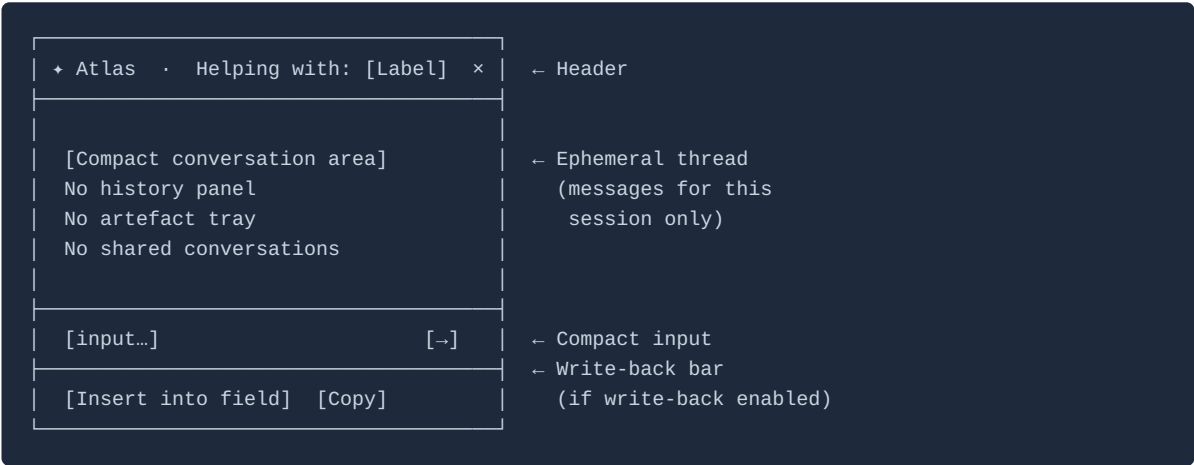




A button or link appears below or beside the field. The host application styles the button; the `<ai-chat-field>` component fires a `field-trigger-click` event the host handles. This variant gives the host full control over the trigger's visual appearance.

The trigger variant is configured via the `trigger` attribute on the component (see Web component below).

Popover layout



Default popover dimensions: **320px wide × flexible height** (grows with conversation, max 480px, then scrolls). The popover anchors to the trigger element and repositions automatically to stay within the viewport.

Ephemeral session model

| Property                 | Specification                                                               |
|--------------------------|-----------------------------------------------------------------------------|
| Conversation persistence | None — the session is discarded when the popover closes                     |
| History access           | No history panel; no access to past conversations                           |
| Shared conversations     | Not available                                                               |
| Model switching          | Not available — uses the tenant's <code>defaultModel</code>                 |
| Workflow Library         | Not available                                                               |
| Artefact tray            | Not available (individual outputs can be inserted or copied)                |
| @-binding typeahead      | Available — useful for referencing application objects in the field context |

| Property            | Specification                                                                                                 |
|---------------------|---------------------------------------------------------------------------------------------------------------|
| MCP tools           | Always-on tools are available by default; can be disabled per-field via <code>tools-disabled</code> attribute |
| Personal memory     | Not injected by default; optionally enabled via <code>inject-personal-memory</code> attribute                 |
| Application context | Always injected — org terminology and standing context is relevant to field composition                       |
| Improvement signals | Captured and attributed to the tenant; no turn-level audit trail (ephemeral session not persisted)            |

### Context injection

When the popover opens, the platform assembles a field-specific system prompt from:

1. **Tenant base prompt** — from `scope.systemPrompt` in the application config
2. **Application context** — `[Application context]` block, always injected
3. **Field context block** — assembled from the component attributes:

```
[Field context]
Field: {{field-label}}
Current value: {{field-value}} (if provided and non-empty)
Form context: {{field-context}} (if provided)
Task: Help the user compose or refine the content for this field.
```

1. **Platform-managed instructions** — write-back guidance (if enabled), tool transparency, injection mitigation

The field context block keeps the model tightly focused on the field's purpose. The model does not reference other conversations or memory items not explicitly injected.

### Write-back behaviour

When `write-back` is configured for a field, a **write-back bar** appears at the bottom of the popover after each assistant response that contains renderable text content.

| Control                  | Behaviour                                                                                                                               |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Insert into field</b> | Fires the <code>field-insert</code> event with the response content. The host application handles this event to update the field value. |
| <b>Copy</b>              | Copies the response text to the clipboard. No host event required.                                                                      |

For multi-turn field sessions, the write-back bar always shows the most recent response. Earlier responses can be copied by selecting text directly in the popover.

Write-back format is configurable per field:

| Format                       | Use case                               |
|------------------------------|----------------------------------------|
| <code>plain</code> (default) | Standard text fields, input fields     |
| <code>markdown</code>        | Rich text editors that accept markdown |
| <code>html</code>            | WYSIWYG editors                        |
| <code>json</code>            | Structured data fields                 |

## Web component

Two component variants are provided:

### Variant A — Wrapper (component wraps the field element)

```
<ai-chat-field
  tenant-id="acme-corp"
  user-token="..."
  field-label="Product Description"
  field-context="Help the user write a compelling product description for an e-commerce
listing. Focus on benefits, not features."
  write-back="true"
  write-back-format="plain"
  trigger="inline"
>
  <textarea name="description" rows="4"></textarea>
</ai-chat-field>
```

### Variant B — Standalone trigger (component placed near the field)

```
<textarea id="desc-field" name="description" rows="4"></textarea>
<ai-chat-field
  tenant-id="acme-corp"
  user-token="..."
  field-label="Product Description"
  field-context="Help the user write a compelling product description."
  field-target="#desc-field"
  write-back="true"
  trigger="button"
  trigger-label="AI assist"
></ai-chat-field>
```

#### `<ai-chat-field>` attributes

| Attribute                  | Required | Description                                                                                                                      |
|----------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------|
| <code>tenant-id</code>     | Yes      | Tenant identifier                                                                                                                |
| <code>user-token</code>    | Yes      | Host-authenticated user JWT                                                                                                      |
| <code>field-label</code>   | Yes      | The field's label — shown in the popover header and injected as field context                                                    |
| <code>field-context</code> | No       | A plain-language description of what this field is for and how the assistant should help. Injected into the field context block. |

| Attribute                           | Required | Description                                                                                                                                                                                             |
|-------------------------------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>field-value</code>            | No       | The field's current value, passed by the host for editing assistance. Injected into the field context block. If omitted, the component reads the value from the wrapped field element (Variant A only). |
| <code>write-back</code>             | No       | <code>"true"</code> enables the write-back bar. Default: <code>"false"</code> .                                                                                                                         |
| <code>write-back-format</code>      | No       | <code>"plain"</code>   <code>"markdown"</code>   <code>"html"</code>   <code>"json"</code> . Default: <code>"plain"</code> .                                                                            |
| <code>trigger</code>                | No       | <code>"inline"</code> — icon inside the field; <code>"button"</code> — external button. Default: <code>"inline"</code> .                                                                                |
| <code>trigger-label</code>          | No       | Button label text when <code>trigger="button"</code> . Default: <code>"AI assist"</code> .                                                                                                              |
| <code>field-target</code>           | No       | CSS selector for the target field element (Variant B).                                                                                                                                                  |
| <code>tools-disabled</code>         | No       | <code>"true"</code> disables always-on MCP tools for this field session. Useful for simple text-generation fields that don't need live data. Default: <code>"false"</code> .                            |
| <code>inject-personal-memory</code> | No       | <code>"true"</code> injects the user's personal memory into the field session. Useful for fields where personal preferences or role context are relevant. Default: <code>"false"</code> .               |
| <code>placeholder-prompt</code>     | No       | Pre-filled text shown in the input field on first open (e.g. <code>"Describe this product in 2-3 sentences"</code> ). Helps orient the user.                                                            |

**<ai-chat-field> events**

| Event                      | Detail payload                               | Description                                                                                           |
|----------------------------|----------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>field-insert</code>  | <code>{ fieldLabel, content, format }</code> | Fired when user clicks "Insert into field". Host updates the field value using <code>content</code> . |
| <code>field-copy</code>    | <code>{ fieldLabel, content }</code>         | Fired when user clicks "Copy"                                                                         |
| <code>field-opened</code>  | <code>{ fieldLabel }</code>                  | Fired when the popover opens                                                                          |
| <code>field-closed</code>  | <code>{ fieldLabel, messageCount }</code>    | Fired when the popover closes                                                                         |
| <code>token-expired</code> | —                                            | Identical to other component variants — host refreshes via <code>updateToken()</code>                 |

**<ai-chat-field> JavaScript API**

| Method                          | Description                                     |
|---------------------------------|-------------------------------------------------|
| <code>open()</code>             | Programmatically open the popover               |
| <code>close()</code>            | Programmatically close and discard the session  |
| <code>updateToken(token)</code> | Refresh the user JWT — same as other components |

| Method                            | Description                                                               |
|-----------------------------------|---------------------------------------------------------------------------|
| <code>setFieldValue(value)</code> | Update the <code>field-value</code> context without reopening the popover |

Handling `field-insert`

```
document.querySelectorAll('ai-chat-field').forEach((field) => {
  field.addEventListener('field-insert', (event) => {
    const { fieldLabel, content, format } = event.detail;
    const textarea = document.querySelector(`[data-label="${fieldLabel}"]`);
    if (textarea) {
      textarea.value = content;
      textarea.dispatchEvent(new Event('input', { bubbles: true }));
    }
  });
});
```

Cross-mode design decisions

| Decision                                           | Rationale                                                                                                                                                                                                                                              |
|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Modes 1 and 2 share conversation history           | Both are persistent conversation modes accessing the same user session and tenant. There is no "page-specific" conversation context in these modes — the user's conversations belong to them, not to a page.                                           |
| Mode 3 is entirely ephemeral                       | Form field interactions are transactional. The user wants help completing a specific field right now. Persisting these as conversations would pollute the history with low-value threads and create privacy/audit implications for draft field values. |
| Mode 3 does not escalate to Modes 1 or 2           | The form field context is scoped to a specific field instance at a specific moment. It would not be useful as a persistent thread — the field context would be stale, and the user's goal (filling the field) would already be complete.               |
| Application context is injected in Mode 3          | Org terminology, standing guidance, and governance decisions are just as relevant when composing field content as in a full conversation. Omitting them would cause inconsistent terminology in AI-generated field values.                             |
| Personal memory is opt-in in Mode 3                | Personal memory may contain preferences relevant to some fields (e.g. writing style, role context for approvals). But for most fields it adds unnecessary token overhead. The host decides per field.                                                  |
| Always-on MCP tools available in Mode 3 by default | Some fields benefit from live data access (e.g. a "domain owner" field might benefit from the governance MCP). Host can disable per field if not needed.                                                                                               |
| Write-back requires host event handling            | The platform does not manipulate the DOM directly — it fires events and the host application updates its own fields. This keeps the component framework-agnostic and respects the host's form state management.                                        |

## Summary: which component to use

| Scenario                                                             | Component                                                                        |
|----------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Provide assistant access from anywhere in the app without navigation | <code>&lt;ai-chat-widget&gt;</code> (Mode 1)                                     |
| Build a dedicated assistant page or embed a full assistant panel     | <code>&lt;ai-chat&gt;</code> (Mode 2)                                            |
| Add contextual AI assistance to individual form fields               | <code>&lt;ai-chat-field&gt;</code> (Mode 3)                                      |
| All three simultaneously                                             | All three components can coexist — they share the same auth and platform backend |

# AI Chat Platform — Roadmap

This file captures planned enhancements beyond the current release. Items here are named and scoped at a level sufficient for future prioritisation. Each requires a Product Design Document and platform team approval before build begins.

|                        |                                                                              |
|------------------------|------------------------------------------------------------------------------|
| <b>Current release</b> | v1 — multi-tenant embed, host config, MCP tool integration, full audit trail |
| <b>Date</b>            | May 2026                                                                     |

## Near-term — Semantic search (RAG)

**Objective:** Allow the assistant to retrieve semantically relevant content from host application data using vector search, as a complement to structured MCP tool calls.

| Item                       | Description                                                                                                                                                                                                                    |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Per-tenant vector search   | Platform provides a vector index per tenant. Host applications push embeddings via the Platform Admin API. The assistant can retrieve semantically similar content when structured MCP tool calls return insufficient results. |
| Conversation memory RAG    | Embed past conversation content per tenant. The assistant retrieves semantically relevant past turns at the start of a new session, surfaced as an optional <code>[Related past conversations]</code> context block.           |
| Embedding refresh pipeline | Host applications trigger re-embedding via webhook when their source data changes. The platform maintains freshness indicators per embedding.                                                                                  |

**Pre-requisites:** pgvector infrastructure provisioned in platform storage; Platform Admin API embedding endpoints designed; per-tenant embedding budget model defined.

## Near-term — Improvement webhook

**Objective:** Allow host applications to receive improvement signals in their own tooling rather than relying solely on the platform's improvement dashboard.

| Item                 | Description                                                                                                                                                                     |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Webhook registration | Host teams register a webhook endpoint via the Platform Admin API. The platform sends structured JSON payloads for qualifying improvement signals.                              |
| Payload schema       | Signal type, turn summary, remediation recommendation, tenant context. Full turn content available on request via the Platform Admin API (not in webhook payload for security). |

| Item                          | Description                                                                                                                   |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Retry and delivery guarantees | Platform retries failed webhook deliveries with exponential backoff. Delivery status tracked in the platform admin dashboard. |

## Phase 2 — Context memory

**Objective:** Allow the assistant to recall relevant context from past conversations at the start of a new session, without the user having to re-establish it manually.

| Item                      | Description                                                                                                                                                                                                                                                      |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Session summary injection | At session start, optionally inject a compressed summary of the user's recent conversations (last 30 days) as a labelled <code>[Recent context]</code> block. Opt-in per session. Transparent — block visible and dismissible. Generated by the fast-tier model. |
| Context globbing          | User-initiated: pull selected context from one or more past conversations into a new session. Requires a conversation selection UI, a context budget allocator, and provenance labelling.                                                                        |

**Pre-requisites:** Auto-summarisation infrastructure (already in v1); per-tenant conversation memory RAG (above).

## Phase 2 — Extended config capabilities

**Objective:** Give host applications more control over conversation behaviour and user experience.

| Item                       | Description                                                                                                                                                                                                                                                             |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Conversation folders       | Host applications can enable user-created conversation folders/projects (grouping by topic, project, or review cycle). User-created and personal — not shared. Requires history panel redesign.                                                                         |
| Real-time presence         | Typing indicators and active-viewer presence for shared conversations. Requires WebSocket infrastructure upgrade.                                                                                                                                                       |
| Extended thinking mode     | Expose the powerful-tier model's extended reasoning budget as an optional session setting (where supported by the configured provider). Displayed as a collapsible <code>&lt;thinking&gt;</code> block in the thread. Host configures whether this option is available. |
| Participant limit increase | Raise the shared conversation participant cap above 10. Requires load and permission testing at higher counts. Platform-level config change.                                                                                                                            |
| Conversation export        | Export a conversation as PDF or markdown. Requires host applications to declare their data classification handling before enabling. Off by default.                                                                                                                     |



## Phase 2 — Analytics and operational tooling

**Objective:** Give Application Admins richer insight into their tenant's assistant usage and quality.

| Item                          | Description                                                                                                                                                                            |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enhanced analytics dashboard  | Expand the tenant analytics dashboard with: query topic clustering, peak usage patterns, workflow completion rates, binding click-through by type, improvement signal resolution time. |
| Signal dashboard              | Dedicated improvement signal view: volume by type, confidence distribution, issue lifecycle status, resolution time by remediation target.                                             |
| Application config change log | Auditable log of all config changes within the tenant, with before/after diff view and the Application Admin who made the change.                                                      |
| MCP server health panel       | Per-server uptime, error rate, and latency trends over time, visible to Application Admins within the tenant analytics dashboard.                                                      |

## Phase 2 — Multi-provider AI models

**Objective:** Allow host applications to configure alternative AI providers as the model backend.

| Item                       | Description                                                                                                                                                                              |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Provider abstraction layer | Formalise provider configuration in the application config. Test OpenAI and Gemini models against the platform's system prompt injection, MCP tool call patterns, and content rendering. |
| Model capability parity    | Validate that alternative models correctly invoke MCP tools, respect system prompt constraints, and produce rendering-compatible outputs (Mermaid, Vega-Lite, math).                     |
| Provider selection UI      | Admin-level provider selection per tenant. End users may be able to select provider per session subject to host policy.                                                                  |

## Phase 2 — SDK and programmatic config

**Objective:** Give host engineering teams a first-class programmatic interface for managing their tenant configuration.

| Item                    | Description                                                                                                                                                                  |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Platform Admin SDK      | TypeScript/Python SDK wrapping the Platform Admin API — typed config objects, config diff preview, validation helpers, rollback support.                                     |
| Config-as-code          | CI/CD integration that treats the application config as a versioned artefact. Config changes trigger validation and deployment via the SDK rather than the Config Editor UI. |
| Config diff and preview | Before applying a config update, show a structured diff of what changes and a preview of how the system prompt will assemble.                                                |

---

## Phase 3 — MCP ecosystem expansion

**Objective:** Grow the MCP ecosystem to accelerate host team onboarding.

| Item                            | Description                                                                                                                                                                                   |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MCP Repository growth           | Increase the number of published, verified MCP servers in the Repository across more domains (HR, finance, engineering, legal, customer success).                                             |
| MCP Resources Service expansion | Add more skills, guidance documents, and prompt templates to the MCP Resources Service, with category-level browsing and search.                                                              |
| MCP server scaffolding tool     | CLI tool that generates a boilerplate MCP server codebase for a given set of capabilities, reducing the time for host teams to publish their own server to the MCP Repository.                |
| Cross-tenant MCP benchmarking   | Anonymised, aggregated MCP tool error rate and latency benchmarks across tenants using the same server — helps host teams identify whether their server performance is typical or an outlier. |

---

## Not in roadmap

| Item                                | Rationale                                                                                |
|-------------------------------------|------------------------------------------------------------------------------------------|
| Voice or multimodal input           | Not aligned with current platform direction                                              |
| General-purpose AI assistant mode   | Outside product vision — every deployment is a specialist scoped by the host application |
| Public shareable conversation links | Conflicts with tenant-scoped privacy model                                               |
| Cross-tenant sharing                | Data confidentiality constraint — tenant boundary is non-configurable                    |
| AI model training or fine-tuning    | Platform uses foundation AI provider models; no training infrastructure                  |

For current product specification, see [README.md](#) and the numbered specification documents.